408 详解 · 2013 年选择题 · 升级版

本册每题先给出原卷题干与选项全文（转录自公开渠道真题，来源与核对见《2013-升级日志》），再按 6 节讲解；讲解重心放在方法、推理、易混点上。

升级版说明：每题标题下新增"**题干与选项：**"节（原卷题干与选项全文）；6 节之后新增 7 类增补小节（【图像】【定位】【易混对比】【坑位】【套路】【类比】【口诀】，无适用内容的小节省略）；重点句按全卷统一配色：红＝关键结论、蓝＝定位与套路关键动作、橙＝坑位警告、绿＝口诀；【口诀】附考频标注（该知识点在 2009–2022 年解析中的出现次数＋高/中/低频档位，高频红标）。

第 1 题 · 两个有序链表归并的时间复杂度（正确答案：D）

题干与选项：

已知两个长度分别为 m 和 n 的升序链表，若将它们合并为一个长度为 m+n 的降序链表，则 最坏情况下的时间复杂度是

- A. O(n)
- B. O(m ⊠ n)
- C. O(min(m, n))
- D. O(max(m, n))

入手点： 归并就是“两个队头比大小，赢的先出队”，每比一次就定一个元素的位置；数一数最坏要比几次，量级就出来了。

专有名词从零解释：

- 先想“有序链表”：结点按大小排好的链，下一个结点总比当前结点大（或者总不比它小）。
- 再想“归并”：把两个有序链表合成一个有序链表，办法是反复比较两个链表的队头，把小的那个摘下来接到新链表尾巴上。
- 再想“时间复杂度”：衡量工作量随问题规模怎么涨的粗略尺子，只看涨势，不抠常数。
- 再想“渐近等价”：两个量级之间只差一个常数倍（比如一个是另一个的 2 倍以内），在大 O 的眼里就算同一个量级。

考查知识点：

- 先想归并的每一步：两个队头比一次，小的那个被摘下来，位置就定了；摘完之后，两个队头又换了新元素，再比一次。
- 再想最坏比几次：两个链表一共 m＋n 个元素，每比一次定一个元素，最后一个元素不用比（剩下的直接接上去），所以最坏比 m＋n－1 次，量级是 O(m＋n)。
- 再想 O(m＋n) 和 O(max(m,n)) 的关系：max(m,n) ⩽ m＋n ⩽ 2×max(m,n)，两者只差不到 2 倍这个常数，所以渐近等价；解析写 O(m＋n)＝O(max(m,n))，就是这个意思，答案 D。
- 拿生活例子验算：两摞按大小排好的牌，每次从两摞顶上各看一张，把小的那张放到新摞上；牌一共 m＋n 张，最多比 m＋n－1 次就全排好了。
- 边界情况：如果一个链表是空的，比 0 次，另一个链表直接接上去；如果两表一样长（m＝n），量级就是 O(n)。
- 易混对子：不要把归并当成双层循环——双层循环是每个元素都要和另一个表的每个元素比一次，那是乘积；归并每步只比一次队头，是加法。不要把“短表比完了”当成“结束了”——短表比完后，长表剩下的尾巴还要接上去，也要算。

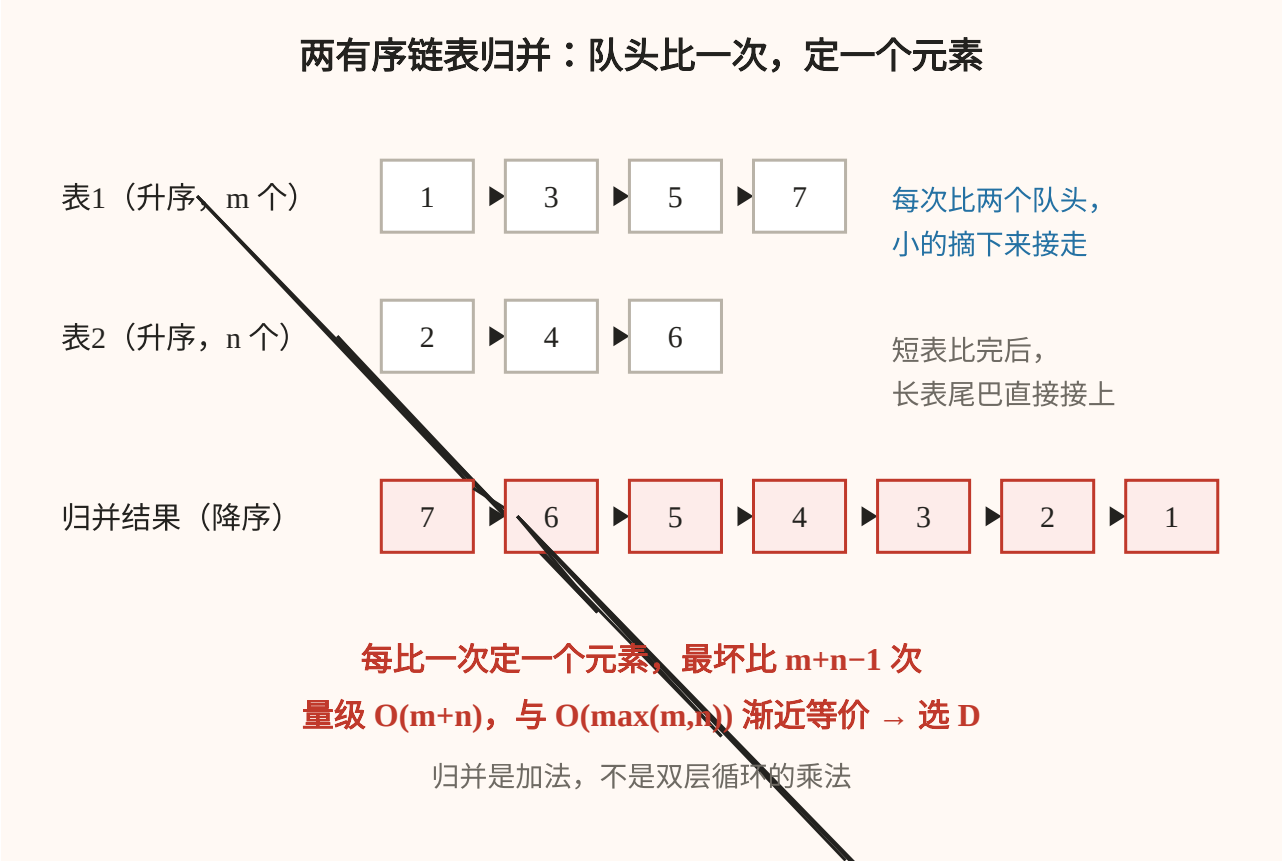
正确选项为什么对： 最坏比 m＋n－1 次，量级 O(m＋n)，和 O(max(m,n)) 渐近等价，就是 D，选 D。

错误选项为什么错＋举例：

- A（O(n)）：只用一个表的规模 n 来概括，丢了另一个表。教训：归并涉及两个表，量级里两个规模都要出现。
- B（O(mn)）：把归并当成了双层循环相乘。教训：归并每步只比一次队头，不是每对元素都比一遍，乘积是错的。
- C（O(min(m,n))）：只算到短表比完为止。教训：短表比完后，长表剩下的元素还要接上去，不能不算。

笔记本一句话： 第 1 题＋归并量级：比一次定一个，最坏 m＋n－1 次，O(m＋n)＝O(max(m,n))。

【图像】



两有序链表队头比较、每比一次定一个元素的归并示意图

【定位】 数据结构 · 第2章 线性表 · 归并的时间复杂度

【易混对比】

- "O(m+n)"和"O(max(m,n))"的区别：max(m,n) ⩽ m+n ⩽ 2×max(m,n)，两者只差不到 2 倍的常数，在大 O 眼里是同一个量级；选项 D 写的就是 O(max(m,n))。
- "归并"和"双层循环"的区别：归并每步只比较一次队头、比一次定一个元素，量级是加法；双层循环是每个元素都和另一表的每个元素比一遍，量级才是乘法。

最坏比较 m+n−1 次，量级 O(m+n)，与 O(max(m,n)) 渐近等价，选 D。

【坑位】

看到两个表先想"比一次定一个"，别顺手写成乘积 O(mn)。

把归并当成双层循环，会乘出 O(mn) 误选 B；只数到短表比完、漏掉长表剩下的尾巴拼接，会写出 O(min(m,n)) 误选 C。正确做法：两个队头比一次、小的那个位置就定了，一共 m+n 个元素，最坏比 m+n−1 次（最后一个不用比、直接接上），量级是 O(m+n)。

【套路】

- 第一步，数比较次数：每比一次定一个元素，m+n 个元素最坏比 m+n−1 次。
- 第二步，写成大 O：m+n−1 的量级就是 O(m+n)。
- 第三步，对上选项的写法：O(m+n) 与 O(max(m,n)) 渐近等价，对应选项 D。

比一次定一个，次数写成量级，再和选项对口径。

【类比】

像两摞按大小排好的牌：每次从两摞顶上各翻一张，把小的那张放到新摞上；牌一共 m+n 张，最多翻 m+n−1 次就全排好了。术语贴回去：翻一次牌就是"比较一次队头"，新摞就是"归并后的链表"，总次数就是"最坏比较次数"。

【口诀】 队头比一次，定住一个数；最坏 m 加 n 减一，max 等价选 D。

【考频】 「时间复杂度」在 2009–2022 年解析中出现 20 次 · 高频

第 2 题 · 栈出栈序列第 3 个位置可取值的个数（正确答案：C）

题干与选项：

一个栈的入栈序列为 1, 2, 3, ,n ，其出栈序列是 p1 , p2 , p3 , , pn 。若 p2 ☒ 3 ，则 p3 可能取值 的个数是

- A. n ☒ 3
- B. n ☒ 2
- C. n ☒ 1
- D. 无法确定

入手点： 先定第 1 个能是谁，再分情况看第 3 个；最后把各种情况能取到的数并起来数个数。

专有名词从零解释：

- 先想“栈”：一摞盘子，只能从顶上放、只能从顶上拿，后放进去的先拿出来，叫后进先出。
- 再想“入栈序列、出栈序列”：入栈序列是数字进栈的顺序（本题是 1、2、3、……、n），出栈序列是数字出栈的顺序（本题记成 p1、p2、p3、……）。
- 再想“第 2 个是 3”这个条件：3 是第 2 个出来的，说明 3 出来之前，只有 1 个数字先出来了。

考查知识点：

- 先想 p1 能是谁：3 出来之前，1、2 已经进了栈（3 是第 3 个进的，前面 1、2 必须先进）；第 1 个出来的只能是 1 或者 2，不可能是 3（3 是第 2 个出来的），也不可能是 4 及以后的数（它们还没进栈）。
- 再想 p1＝1 的情况：1 先出来，栈里剩 2；接着 3 进来、3 出来（p2＝3），栈里剩 2；这时 p3 可以是 2（把栈里的 2 拿出来），也可以是 4、5、……、n（新数字进栈后马上出来）。
- 再想 p1＝2 的情况：1 进、2 进、2 先出来，栈里剩 1；接着 3 进来、3 出来（p2＝3），栈里剩 1；这时 p3 可以是 1，也可以是 4、5、……、n。
- 再想并起来：两种情况合起来，p3 能取 1、2、4、5、……、n，就是除了 3 以外的所有数，一共 n－1 个，就是 C。
- 拿小例子验算：n＝5 时，p3 能取 1、2、4、5，共 4 个＝n－1；3 已经是 p2 了，不能再当 p3。
- 边界情况：n＝3 时，p3 能取 1、2，共 2 个＝n－1，公式照样成立。
- 易混对子：不要把 3 也算进 p3 的候选——3 已经是 p2 了，一个数字不能在出栈序列里出现两次；不要只算 4 到 n——1、2 里那个没当 p1 的，也能当 p3。

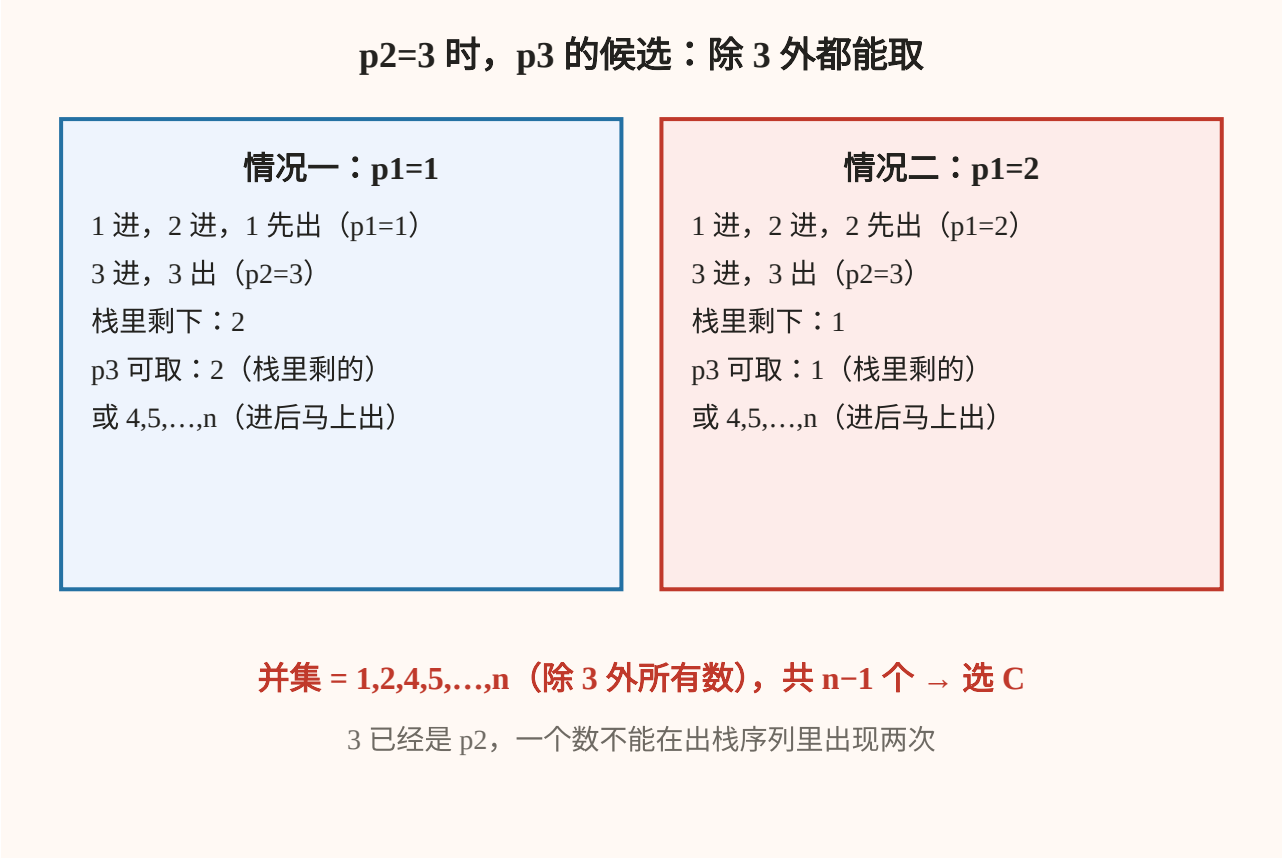
正确选项为什么对： p3 能取除 3 外的所有数，共 n－1 个，就是 C，选 C。

错误选项为什么错＋举例：

- 个数算成 n 的：把 3 也算进了 p3 的候选。教训：3 已经是 p2 了，不能重复出现。
- 个数算成 n－2 的：只算了 4 到 n，漏了 1、2 里那个没当 p1 的。教训：分情况要分全，两种 p1 的情况都要看。
- 个数算成 2 的：只看到 1、2 两种，漏了 4 到 n 都能“进栈后马上出来”。教训：新数字可以一直进栈、马上出栈，4 到 n 每个都能当 p3。

笔记本一句话： 第 2 题＋栈出栈序列：p2＝3 时 p1 只能是 1 或 2，p3 除 3 外都行，共 n－1 个。

【图像】



p2=3 时两种 p1 情况及 p3 候选的栈示意图

【定位】 数据结构 · 第3章 栈、队列和数组 · 出栈序列合法性

【易混对比】

- "p3 能取哪些数"和"p3 有几个"的区别：前者是中间步骤，要把两种 p1 情况的候选并起来；后者是最后一步，数一数并集里有几个数。
- "3 是 p2"和"3 不能当 p3"的区别：同一个数字在出栈序列里只能出现一次，3 已经占了 p2 的位置，p3 的候选里就没有它。
- "还没进栈的数"和"栈里剩的数"的区别：还没进栈的数（4 到 n）可以"进来马上出去"当 p3；栈里剩的数（1 或 2 里没当 p1 的那个）也可以直接出来当 p3。

p3 除了 3 都能取，共 n−1 个，选 C。

【坑位】

先定 p1 能是谁，再分情况看 p3，别直接从 4 数到 n。

只数 4 到 n 会得到 n−3，漏了 1、2 里那个没当 p1 的；把 3 也算进候选会多算一个。正确做法：p2=3 说明 3 出来之前只有 1 个数先出来，1、2 已经进栈，所以 p1 只能是 1 或 2；p1=1 时栈里剩 2，p3 可取 2 或 4 到 n；p1=2 时栈里剩 1，p3 可取 1 或 4 到 n；并起来就是除 3 外的所有数，共 n−1 个。

【套路】

- 第一步，定 p1 的候选：3 是第 2 个出来的，它出来前 1、2 已经进栈，p1 只能是 1 或 2。
- 第二步，分情况列 p3：p1=1 时栈里剩 2，p3 可取 2 或 4 到 n；p1=2 时栈里剩 1，p3 可取 1 或 4 到 n。
- 第三步，并起来数个数：两情况的并集是除 3 外的所有数，共 n−1 个。

先定 p1，再分情况，最后并起来数。

【类比】

像排队进场再出场：3 是第 2 个出来的，说明它出来时 1、2 已经进去了，所以第 1 个出来的只能是 1 或 2；第 3 个出来的，要么是还留在场内的那个（1 或 2 里剩下的），要么是后面新进来的（4 到 n，进来马上出去）。术语贴回去：进场就是"入栈"，出场就是"出栈"，留在场内的人就是"栈里的元素"。

【口诀】 p2 是三，p1 一或二；p3 除三都能取，个数 n 减一。

【考频】 「栈」在 2009–2022 年解析中出现 24 次 · 高频

第 3 题 · 用 7 个关键字建平衡二叉树、平衡因子为 0 的分支结点个数（正确答案：D）

题干与选项：

若将关键字 1，2，3，4，5，6，7 依次插入到初始为空的平衡二叉树 T 中，则 T 中平衡因 子为 0 的分支结点的个数是

- A. 0
- B. 1
- C. 2
- D. 3

入手点： 按插入顺序画树，树一不平衡就做 RR 调整；画出终态树，再一个一个数平衡因子。

专有名词从零解释：

- 先想“平衡二叉树”：一棵二叉排序树，并且每个结点的左右子树高度差不超过 1；这样树不会长成一条链，查找才快。
- 再想“平衡因子”：左子树高度减去右子树高度；平衡二叉树里每个结点的平衡因子只能是 -1、0、1。
- 再想“分支结点”：有孩子的结点；没有孩子的结点叫叶子结点，本题只数分支结点，叶子不算。
- 再想“RR 型调整”：新结点插进了右孩子的右子树，树向左歪了，就把这棵小子树向左转一下，转完恢复平衡。

考查知识点：

- 先想插入过程的关键状态：一共 7 幅关键状态：只有结点 1；1 右边接 2；1、2、3 成右斜链，对最小不平衡子树作 RR 调整，2 成为根；继续插入，2 为根时再作 RR 调整；4 为根时再作 RR 调整；最后得到终态树。
- 再想终态树：根是 4，左子树根是 2（子女是 1、3），右子树根是 5（子女是 6、7）。
- 再想每个分支结点的平衡因子：结点 2 的左右子树都只有 1 层，平衡因子是 0；结点 5 的左右子树都只有 1 层，平衡因子是 0；结点 4 的左子树 2 层、右子树 2 层，平衡因子是 0。
- 再想个数：平衡因子为 0 的分支结点是 2、4、5，一共 3 个，就是 D。
- 拿生活例子验算：天平两边一样重，平衡因子就是 0；结点 2、4、5 的左右两边高度都一样，都是“天平平了”的状态。
- 边界情况：平衡因子允许 -1、0、1，绝对值超过 1 才要调整；叶子结点的平衡因子也是 0，但本题问的是分支结点，叶子不能算进来。
- 易混对子：不要把叶子结点也算成“分支结点”——叶子没有孩子，不是分支结点；不要只数根——根的平衡因子是 0，但 2 和 5 的平衡因子也是 0。

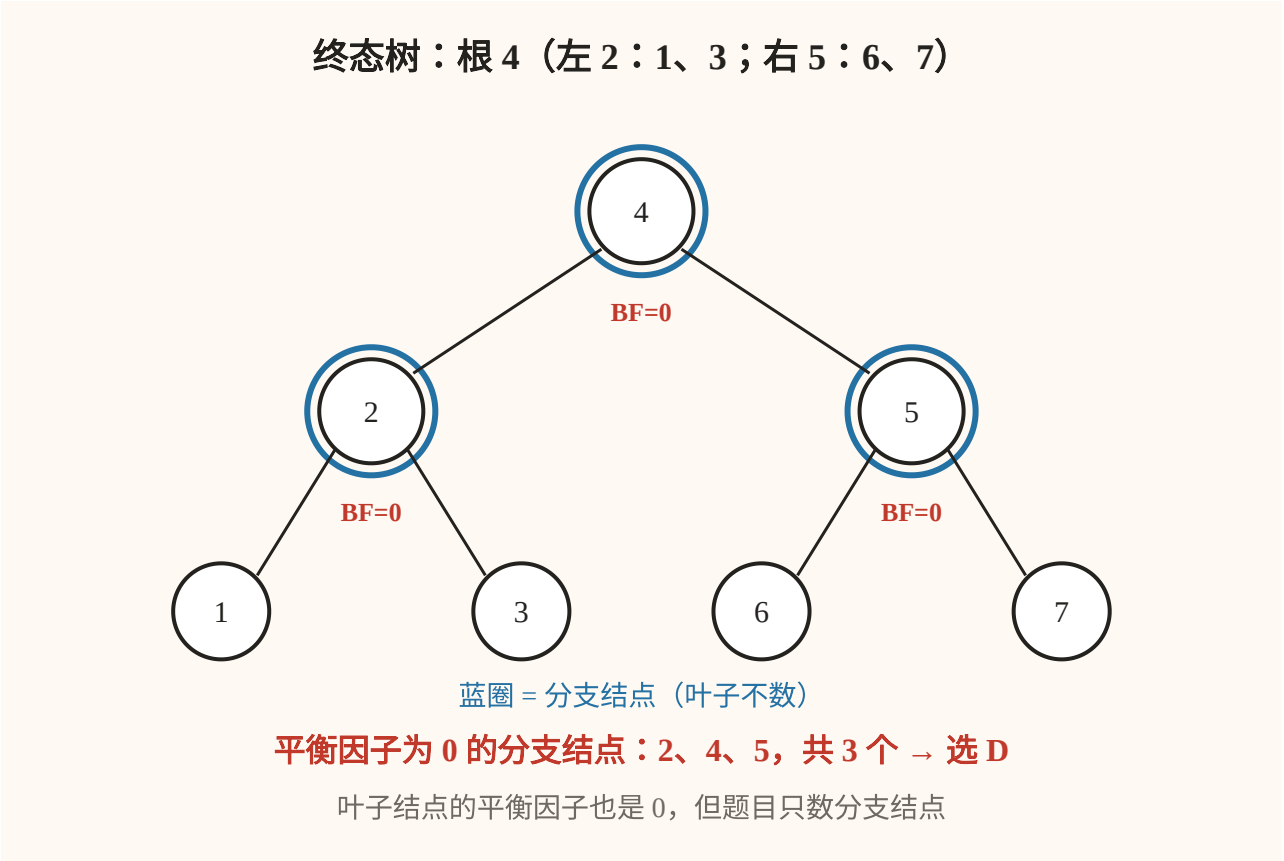
正确选项为什么对： 终态树里平衡因子为 0 的分支结点是 2、4、5，共 3 个，就是 D，选 D。

错误选项为什么错＋举例：

- 个数数少了（比如只数出 2 个）：漏了 5 或者漏了 4。教训：终态树画出来后，3 个分支结点要一个一个验平衡因子。
- 个数数多了（比如把叶子也算进去）：叶子结点的平衡因子也是 0，但题目问的是分支结点。教训：先分清“分支结点”和“叶子结点”，再数。
- 个数算错是因为终态树画错：RR 调整没做对，终态树的形状就错了，平衡因子跟着错。教训：每次不平衡都要对最小不平衡子树做调整，不能跳过。

笔记本一句话： 第 3 题＋平衡二叉树终态：根 4（左 2：1、3；右 5：6、7），平衡因子为 0 的分支结点是 2、4、5，共 3 个。

【图像】



平衡二叉树终态树及各分支结点平衡因子示意图

【定位】 数据结构 · 第6章 查找 · 平衡二叉树

【易混对比】

- "分支结点"和"叶子结点"的区别：分支结点有孩子，叶子结点没有；本题只数分支结点，平衡因子为 0 的叶子不能算进来。
- "平衡因子为 0"和"结点平衡"的区别：平衡因子为 0 是说这个结点左右等高；结点平衡只要求平衡因子是 −1、0、1，平衡因子为 1 的结点也是平衡的，但本题不数它。

终态树是根 4（左 2 带 1、3，右 5 带 6、7），平衡因子为 0 的分支结点是 2、4、5，共 3 个，选 D。

【坑位】

插入时一出现右斜链，就对最小不平衡子树做 RR 调整，一次都不能跳。

1、2、3 插完是右斜链，不做 RR 调整的话终态树形全错；数个数时把叶子也算进来会数多。正确做法：按插入顺序画树，每次不平衡都对最小不平衡子树作 RR 调整，最终得到根 4、左子树根 2（子女 1、3）、右子树根 5（子女 6、7）；逐个验平衡因子：2 左右等高为 0，5 左右等高为 0，4 左右都是 2 层为 0。

【套路】

- 第一步，按序插入画树：1、2、3 插完成右斜链，对最小不平衡子树作 RR 调整，2 成为根；继续插入，右斜链再出现就再调。
- 第二步，定终态树形：根 4，左子树根 2（子女 1、3），右子树根 5（子女 6、7）。

- 第三步，逐个验平衡因子：2、4、5 的平衡因子都是 0；叶子不数。

画树步步调，终态逐个验，叶子不算数。

【类比】

像搭积木塔：每放一块就看塔歪没歪，往右歪了就往左扶正；最后塔是根 4 压阵、左右两边一样高。术语贴回去：往左扶正就是"RR 调整"，左右两边一样高就是"平衡因子为 0"。

【口诀】 依次插入画，右斜就左转；终态根为四，二四五皆零。

【考频】 「平衡二叉树」在 2009–2022 年解析中出现 8 次 · 中频

第 4 题 · 哈夫曼思想推广到三叉树、最小带权路径长度（正确答案：B）

题干与选项：

已知三叉树 T 中 6 个叶结点的权分别是 2，3，4，5，6，7，T 的带权（外部）路径长度最 小是

- A. 27
- B. 46
- C. 54
- D. 56

入手点： 先看叶子个数能不能凑成严格三叉树，不能就补权为 0 的虚叶子；再按“每次合并最小的 3 个”构造，最后算带权路径长度。

专有名词从零解释：

- 先想“哈夫曼树”：带权路径长度最小的树；权越大的叶子离根越近，权越小的叶子离根越远，这样总代价最小。
- 再想“带权路径长度”：每个叶子的权乘以它到根的层数（根是第 0 层），再全部加起来，记成 WPL。
- 再想“严格三叉树”：每个分支结点恰好有 3 个孩子的树；一个孩子、两个孩子的分支结点是不允许的。
- 再想“虚叶子”：权为 0、额外补上的叶子，专门用来把叶子个数凑整；权是 0，放在哪层都不增加 WPL。

考查知识点：

- 先想权值：6 个权值是 2、3、4、5、6、7。
- 再想补几个虚叶子：严格三叉树要求（叶子数－1）能被（3－1）＝2 整除；6 个叶子时（6－1）除以 2 余 1，不整除，所以补 1 个权为 0 的虚叶子，变成 7 个叶子。
- 再想构造过程：最小的 3 个是 0、2、3，合并成 5；剩下 4、5、5，合并成 14；剩下 6、7、14，合并成 27（根）。
- 再想 WPL：0×3＋2×3＋3×3＋4×2＋5×2＋6×1＋7×1＝46；也可以按合并过程算：(2＋3)×3＋(4＋5)×2＋(6＋7)×1＝46，就是 B。
- 再想虚叶子放哪层：权为 0，放最深的第 3 层，WPL 不增加；规范放法就是最深层。
- 边界情况：如果叶子个数本来就能被 2 整除，不用补虚叶子；如果权值里本来就有 0，虚叶子和它一样处理。
- 易混对子：不要不补虚叶子直接算——6 个叶子凑不成严格三叉树，算出来的不是合法的三叉哈夫曼树；不要按二叉哈夫曼算——二叉是每次合并 2 个，三叉是每次合并 3 个。

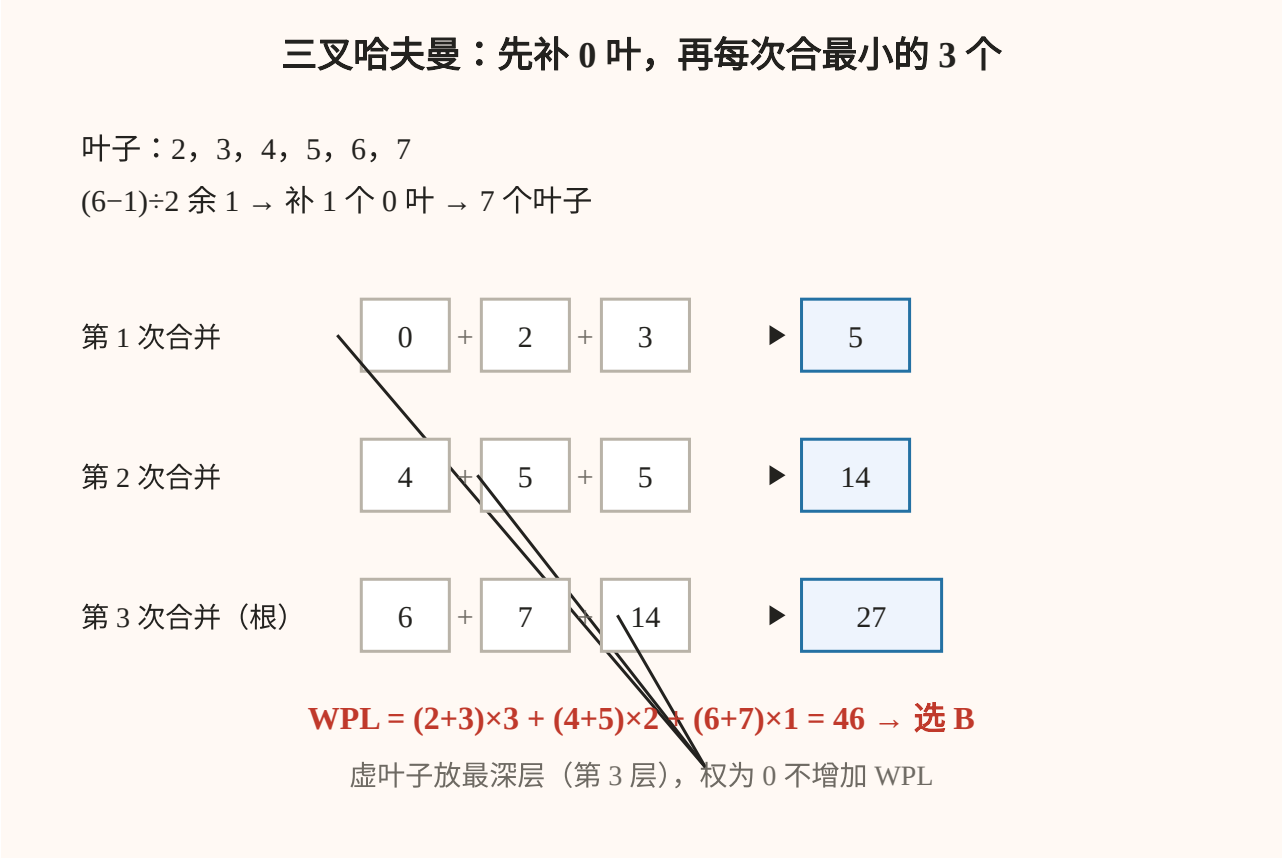
正确选项为什么对： 补 1 个 0 权虚叶子后构造三叉哈夫曼树，WPL＝46，就是 B，选 B。

错误选项为什么错＋举例：

- 比 46 小的：少算了一层，或者把 0 权叶子的层数算错了。教训：WPL 要按最终树的层数一个一个乘。
- 比 46 大的：没补虚叶子、合并顺序错了（先合了大权），或者按二叉哈夫曼算了。教训：三叉先凑整、每次合最小的 3 个。
- 把虚叶子放在浅层的：权为 0 不影响数值，但构造过程不规范，容易连带把别的叶子的层数算错。教训：虚叶子放最深层。

笔记本一句话： 第 4 题＋三叉哈夫曼：先看（叶子数－1）能不能被 2 整除，6 叶补 1 个 0 叶，WPL＝46。

【图像】



三叉哈夫曼树补虚叶与合并过程示意图

【定位】 数据结构 · 第4章 树与二叉树 · 哈夫曼树

【易混对比】

- "三叉哈夫曼"和"二叉哈夫曼"的区别：三叉每次合并最小的 3 个权，二叉每次合并最小的 2 个；合并个数不同，叶子的层数和 WPL 都不同。
- "补虚叶子"和"直接算"的区别：6 个叶子凑不成严格三叉树，必须先补 1 个权 0 的虚叶子凑成 7 个；直接算会合并出只有 2 个孩子的非法结点。

补 1 个 0 权虚叶子，每次合并最小的 3 个，WPL=(2+3)×3+(4+5)×2+(6+7)×1=46，选 B。

【坑位】

先看叶子数能不能凑成严格三叉树，不能就先补 0 权虚叶子。

不补虚叶子直接合并，中间会出现只有 2 个孩子的结点，树不合法、层数跟着算错；按二叉哈夫曼每次合 2 个，层数也全错。正确做法：严格三叉树要求（叶子数−1）能被（3−1）整除，6 个叶子时余 1，补 1 个 0 叶成 7 个；合并过程：0、2、3 合成 5，4、5、5 合成 14，6、7、14 合成 27；WPL=(2+3)×3+(4+5)×2+(6+7)×1=46。

【套路】

- 第一步，看要不要补虚叶子：（叶子数−1）除以（叉数−1），除不尽就补权 0 的虚叶子凑整。
- 第二步，每次合最小的 k 个：三叉就每次合最小的 3 个，直到剩一个根。
- 第三步，按层算 WPL：权乘层数逐个加，或按合并过程 (2+3)×3+(4+5)×2+(6+7)×1=46。

先凑整，再合最小的 k 个，最后权乘层数加起来。

【类比】

像把 6 队人编成"每 3 队一组"的梯队：6 队凑不齐整组，先拉 1 个"影子队员"（权 0、不增加总代价）凑成 7 队；每次把最弱的 3 队并成一队。术语贴回去：影子队员就是"0 权虚叶子"，最弱 3 队合并就是"每次合并最小的 3 个权"，梯队层数就是"叶子的层数"，总代价就是"WPL"。

【口诀】 三叉先凑整，零叶补上来；最小三合一，权乘层数加。

【考频】 「哈夫曼树」在 2009–2022 年解析中出现 6 次 · 中频

第 5 题 · 后序线索二叉树中某叶结点的后继（正确答案：A）

题干与选项：

若 X 是后序线索二叉树中的叶结点，且 X 存在左兄弟结点 Y，则 X 的右线索指向的是

- A. X 的父结点
- B. 以 Y 为根的子树的最左下结点
- C. X 的左兄弟结点 Y
- D. 以 Y 为根的子树的最右下结点

入手点： 先写出这棵树的后序序列，再看 X 在序列里的下一个是谁；下一个就是 X 的后继，右线索就指它。

专有名词从零解释：

- 先想“后序遍历”：按“左子树、右子树、根”的顺序走一棵树，把结点排成一串；这串顺序就是后序序列。
- 再想“线索二叉树”：普通二叉树里，叶子结点的左右指针是空的，白白浪费了；线索二叉树把这些空指针利用起来，让它们指向“序列里的前一个结点”或者“后一个结点”。
- 再想“前驱、后继”：序列里某个结点的前一个叫前驱，后一个叫后继。
- 再想“右线索”：右指针改成线索时，它指向后继。

考查知识点：

- 先想这棵树里 X 的位置：配图是父结点下面左孩子 Y、右孩子 X；X 有左兄弟 Y，说明 X 是右孩子。
- 再想后序序列里 X 的下一个：后序是左、右、根；左子树（Y 这一边）走完，右子树（X）走完，下一个就是父结点；X 是右孩子又是叶子，X 之后紧接着就是父结点。
- 再想结论：X 的后继是父结点，X 的右线索指向父结点，就是 A。
- 再想分情况（后序线索找后继的完整情形）：叶结点是右孩子，后继是父结点；叶结点是左孩子、父结点没有右子树，后继也是父结点；叶结点是左孩子、父结点有右子树，后继是右子树按后序第一个访问的结点。本题 X 有左兄弟，属于第一种。
- 拿生活例子验算：排队时，右边的孩子走完，下一个就是站在后面的家长；X 是右孩子，X 走完，下一个就是父结点。
- 边界情况：如果 X 是整棵树后序序列的最后一个，它就没有后继，右线索就是空的；本题 X 后面还有父结点，后继存在。
- 易混对子：不要把“左兄弟”当成“后继”——左兄弟 Y 在后序里排在 X 前面，是前驱方向；不要跳过父结点指到祖父——后继是序列里紧挨着的下一个，不能隔着结点。

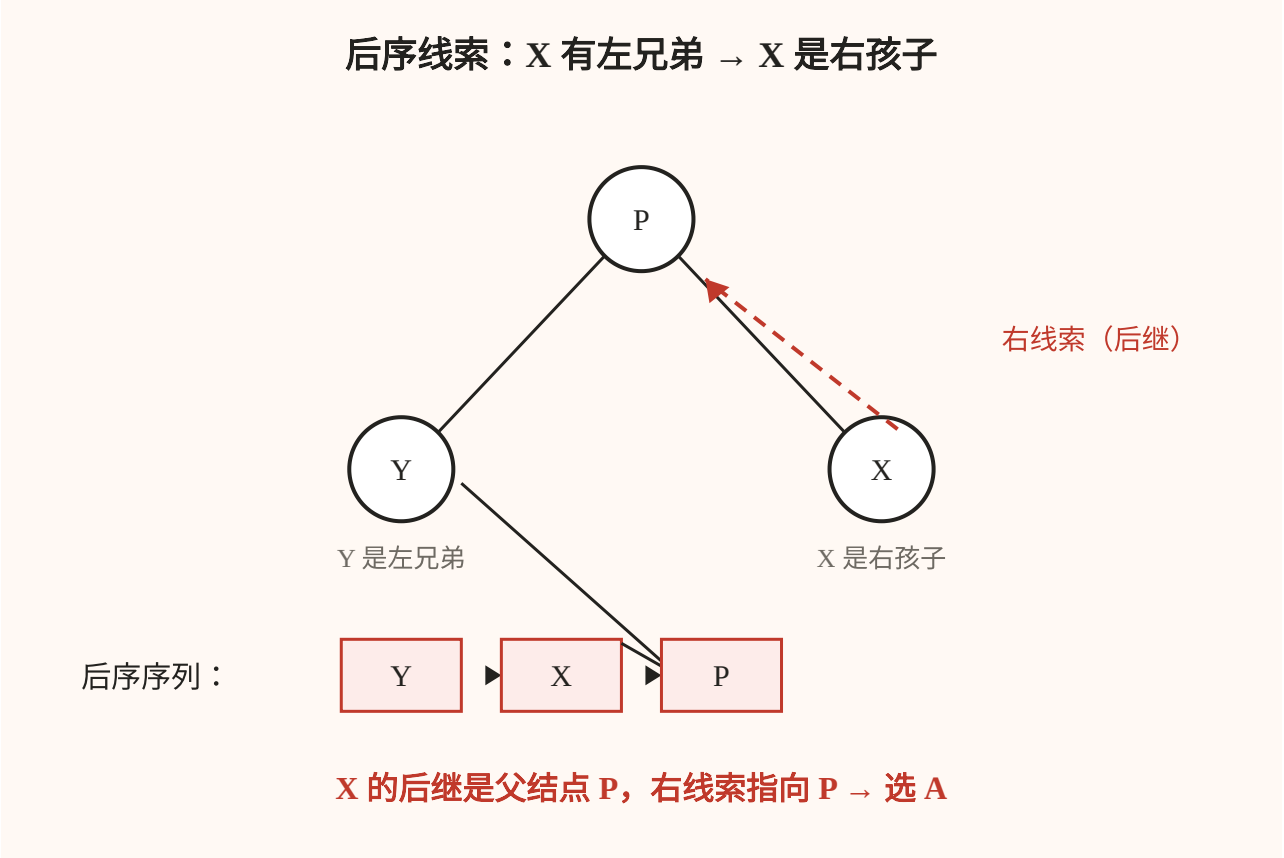
正确选项为什么对： X 是右孩子又是叶子，后序后继是父结点，右线索指向父结点，就是 A，选 A。

错误选项为什么错＋举例：

- 指向左兄弟 Y 的：Y 在后序里排在 X 前面，是前驱方向，不是后继。教训：后继是序列里的下一个，不是兄弟。
- 指向祖父的：跳过了一层，X 的后继是紧挨着的父结点。教训：后继是紧挨着的那个，不能隔着结点。
- 指向“无后继”的：X 不是整棵树的最后一个，父结点还在后面。教训：先确认 X 是不是序列最后一个，再说有没有后继。

笔记本一句话： 第 5 题＋后序线索：X 有左兄弟就是右孩子，后序后继是父结点，右线索指父结点。

【图像】



后序线索二叉树中 X 的右线索指向父结点示意图

【定位】 数据结构 · 第4章 树与二叉树 · 线索二叉树

【易混对比】

- "右线索"和"右孩子"的区别：右孩子是树里真正的右子树根；右线索是空指针改成的线，指向后序序列里的下一个结点。
- "左兄弟"和"后继"的区别：左兄弟 Y 在后序序列里排在 X 前面，是前驱方向；后继是 X 在序列里的下一个，只能是父结点。

X 有左兄弟说明 X 是右孩子，后序里 X 的下一个是父结点，右线索指向父结点，选 A。

【坑位】

别把左兄弟当成后继，兄弟在序列里是前后关系，不是后继。

看到"左兄弟"就想选 B 或 C 是错的；跳过父结点指到祖父也是错的。正确做法：后序是左、右、根；X 是右孩子又是叶子，左子树（Y 这一边）走完、X 走完，下一个就是父结点；后继就是序列里紧挨着的下一个，右线索指向它。

【类比】

像排队报数：按"左、右、根"的顺序报，右边的孩子报完，下一个就是站在后面的家长。术语贴回去：报数顺序就是"后序序列"，站在后面的家长就是"父结点"，家长是孩子的"后继"。

【口诀】 左兄弟在，X 是右孩；后序下一个，就是父来。

【考频】 「线索二叉树」在 2009–2022 年解析中出现 5 次 · 中频

第 6 题 · 二叉排序树删除一结点后再插回、树是否恢复原状（正确答案：C）

题干与选项：

在任意一棵非空二叉排序树 T1 中，删除某结点 v 之后形成二叉排序树 T2，再将 v 插入 T2 形 成二叉排序树 T3。下列关于 T1 与 T3 的叙述中，正确的是 I. 若 v 是 T1 的叶结点，则 T1 与 T3 不同 II. 若 v 是 T1 的叶结点，则 T1 与 T3 相同 III. 若 v 不是 T1 的叶结点，则 T1 与 T3 不同 IV. 若 v 不是 T1 的叶结点，则 T1 与 T3 相同

- A. 仅 I、III
- B. 仅 I、IV
- C. 仅 II、III
- D. 仅 II、IV

入手点： 删叶子和删非叶子是两回事，分开想：删叶子动不动树形，删非叶子动不动树形。

专有名词从零解释：

- 先想“二叉排序树”：左子树所有结点都比根小，右子树所有结点都比根大；中序遍历出来就是从小到大的顺序。
- 再想“删除叶子结点”：叶子没有孩子，直接把它从父结点上摘掉，树上别的地方一点都不动。
- 再想“删除非叶子结点”：这个结点有孩子，不能直接摘；办法是用它的前驱（左子树里最大的）或者后继（右子树里最小的）顶替它原来的位置，顶替的结点从原位置移走。
- 再想“插回”：按二叉排序树的插入规矩，从根开始比大小，一路向下找到空位置放进去。

考查知识点：

- 先想 II（叶结点→相同）：删叶子只摘掉它一个，树上别的地方不动；再插回去，按大小比较又回到原来那个空位置，树恢复原状。II 正确。
- 再想 III（非叶结点→不同）：删非叶子时，顶替的结点从原位置移走了，树形已经变了；再把原结点插回去，它只能落在“变化后的树”里的某个叶子位置，而原结点是非叶子，位置和子树都不可能复原，树一定不同。III正确。
- 再想 I（叶结点→不同）：叶子删了插回明明相同，I 错误。
- 再想 IV（非叶结点→相同）：非叶子删了插回不能保证相同，IV错误。
- 拿小例子验算：树是 10（左 5（左 2，右 7），右 15）；删非叶子 5，用后继 7 顶替，变成 10（左 7（左 2），右 15）；再插 5，5 比 10 小往左、比 7 小往左、比 2 大往右，落到 2 的右孩子，不再是 10 的左孩子，树变了。删叶子 2 再插 2，2 又回到 7 的左孩子，树不变。
- 边界情况：删非叶子再插回一定不同，因为插回的结点只能是叶子位置，而原结点是非叶子；原结点的子树也已经跟着顶替结点变了位置，不可能复原。
- 易混对子：不要以为“删了再插”就一定复原——复原只对叶子成立；不要把“中序序列相同”当成“树形相同”——删插前后中序序列确实一样（都是从小到大），但树形可以不一样。

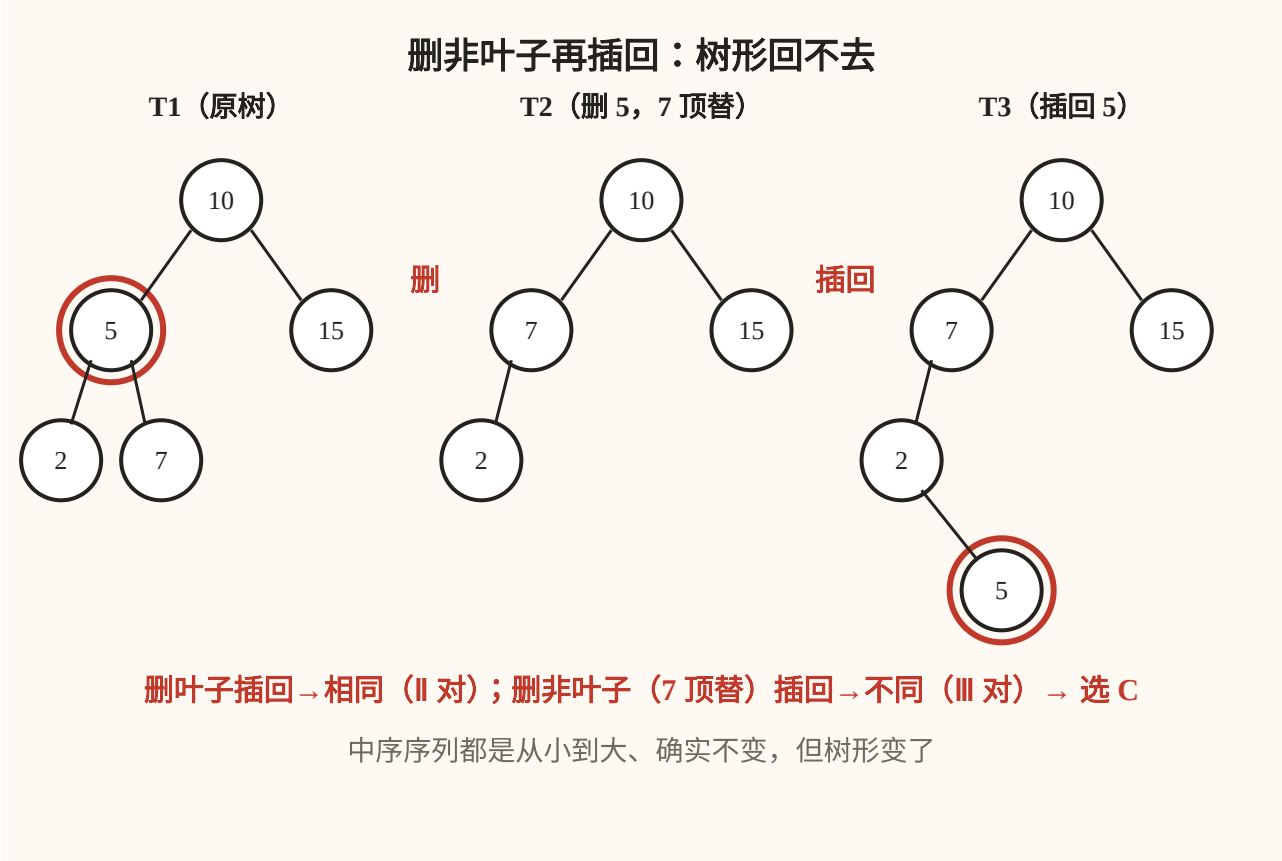
正确选项为什么对： 只有 II、III 正确，组合就是 C，选 C。

错误选项为什么错＋举例：

- A（仅 I、III）：I 错了，叶子删了插回是相同的。教训：删叶子不动树形，插回原位。
- B（仅 I、IV）：I、IV都错了。教训：叶子相同、非叶子不一定相同，两个方向都不能反。
- D（仅 II、IV）：IV错了，非叶子删了插回不能保证相同。教训：顶替结点移走后，树形已经变了。

笔记本一句话： 第 6 题＋二叉排序树删插：删叶子插回相同，删非叶子插回不一定相同，选仅 II 、III。

【图像】



二叉排序树删非叶子再插回树形变化对比示意图

【定位】 数据结构 · 第6章 查找 · 二叉排序树

【易混对比】

- "删叶子再插回"和"删非叶子再插回"的区别：删叶子只摘掉它一个、树形不动，插回去落在原位；删非叶子要用前驱或后继顶替、树形变了，插回去只能落在新树的叶子位置，复原不了。
- "中序序列相同"和"树形相同"的区别：删插前后中序序列都是从小到大、确实相同，但树形可以不一样；题目问的是树形。

叶结点删了插回相同（II 对），非叶结点删了插回不同（III 对），仅 II 、III，选 C。

【坑位】

中序序列不变不等于树形不变，别被"都是从小到大"骗了。

觉得"删了再插同一个结点、树肯定复原"，会误选含 IV 的 D。正确做法：分开看——删叶子不动树形、插回原位，II 正确；删非叶子时顶替结点从原位置移走、树形已经变了，插回去只能落在变化后树形的叶子位置，原结点是非叶子、位置和子树都不可能复原，III 正确。

【套路】

- 第一步，看删的是不是叶子：叶子直接摘，树形不动；非叶子用前驱或后继顶替，树形变了。
- 第二步，看插回落在哪：按大小比较从根往下找空位，落到的都是叶子位置。
- 第三步，对号入座：叶子删插→相同（II）；非叶子删插→不同（III）。

叶子删插能复原，非叶一顶替就回不去。

【类比】

像拔牙再种回去：拔的是不顶事的乳牙（叶子），牙床没动，种回去严丝合缝；拔的是恒牙（非叶子），旁边的牙已经挪位补上了，再种一颗只能种在牙床的新空位上，牙列变了。术语贴回去：旁边的牙挪位就是"前驱或后继顶替"，牙列就是"树形"。

【口诀】 叶子删插能复原，非叶顶替变了样；仅二仅三组合对，答案 C 别选错。

【考频】 「二叉排序树」在 2009–2022 年解析中出现 14 次 · 高频

第 7 题 · 由邻接矩阵判断有向图各顶点度数（正确答案：C）

题干与选项：

设图的邻接矩阵 A 如下所示。各顶点的度依次是（ ）。

```
0 1 0 1
0 0 1 1
0 1 0 0
1 0 0 0
```

注：邻接矩阵以原卷为准。

- A. 1，2，1，2
- B. 2，2，1，1
- C. 3，4，2，3
- D. 4，4，2，2

入手点： 先定这张图是有向图还是无向图，再把“行”和“列”分开数：行管出去的边，列管进来的边。

专有名词从零解释：

- 先想“邻接矩阵”：一个方阵，第 i 行第 j 列的元素表示“顶点 i 到顶点 j 有没有边”；有边记 1（或者记权值），没边记 0。
- 再想“有向图”：边有方向，从一个顶点指向另一个顶点；邻接矩阵一般不对称。
- 再想“入度、出度、度”：从这个顶点出去的边数叫出度，指到这个顶点的边数叫入度，两者加起来叫度。
- 再想“非零元素”：矩阵里不是 0 的格子，就是一条边。

考查知识点：

- 先想定性：邻接矩阵是非对称矩阵，说明这张图是有向图（无向图的邻接矩阵一定对称）。
- 再想出度：第 i 行的非零元素个数，就是从顶点 i 出去的边数，就是出度。
- 再想入度：第 i 列的非零元素个数，就是指到顶点 i 的边数，就是入度。
- 再想度：度＝入度＋出度＝第 i 行非零元个数＋第 i 列非零元个数，就是 C。
- 拿生活例子验算：微信里，关注别人是出度，被别人关注是入度，总的“关联数”就是两者相加。
- 边界情况：自环（顶点 i 到自己的一条边）在第 i 行和第 i 列各出现一次，度里算 2；无向图里矩阵对称，度就是一行的非零元个数（或者一列的），不用行列都加。
- 易混对子：不要把行和列的含义弄反——行是出去的边，列是进来的边；不要把有向图按无向图算——无向图只数一行就够了，有向图必须行列都数。

正确选项为什么对： 度＝行非零元（出度）＋列非零元（入度），就是 C，选 C。

错误选项为什么错＋举例：

- 只数行的：只算了出度，漏了入度。教训：有向图的度是入度加出度。
- 只数列的：只算了入度，漏了出度。教训：行列都要数。
- 行列相乘的：度是加法，不是乘法。教训：入度和出度是两类边，加起来才是总数。

- 按无向图算的（只数一行）：无向图矩阵对称，数一行就够了；有向图矩阵不对称，只数一行会漏一半。教训：先看矩阵对称不对称，再定算法。

笔记本一句话： 第 7 题＋有向图度数：度＝行非零元（出度）＋列非零元（入度）。

【图像】



有向图邻接矩阵行列非零元与顶点度数示意图

【定位】 数据结构 · 第5章 图 · 图的存储（邻接矩阵）

【易混对比】

- "行"和"列"的区别：行管出去的边（出度），列管进来的边（入度）；行列弄反，入度出度就全反了。
- "有向图"和"无向图"的区别：有向图的矩阵不对称，度要行列都数再相加；无向图的矩阵对称，数一行就够了。

度=行非零元（出度）+列非零元（入度），各顶点度依次是 3、4、2、3，选 C。

【坑位】

先看矩阵对称不对称，定下有向还是无向再动手数。

把有向图当无向图、只数一行，会漏掉一半的边；行列弄反会把入度出度算反。正确做法：矩阵不对称→有向图；顶点 i 的度=第 i 行非零元个数（出度）+第 i 列非零元个数（入度）；v1=2+1=3，v2=2+2=4，v3=1+1=2，v4=1+2=3。

【套路】

- 第一步，定图的类型：矩阵不对称→有向图。
- 第二步，数行：第 i 行非零元个数=出度。

- 第三步，数列再相加：第 i 列非零元个数=入度，度=出度+入度。

先定有向无向，再行列分开数，加起来是度数。

【类比】

像微信：你关注的人是出度（行），关注你的人是入度（列），总的"关联数"就是两者相加。术语贴回去：关注别人就是"出边"，被别人关注就是"入边"，关联数就是"顶点的度"。

【口诀】 行管出去列管进，行列相加是度数；先看对称定有向，再数再加不糊涂。

【考频】 「邻接矩阵」在 2009–2022 年解析中出现 6 次 · 中频

第 8 题 · 辨别图的广度优先遍历序列（正确答案：D）

题干与选项：

若对如下无向图进行遍历，则下列选项中，不是广度优先遍历序列的是（）。

> 注：无向图以原卷为准。

- A. h，c，a，b，d，e，g，f
- B. e，a，f，g，b，h，c，d
- C. d，b，c，a，h，e，f，g
- D. a，b，c，d，h，e，f，g

入手点： 先看起点：从 a 出发的遍历，第一个必须是 a；再看走法：深度优先是一路向下再回溯，广度优先是按层推进。

专有名词从零解释：

- 先想“深度优先遍历”：从起点出发，沿着一条路一直往深处走，走到头了再回溯，换一条路继续；像走迷宫时一条道走到黑。
- 再想“广度优先遍历”：从起点出发，先把起点的全部邻居访问完，再访问邻居的邻居，一层一层往外推；像水波纹一样扩散。
- 再想“遍历序列”：按访问顺序把顶点排成一串；不同的走法，排出来的串不一样。

考查知识点：

- 先想起点：遍历从 a 出发，序列第一个必须是 a；A 的第一个是 h，B 的第一个是 e，C 的第一个是 d，都不是从 a 出发的有效遍历。
- 再想 D 的走法：a→b→c→d→h→e→f→g，先一路向下走到 h，再回溯到 e、f、g，这是深度优先的走法。
- 再想 D 为什么不是广度优先：广度优先是按层推进，先把 a 的邻居全部访问完，再访问下一层；D 的第 5 个就走到了深处的 h，不是按层推进的走法，所以 D 是深度优先序列，不是广度优先序列。
- 拿生活例子验算：深度优先像逛超市时沿着一条货架走到头再折回来；广度优先像先把离门口最近的一圈货架全逛完，再逛下一圈。
- 边界情况：如果图里某个顶点和起点不连通，它不会出现在从起点出发的遍历序列里；本题四个序列讨论的是同一张连通图。
- 易混对子：不要看到序列里顶点全就以为是合法遍历——起点不对，序列就不是从 a 出发的遍历；不要把“按层推进”和“一路向下”混为一谈——前者是广度优先，后者是深度优先。

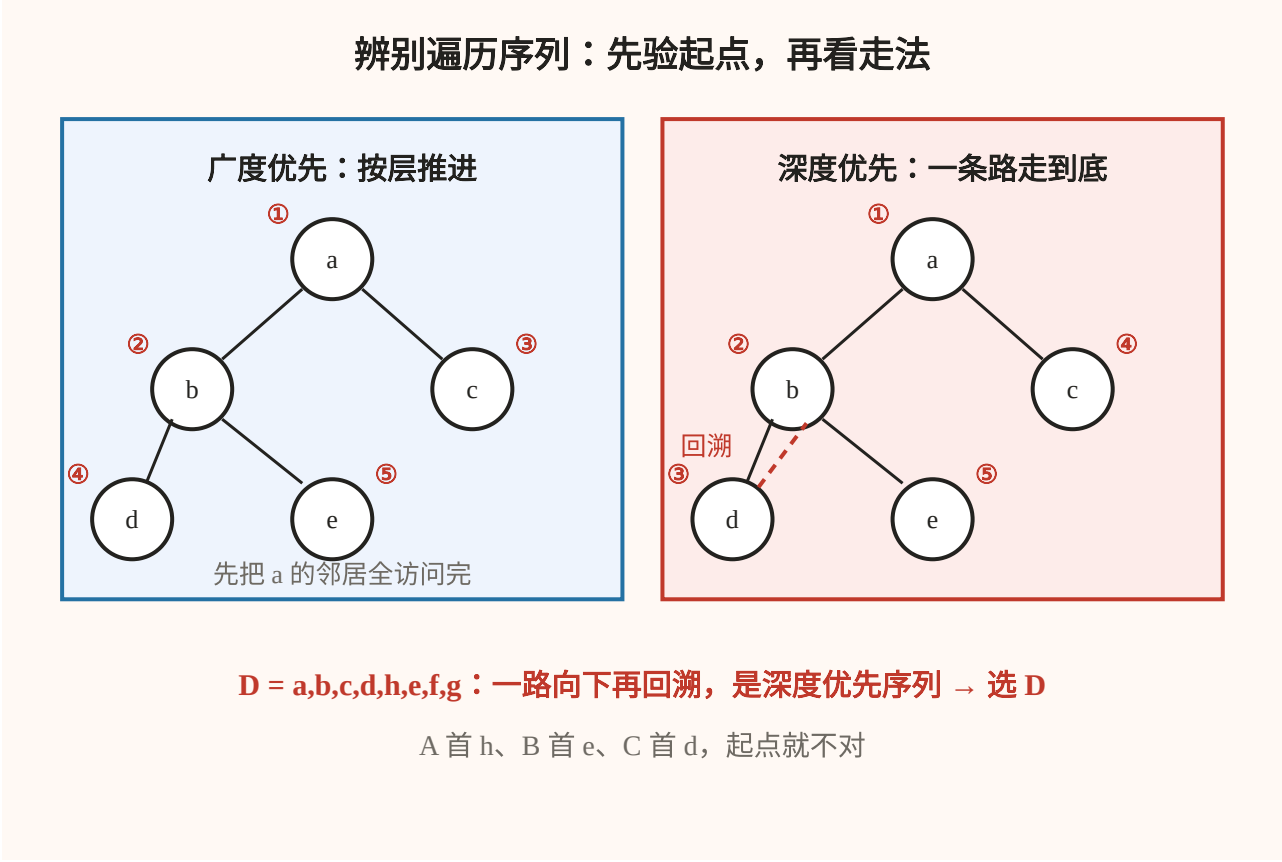
正确选项为什么对： D 以 a 开头、走法是一路向下再回溯，是深度优先遍历序列，就是 D，选 D。

错误选项为什么错＋举例：

- A（h,c,a,b,d,e,g,f）：第一个是 h，不是 a。教训：从 a 出发的遍历，第一个必须是 a。
- B（e,a,f,g,b,h,c,d）：第一个是 e，不是 a。教训：起点不对，序列就不是从 a 出发的遍历。
- C（d,b,c,a,h,e,f,g）：第一个是 d，不是 a。教训：同上，起点先验。

笔记本一句话： 第 8 题＋图遍历：从 a 出发先看首元素；D 一路向下再回溯，是深度优先序列，不是广度优先序列。

【图像】



广度优先按层推进与深度优先一路走到底对比示意图

【定位】 数据结构 · 第5章 图 · 图的遍历

【易混对比】

- "广度优先"和"深度优先"的区别：广度是一层一层往外推，先把当前顶点的邻居全访问完；深度是一条路走到底、走不通再回溯，走法完全不同。
- "起点对"和"走法对"的区别：序列起点必须是从 a 出发（A、B、C 起点就不对）；起点对了还要看走法，D 起点对但走法是深度优先。

A、B、C 都不是从 a 出发的序列；D 以 a 开头但一路向下再回溯，是深度优先序列、不是广度优先，选 D。

【坑位】

先验起点：从 a 出发的遍历，序列第一个必须是 a。

只看走法不看起点，或把"一路向下"当成广度优先，都会选错。正确做法：A 首元素是 h、B 是 e、C 是 d，都不是从 a 出发的有效遍历；D 是 a→b→c→d→h→e→f→g，先一路向下走到深处的 h、再回溯到 e、f、g，是深度优先的走法，不是按层推进的广度优先。

【套路】

- 第一步，验起点：从 a 出发，首元素必须是 a，先筛掉 A、B、C。
- 第二步，看走法：按层推进是广度，一路向下再回溯是深度。
- 第三步，定答案：D 是深度优先序列，就是那个"不是广度优先"的选项。

先看首元素，再看走法是按层还是一条路。

【类比】

像逛超市：广度优先是先把离门口最近的一圈货架全逛完，再逛下一圈；深度优先是沿着一条货架走到头再折回来。术语贴回去：一圈货架就是"同一层的顶点"，走到头再折回就是"回溯"。

【口诀】 起点先看首元素，广度按层深度钻；一路到底再回溯，D 是深度非广度。

【考频】 「图的遍历」在 2009–2022 年解析中出现 2 次 · 低频

第 9 题 · AOE 网中缩短工期应加快哪些活动（正确答案：C）

题干与选项：

下列 AOE 网表示一项包含 8 个活动的工程。通过同时加快若干活动的进度可以缩短整个工程的工期。下列选项中，加快其进度就可以缩短工程工期的是（ ）。

> 注：AOE 网图以原卷为准。

- A. c 和 e
- B. d 和 e
- C. f 和 d
- D. f 和 h

入手点： 先找出全部关键路径，再看：要缩短工期，必须把每条关键路径都变短，漏掉一条都不行。

专有名词从零解释：

- 先想“AOE 网”：顶点表示事件（某个事情完成的那一刻），边表示活动（要干的一件事），边上的数字表示这个活动要花的时间。
- 再想“关键路径”：从起点到终点，花费时间最长的那条路；它的长度就是整个工程的工期。
- 再想“关键活动”：关键路径上的活动；关键活动慢了，工期就慢；非关键活动有一定的机动时间（松弛时间），在机动时间里快慢不影响工期。

考查知识点：

- 先想这张网的关键路径：一共有三条：(b,d,c,g)、(b,d,e,h)、(b,f,h)。
- 再想 d 覆盖了哪几条：d 在前两条关键路径上（(b,d,c,g) 和 (b,d,e,h) 都有 d），加快 d 能让前两条变短，但第三条 (b,f,h) 里没有 d，不变。
- 再想 f 覆盖了哪几条：f 只在第三条关键路径 (b,f,h) 上，加快 f 能让第三条变短，但前两条不变。
- 再想结论：只加快 d，第三条没变，工期还是第三条的长度，缩短不了；只加快 f，前两条没变，工期还是前两条的长度，缩短不了；d 和 f 一起加快，三条关键路径都变短了，工期才缩短。所以答案是 C。
- 拿生活例子验算：装修有三条线要同时干完才能交房（水电、木工、油漆），只催水电，木工和油漆还是老速度，交房时间不变；三条线一起催，交房才能提前。
- 边界情况：关键路径可能不止一条；加快关键活动后，关键路径可能转移（原来非关键的路变成新的关键路径），这是进阶情形；本题只问“加快哪些活动能缩短工期”这个基本判断。
- 易混对子：不要以为“加快一条关键路径上的活动就能缩短工期”——多条关键路径时，必须全部覆盖；不要把非关键活动当成关键活动——非关键活动在机动时间里快慢不影响工期。

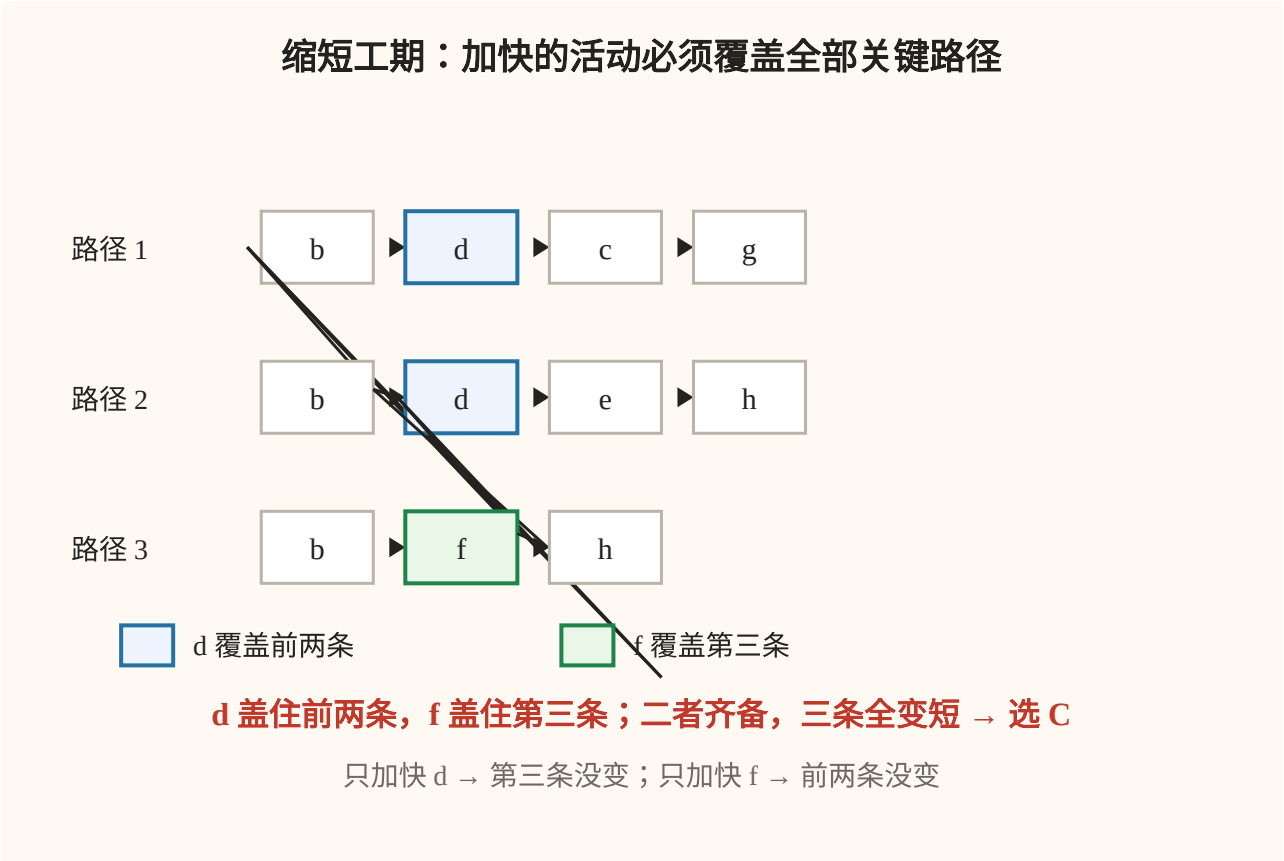
正确选项为什么对： d 覆盖前两条关键路径，f 覆盖第三条，二者齐备，三条都变短，工期才缩短，就是 C，选 C。

错误选项为什么错＋举例：

- 只含 d 的：漏了第三条 (b,f,h)，第三条没变，工期不变。教训：多条关键路径要全覆盖。
- 只含 f 的：漏了前两条，前两条没变，工期不变。教训：同上。
- 含非关键活动的：非关键活动在机动时间里快慢不影响工期。教训：先找关键路径，再看活动在不在关键路径上。

笔记本一句话： 第 9 题＋AOE 网缩短工期：多条关键路径时，加快活动必须覆盖全部关键路径；d 盖前两条、f 盖第三条，二者齐备才行。

【图像】



三条关键路径与活动 d、f 覆盖关系示意图

【定位】 数据结构 · 第5章 图 · 关键路径（AOE 网）

【易混对比】

- "关键活动"和"非关键活动"的区别：关键活动在关键路径上，慢了工期就慢；非关键活动有松弛时间，在机动时间里快慢不影响工期。
- "覆盖一条关键路径"和"覆盖全部关键路径"的区别：只盖一条，另外的路径还是老长度，工期由最长的那条决定，缩短不了；必须全部覆盖才行。

d 覆盖前两条关键路径，f 覆盖第三条 (b,f,h)，二者齐备、三条全变短，工期才缩短，选 C。

【坑位】

多条关键路径时，加快活动必须覆盖全部关键路径，漏一条都不行。

只加快 d，第三条 (b,f,h) 没变，工期还是第三条的长度；只加快 f，前两条没变，工期还是前两条的长度。正确做法：先列出三条关键路径 (b,d,c,g)、(b,d,e,h)、(b,f,h)；d 在前两条上，f 只在第三条上；d 和 f 一起加快，三条全变短，工期才缩短。

【套路】

- 第一步，列出全部关键路径：(b,d,c,g)、(b,d,e,h)、(b,f,h)。
- 第二步，看每个活动盖住哪几条：d 盖前两条，f 盖第三条。
- 第三步，找全覆盖的组合：只有 d+f 盖住全部三条。

列全路径，看谁盖谁，找全覆盖组合。

【类比】

像装修有三条线要同时干完才能交房（水电、木工、油漆）：只催水电，木工油漆还是老速度，交房时间不变；三条线一起催，交房才能提前。术语贴回去：三条线就是"三条关键路径"，催就是"加快活动进度"，交房时间就是"工程工期"。

【口诀】 路径有三条，全盖才缩短；d 盖前两条，f 补第三条。

【考频】 「关键路径」在 2009–2022 年解析中出现 6 次 · 中频

第 10 题 · 5 阶 B 树高度为 2 时最少关键字个数（正确答案：A）

题干与选项：

在一株高度为 2 的 5 阶 B 树中，所含关键字的个数最少是

- A. 5
- B. 7
- C. 8
- D. 14

入手点： 先算每层最少几个关键字，再加起来：根一层，孩子一层。

专有名词从零解释：

- 先想“B 树”：一种多路平衡查找树，每个结点里可以放多个关键字、带多个孩子；查找时先在结点里二分，再往孩子里走。
- 再想“阶”：m 阶 B 树，每个结点最多有 m 个孩子、最多 m－1 个关键字。
- 再想“高度 2”：本题的高度数的是结点层数：根是一层，根的孩子是第二层，一共两层。
- 再想“最少关键字”：每个结点在满足 B 树规矩的前提下，能少放几个关键字就放几个。

考查知识点：

- 先想根最少几个关键字：根比较特殊，最少只要 1 个关键字（根可以只有一个关键字、带两个孩子）。
- 再想孩子最少几个关键字：非根结点最少有 ⌈5/2⌉＝3 个孩子，孩子数比关键字数多 1，所以最少有 2 个关键字。这里要算清楚：5 阶 B 树，非根结点最少 ⌈5/2⌉＝3 个孩子，对应最少 2 个关键字。
- 再想孩子最少几个：根如果不是叶子，最少有 2 个孩子；高度是 2，根下面至少 2 个孩子。
- 再想总数：根 1 个＋2 个孩子各 2 个＝1＋2×2＝5，就是 A。
- 拿生活例子验算：一个文件夹柜，顶层文件夹最少放 1 份文件，下面至少 2 个文件夹、每个最少放 2 份文件，总共最少 5 份。
- 边界情况：高度是 1 时（只有根、根就是叶子），最少只要 1 个关键字；高度是 3 时，算法一样：根 1 个＋第二层 2 个结点各 2 个＋第三层 6 个结点各 2 个，层层算下去。
- 易混对子：不要把“阶”当成“每结点最少关键字”——5 阶是最多 5 个孩子，不是每结点 5 个关键字；不要把根的最少关键字也算成 2——根特殊，最少 1 个。

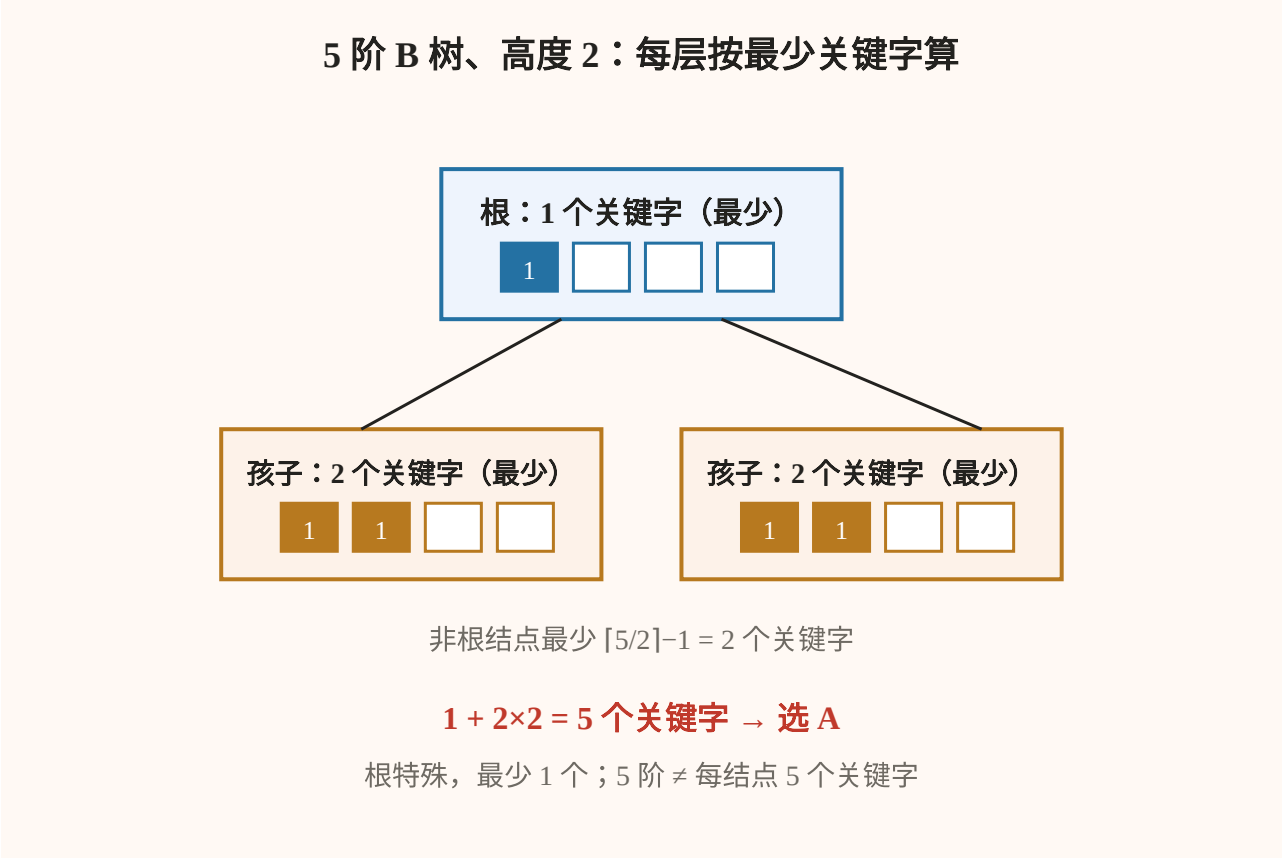
正确选项为什么对： 根 1 个＋两孩子各 2 个＝5，就是 A，选 A。

错误选项为什么错＋举例：

- 算成 4 的：根算成 0 个，或者孩子只算了 1 个。教训：根最少 1 个，孩子最少各 2 个，一项都不能少。
- 算成 7 个、9 个的：把每结点最多关键字当成最少，或者多算了一层。教训：最少是按“每结点最少关键字”算，不是按“最多”算；高度 2 只有两层。
- 把根也算成 2 个关键字的：根特殊，最少 1 个。教训：根和非根的最少关键字不一样。

笔记本一句话： 第 10 题＋5 阶 B 树高 2 最少关键字：根 1 个＋两孩子各 2 个＝5。

【图像】



高度为 2 的 5 阶 B 树最少关键字构成示意图

【定位】 数据结构 · 第6章 查找 · B 树

【易混对比】

- "根"和"非根结点"的区别：根最少只要 1 个关键字（带 2 个孩子）；非根结点最少 ⌈5/2⌉−1=2 个关键字，根不能按 2 算。
- "阶"和"每结点最少关键字"的区别：5 阶是说每结点最多 5 个孩子，不是最少 5 个关键字；最少关键字要按 ⌈m/2⌉−1 算。

根最少 1 个关键字，两个孩子各最少 2 个，1+2×2=5，选 A。

【坑位】

根特殊、最少 1 个关键字，别和非根结点一样按 2 算。

把根也算成 2 个关键字会得出 6；把 5 阶当成每结点 5 个关键字会错得更离谱。正确做法：根最少 1 个关键字（带 2 个孩子）；非根结点最少 ⌈5/2⌉=3 个孩子，对应 2 个关键字；高度 2 只有两层，总数 1+2×2=5。

【套路】

- 第一步，算根最少：根最少 1 个关键字，带 2 个孩子。
- 第二步，算孩子最少：非根结点最少 ⌈5/2⌉−1=2 个关键字。
- 第三步，加总：1+2×2=5。

根按 1 算，孩子按 ⌈m/2⌉−1 算，加起来。

【类比】

像文件夹柜：顶层文件夹最少放 1 份文件，下面至少 2 个文件夹、每个最少放 2 份文件，总共最少 5 份。术语贴回去：顶层文件夹就是"根结点"，下面的文件夹就是"孩子结点"，文件份数就是"关键字个数"。

【口诀】 根一孩二两层加，五阶高二最少五。

【考频】 「B 树」在 2009–2022 年解析中出现 11 次 · 高频

第 11 题 · 基数排序两趟分配收集后的序列（正确答案：C）

题干与选项：

对给定的关键字序列 110，119，007，911，114，120，122 进行基数排序，则第 2 趟分配 收集后得到的关键字序列是

- A. 007，110，119，114，911，120，122
- B. 007，110，119，114，911，122，120
- C. 007，110，911，114，119，120，122
- D. 110，120，911，122，114，007，119

入手点： 每趟都是“分配再收集”：按某一位把数放进 10 个桶，再按桶号从小到大收回来；同桶里的数保持上一趟的相对顺序。

专有名词从零解释：

- 先想“基数排序”：不比较大小、按数位排序的办法；从低位到高位，一位一位地分桶、收回。
- 再想“分配”：按某一位的数字（0 到 9），把数放进对应的桶里。
- 再想“收集”：按桶号从小到大，把桶里的数依次收回来，排成一串。
- 再想“稳定性”：同桶里的数，收集时保持它们上一趟的相对顺序不变；基数排序靠稳定性才能排对。

考查知识点：

- 先想第一趟（按个位）：个位是 0 的有 110、120；个位是 1 的有 911；个位是 2 的有 122；个位是 4 的有 114；个位是 7 的有 007；个位是 9 的有 119；按桶号收回：110,120,911,122,114,007,119。
- 再想第二趟（按十位）：十位是 0 的有 007；十位是 1 的有 110、911、114、119（按第一趟的相对顺序）；十位是 2 的有 120、122（按第一趟的相对顺序）；按桶号收回：007,110,911,114,119,120,122。
- 再想结论：最终序列就是 C。
- 拿生活例子验算：先按“个位数”把卡片分到 10 个盒子、再收回；再按“十位数”分盒、收回；两趟下来，卡片就按数值排好了。
- 边界情况：三位数要排三趟（个位、十位、百位）才算完；本题只问两趟后的结果，百位还没排。
- 易混对子：不要把第一趟的结果当成最终答案——第一趟只排了个位，十位还没排；不要在第二趟里打乱同桶顺序——同桶里必须保持第一趟的相对顺序。

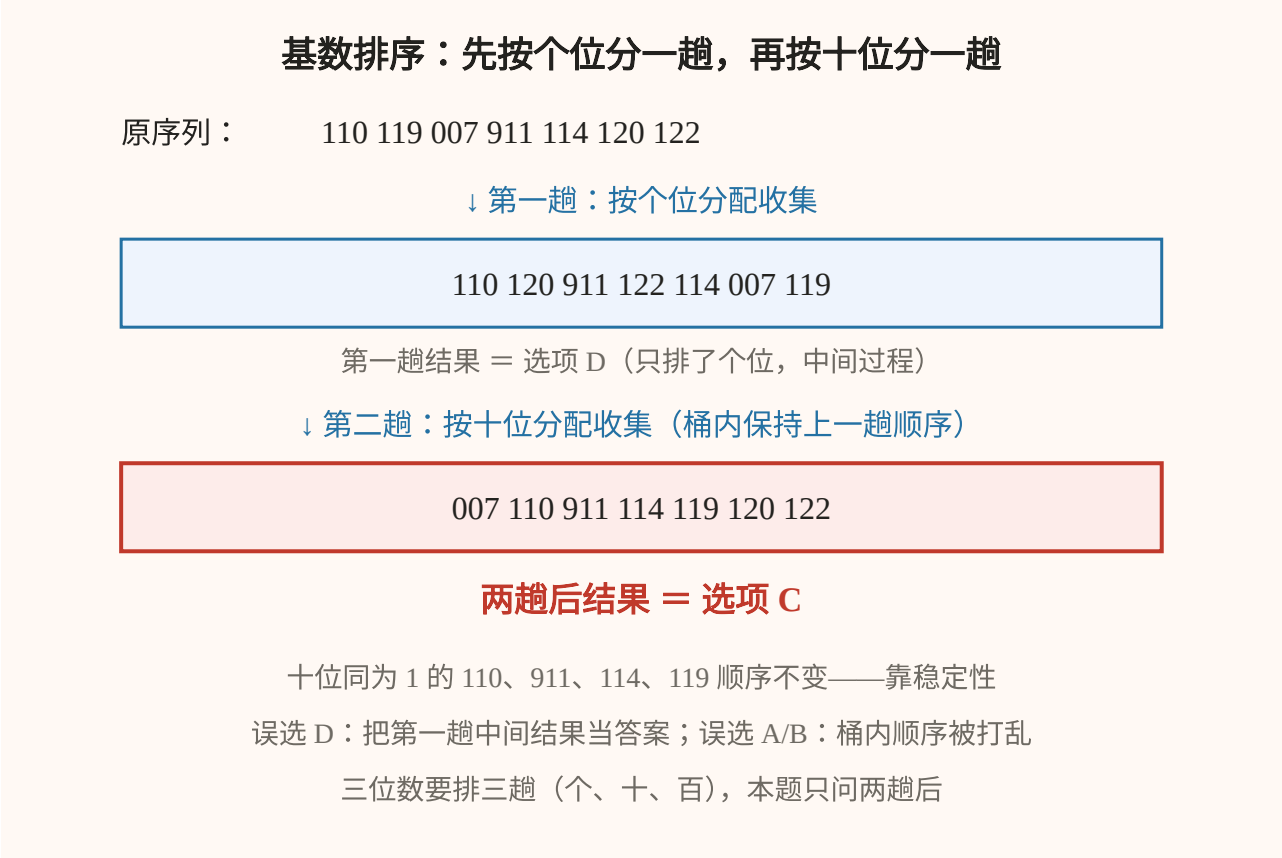
正确选项为什么对： 两趟分配收集后是 007,110,911,114,119,120,122，就是 C，选 C。

错误选项为什么错＋举例：

- A（007,110,119,114,911,120,122）：第二趟十位桶内顺序乱了，119 跑到了 114 前面、911 落后了。教训：同桶里保持上一趟的相对顺序，110、911、114、119 的顺序不能变。
- B（007,110,119,114,911,122,120）：除了 A 的桶内错误，还把 122、120 的顺序弄反了。教训：十位都是 2 的两个数，第一趟里 120 在 122 前面，第二趟里也必须是 120 在前。
- D（110,120,911,122,114,007,119）：这只是第一趟个位收集的结果，少了第二趟十位排序。教训：第一趟是中间过程，不是最终答案。

笔记本一句话： 第 11 题＋基数排序：每趟分配再收集，同桶内保持上一趟顺序；D 是第一趟结果，C 是最终答案。

【图像】



基数排序两趟分配收集：第一趟按个位、第二趟按十位

【定位】 数据结构 · 第8章 排序 · 基数排序（分配与收集）

【易混对比】

- "分配"和"收集"的区别：分配是按某一位把数放进 10 个桶，收集是按桶号从小到大把桶里的数收回来；一趟排序就是一次分配加一次收集。
- "第一趟结果"和"两趟后答案"的区别：第一趟只排了个位，十位还没排；D 只是第一趟的中间结果，C 才是两趟后的最终序列。

两趟分配收集后序列是 007,110,911,114,119,120,122，选 C。

【坑位】

同桶里的数必须保持上一趟的相对顺序，顺序一乱就掉进 A 或 B。

A 把十位同为 1 的 119 放到了 114 前面，B 还把十位同为 2 的 122、120 弄反了——都是第二趟里没保住第一趟的桶内顺序。正确做法：第二趟按十位进桶时，数的先后按第一趟的序列走，收集时桶内不再重排。

【套路】

- 第一步，第一趟按个位分桶收集：得 110,120,911,122,114,007,119。
- 第二步，第二趟按十位分桶，桶内保持第一趟顺序：得 007,110,911,114,119,120,122。
- 第三步，对照选项：与 C 一致。

每趟分桶加收集，同桶旧序照旧不动。

【类比】

像给卡片排序：先按个位数分进 10 个盒子再收回，再按十位数分盒收回，两趟下来卡片就按大小排好了。术语贴回去：盒子就是"桶"，分盒就是"分配"，收回就是"收集"，桶内不许重排就是"稳定性"。

【口诀】 逐位分桶再收回，桶内旧序不许改。

【考频】 「基数排序」在 2009–2022 年解析中出现 4 次 · 中频

第 12 题 · 由 CPI 和主频求 MIPS（正确答案：C）

题干与选项：

某计算机主频为 1.2GHz，其指令分为 4 类，它们在基准程序中所占比例及 CPI 如下表所示。

| 指令类型 | 所占比例 | CPI |

|---|---|---|

| A | 50% | 2 |

| B | 20% | 3 |

| C | 10% | 4 |

| D | 20% | 5 |

该机的 MIPS 数是（）。

- A. 100
- B. 200
- C. 400
- D. 600

入手点： 先算 CPI（每条指令平均吃几个周期），再用主频除以 CPI。

专有名词从零解释：

- 先想“CPI”：每条指令平均需要的时钟周期数；不同指令吃的周期不一样，CPI 是按占比加权平均出来的。
- 再想“主频”：CPU 每秒发多少个时钟周期，比如 1.2GHz 就是每秒 12 亿个周期。
- 再想“MIPS”：每秒能执行多少百万条指令；是衡量 CPU 速度的一个常用单位。

考查知识点：

- 先想 CPI：2×0.5＋3×0.2＋4×0.1＋5×0.2＝1.0＋0.6＋0.4＋1.0＝3。
- 再想主频换算：1.2GHz＝1200MHz，也就是每秒 1200 百万个周期。
- 再想 MIPS：每秒周期数除以每条指令的周期数＝1200÷3＝400，就是 C。
- 拿生活例子验算：主频是“每秒发多少拍”，CPI 是“每条指令吃几拍”，每秒指令数＝拍数除以每条指令的拍数。
- 边界情况：单位要对齐——主频用 MHz，MIPS 的 M 就是百万，1200MHz 除以 3 正好是 400MIPS；如果主频给的是 GHz，先乘 1000 换成 MHz。
- 易混对子：不要用主频乘 CPI——乘出来是“每秒周期数×每条指令周期数”，单位都不对；不要把各类指令的周期直接平均——占比不一样，必须加权。

正确选项为什么对： CPI＝3，主频 1200MHz，MIPS＝1200÷3＝400，就是 C，选 C。

错误选项为什么错＋举例：

- 算成 1200 的：忘了除以 CPI。教训：主频是周期数，不是指令数，要除以每条指令的周期数。
- 算成 3600 的：主频乘了 CPI。教训：MIPS＝主频÷CPI，不是乘。
- 算成 133 的：CPI 算错了（比如占比没乘对）。教训：CPI 是加权平均，占比要一个一个乘。

笔记本一句话： 第 12 题＋MIPS：先算 CPI＝3，再用主频÷CPI＝1200÷3＝400。

【图像】



CPI 加权平均为 3，主频 1200MHz 除以 3 得 400MIPS

【定位】 计算机组成原理 · 第1章 计算机系统概述 · 计算机性能指标（CPI 与 MIPS）

【易混对比】

- "CPI"和"主频"的区别：CPI 是每条指令平均吃几个时钟周期，主频是每秒发出多少个时钟周期；MIPS=主频÷CPI，不是乘。
- "加权平均"和"直接平均"的区别：四类指令占比不同，CPI 必须按占比加权；把 2、3、4、5 直接平均会算错。

CPI=3，主频 1200MHz，MIPS=1200÷3=400，选 C。

【坑位】

题干表格里的 A、B、C、D 是四类指令的编号，不是选项 A、B、C、D，别张冠李戴。

错在把"A 类指令"当成选项 A 去凑答案。正确做法：先无视表格字母，按占比加权算 CPI=2×0.5+3×0.2+4×0.1+5×0.2=3，再用主频 1.2GHz=1200MHz 除以 3。

【套路】

- 第一步，算 CPI：2×0.5+3×0.2+4×0.1+5×0.2=3。
- 第二步，主频换算：1.2GHz=1200MHz。
- 第三步，相除：1200÷3=400MIPS。

先加权算 CPI，再用主频除以 CPI。

【类比】

像算一支队伍的平均配速：主频是每秒发多少拍，CPI 是每条指令吃几拍，每秒能执行多少条指令就是拍数除以每条的拍数。术语贴回去：每秒的拍数就是"主频"，每条指令的拍数就是"CPI"，每秒百万条指令就是"MIPS"。

【口诀】 占比加权算 CPI，主频一除 MIPS 来。

【考频】 「CPI」在 2009–2022 年解析中出现 9 次 · 中频

第 13 题 · IEEE 754 单精度浮点数的真值（正确答案：A）

题干与选项：

某数采用 IEEE 754 单精度浮点数格式表示为 C640 0000H，则该数的值是 × ×

- A. -1.5×2^13
- B. -1.5×2^12
- C. -0.5x×2^13
- D. -0.5×2^12

入手点： 先把 32 位拆成三段：符号位、阶码、尾数；再一段一段翻译，最后套公式。

专有名词从零解释：

- 先想“IEEE 754 单精度”：32 位浮点数的标准格式：1 位符号位＋8 位阶码＋23 位尾数。
- 再想“符号位”：最高位，0 是正数，1 是负数。
- 再想“阶码”：8 位，用移码表示；真实指数＝阶码减去偏置 127。
- 再想“尾数”：23 位小数部分；规格化数要在前面补一个隐含的 1，变成 1.尾数。

考查知识点：

- 先想拆段：C640 0000H 写成二进制是 1100 0110 0100 0000 0000 0000 0000 0000；符号位 S＝1，阶码是 1000 1100，尾数是 100 0000 0000 0000 0000 0000。
- 再想符号：S＝1，说明是负数。
- 再想指数：阶码 1000 1100 换成十进制是 140，减去偏置 127，得 13。
- 再想尾数：尾数 100……0，前面补隐含的 1，变成 1.5。
- 再想真值：－1.5×2^13，就是 A。
- 拿生活例子验算：科学计数法里，1.5×10^3 是 1500；浮点数也一样，是“尾数×2 的指数次方”，只不过底数是 2、指数要减偏置。
- 边界情况：阶码全 0 是非规格化数，没有隐含的 1；阶码全 1 是无穷大或者 NaN；本题阶码是 140，是正常规格化数。
- 易混对子：不要忘了减偏置 127——阶码 140 不是指数，指数是 140－127＝13；不要忘了符号位——S＝1 是负数；不要忘了隐含的 1——尾数 100……0 对应的是 1.5，不是 0.5。

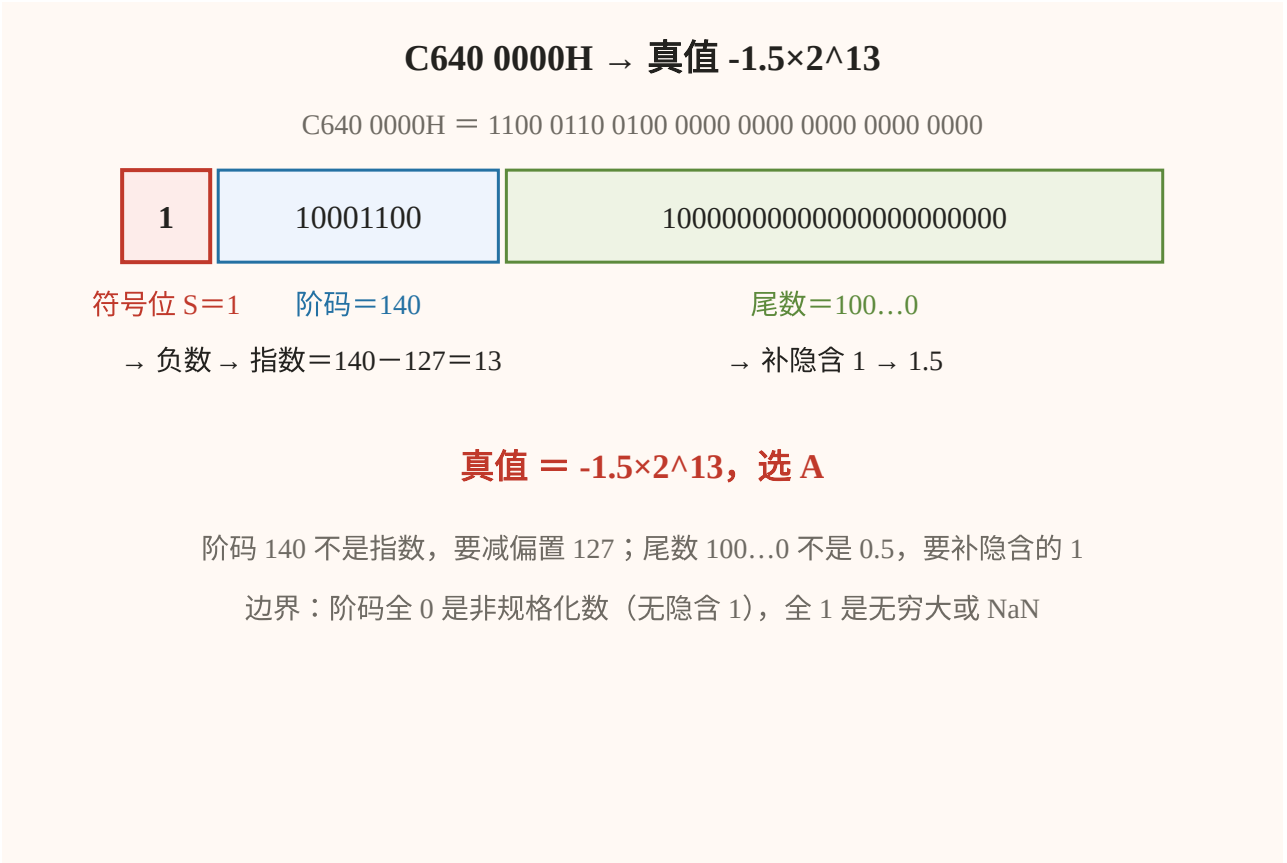
正确选项为什么对： 符号负、指数 13、尾数 1.5，真值是 －1.5×2^13，就是 A，选 A。

错误选项为什么错＋举例：

- 算成正数的：符号位看错了，S＝1 是负数。教训：最高位先看符号。
- 指数写成 140 的：没减偏置 127。教训：阶码减 127 才是真实指数。
- 尾数写成 0.5 的：漏了隐含的 1。教训：规格化数的尾数前面要补 1。

笔记本一句话： 第 13 题＋IEEE 754 单精度：拆 1＋8＋23，指数＝阶码－127，尾数加隐含的 1，真值 －1.5×2^13。

【图像】



C6400000H 拆成符号位、阶码、尾数三段再翻译成真值

【定位】 计算机组成原理 · 第2章 数据的表示和运算 · IEEE 754 浮点数表示

【易混对比】

- "阶码"和"指数"的区别：阶码是 8 位移码 140，指数是阶码减偏置 127 得 13；选项里的 2^13 用的是指数。
- "尾数"和"尾数真值"的区别：23 位尾数 100…0 对应小数 0.5，规格化数要在前面补隐含的 1，真值才是 1.5。

符号为负、指数 13、尾数 1.5，真值是 -1.5×2^13，选 A。

【坑位】

阶码 140 不是指数，减 127 才是；尾数 100…0 不是 0.5，补隐含的 1 才是 1.5。

漏减偏置会把指数写成 140，漏补 1 会把尾数写成 0.5 误选 C。正确做法：拆成 1+8+23 三段，指数=阶码-127，尾数前面补 1，最后看符号位定正负。

【套路】

- 第一步，拆段：C6400000H→符号位 S=1，阶码=10001100B=140，尾数=100…0。
- 第二步，算指数：140-127=13。
- 第三步，补 1 拼真值：尾数真值 1.5，符号为负，得 -1.5×2^13。

拆三段，指数减偏置，尾数补 1 再拼。

【类比】

像科学计数法 1.5×10^3：1.5 是系数，3 是指数；浮点数也一样，只是底数换成 2，指数要先减偏置。术语贴回去：科学计数法的系数就是"尾数"，幂就是"阶码"，小数点前补的 1 就是规格化数的"隐含位"。

【口诀】 一八二三拆三段，阶码减偏置，尾数补一再相乘。

【考频】 「IEEE 754」在 2009–2022 年解析中出现 15 次 · 高频

第 14 题 · 移位运算与补码相加是否溢出、是否丢失精度（正确答案：A）

题干与选项：

某字长为 8 位的计算机中，已知整型变量 x、y 的机器数分别为[x]补=1 1110100，[y]补=1 0110000。若整型变量 z=2*x+y/2，则 z 的机器数为

- A. 1 1000000
- B. 0 0100100
- C. 1 0101010
- D. 溢出

入手点： 先移位，再相加；移位时分开验溢出和精度，相加时看符号变没变。

专有名词从零解释：

- 先想“补码”：计算机里表示有符号整数的办法；正数的补码就是它本身，负数的补码是“符号位 1、其余位取反加 1”。
- 再想“算术左移”：整个数向左挪一位，右边补 0；相当于乘 2。
- 再想“算术右移”：整个数向右挪一位，左边补符号位（正数补 0，负数补 1）；相当于除以 2。
- 再想“溢出”：结果超出了补码能表示的范围；补码加法里，同号相加、结果变号，就是溢出。

考查知识点：

- 先想 x×2：x 算术左移一位，得 1 1101000；符号位还是 1，没有溢出，精度也没有丢失。
- 再想 y/2：y 算术右移一位，得 1 1011000；左边补的是符号位 1，没有溢出，精度也没有丢失（本题是能整除的情形）。
- 再想相加：1 1101000＋1 1011000＝1 1000000（最高位的进位丢弃）；两个负数相加，结果还是负数，符号没变，没有溢出。
- 再想结论：相加结果是 1 1000000，就是 A。
- 拿生活例子验算：算术右移补符号位，就像负数除以 2 后还是负数，符号不能变；左移补 0，就像乘 2 后末尾多个 0。
- 边界情况：左移后符号位变了，就是溢出（比如正数左移变成负数）；右移时被移出去的那位是 1，说明除不尽、丢失了精度；本题两步都没有这些问题。
- 易混对子：不要把“最高位有进位”当成溢出——补码加法里，进位丢弃是正常的，溢出要看符号变没变；不要把算术右移当成逻辑右移——逻辑右移左边补 0，负数右移会变成正数，那是错的。

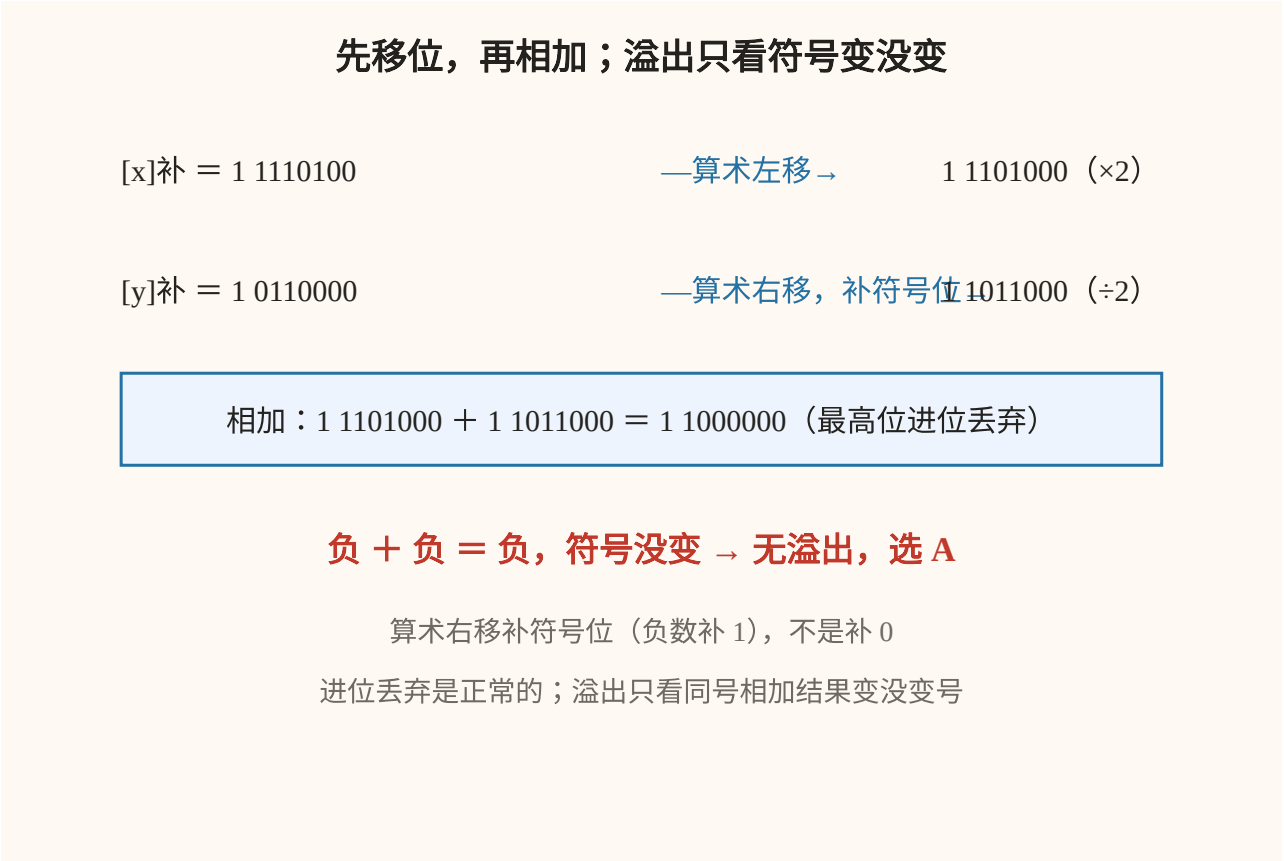
正确选项为什么对： 两步移位都不溢出、不丢精度，相加得 1 1000000、无溢出，就是 A，选 A。

错误选项为什么错＋举例：

- B（0 0100100）：符号位算错了，把负数和当成了正数。教训：两个负数相加，结果还是负数，符号位是 1。
- C（1 0101010）：移位或者加法算错了，比如右移时左边补了 0 而不是补符号位。教训：算术右移补符号位，不是补 0。
- D（溢出）：把“最高位进位丢弃”误当成了溢出。教训：补码溢出看同号相加结果是否变号，进位丢弃不是溢出。

笔记本一句话： 第 14 题＋补码移位相加：算术右移补符号位；补码溢出看同号相加结果是否变号，进位丢弃不是溢出。

【图像】



x 算术左移、y 算术右移补符号位，再相加判溢出的流程

【定位】 计算机组成原理 · 第2章 数据的表示和运算 · 补码的移位运算与溢出判断

【易混对比】

- "算术右移"和"逻辑右移"的区别：算术右移左边补符号位（负数补 1），逻辑右移左边补 0；本题 y 是负数，补 0 会把负数变成正数。
- "最高位进位"和"溢出"的区别：补码加法丢弃最高位进位是正常的；溢出只看同号相加结果有没有变号。

2x 得 1 1101000，y/2 得 1 1011000，相加得 1 1000000 且无溢出，选 A。

【坑位】

看到最高位有进位就喊溢出是错的；补码溢出只看同号相加结果变没变号。

错在把进位丢弃当成溢出，误选 D。正确做法：两个负数相加结果还是负数，符号没变，无溢出，结果取低 8 位 1 1000000。

【套路】

- 第一步，移位：x 算术左移一位得 1 1101000；y 算术右移一位、左边补符号位 1，得 1 1011000。
- 第二步，相加：1 1101000+1 1011000=1 1000000（最高位进位丢弃）。
- 第三步，判溢出：同号相加结果同号，无溢出。

先移位再相加，溢出只看符号变没变。

【类比】

像两个人各挪一步再报数：负数右移时左边要补上"负号"（符号位），就像往后退一步方向还是负的。术语贴回去：挪步就是"移位"，补负号就是"算术右移补符号位"，报数时符号变了才是"溢出"。

【口诀】 左移乘二右移除，右移补符别补 0；溢出只看符号变，进位丢弃不算数。

【考频】 「补码」在 2009–2022 年解析中出现 17 次 · 高频

第 15 题 · 海明码纠一位错至少需要几位校验位（正确答案：C）

题干与选项：

用海明码对长度为 8 位的数据进行检/纠错时，若能纠正一位错。则校验位数至少为

- A. 2
- B. 3
- C. 4
- D. 5

入手点： 先写出校验位个数的不等式，再从小到大试 k，第一个满足的 k 就是最少位数。

专有名词从零解释：

- 先想“海明码”：一种能发现错误、还能定位到是哪一位错了的校验码；纠一位错的意思是，任何一位出错，都能找出来并改对。
- 再想“校验位”：额外加上的位，不传送数据，专门用来校验；数据位出错时，靠校验位算出来的“指错字”定位到出错的位置。
- 再想“为什么是 2 的 k 次方”：k 位校验位能组成 2^k 种不同的指错字，每种指错字对应一种情况。

考查知识点：

- 先想不等式：设校验位 k 位、数据位 n 位，需要 2^k ⩾ n＋k＋1；右边的 n＋k＋1 是全部可能的情况：n 个数据位各可能错、k 个校验位各可能错、再加上 1 种“都没错”的情况。
- 再想 n＝8 时试 k：k＝3 时，2^3＝8，右边是 8＋3＋1＝12，8 比 12 小，不成立；k＝4 时，2^4＝16，右边是 8＋4＋1＝13，16 比 13 大，成立。
- 再想结论：最少需要 4 位校验位，就是 C。
- 拿生活例子验算：校验位像“暗号位数”，暗号要能指到每一个可能出错的位置，还要能表示“没错”；3 位暗号只有 8 种说法，盖不住 12 种情况；4 位暗号有 16 种说法，才够用。
- 边界情况：k 再大也行，但题目问的是“最少”；n 变化时，k 要重新按不等式算。
- 易混对子：不要把不等式写成 2^k ⩾ n＋1——校验位自身也可能出错，所以右边要加上 k；不要把“2^k ⩾ n＋k＋1”里的 k 和 n 弄混——n 是数据位，k 是校验位。

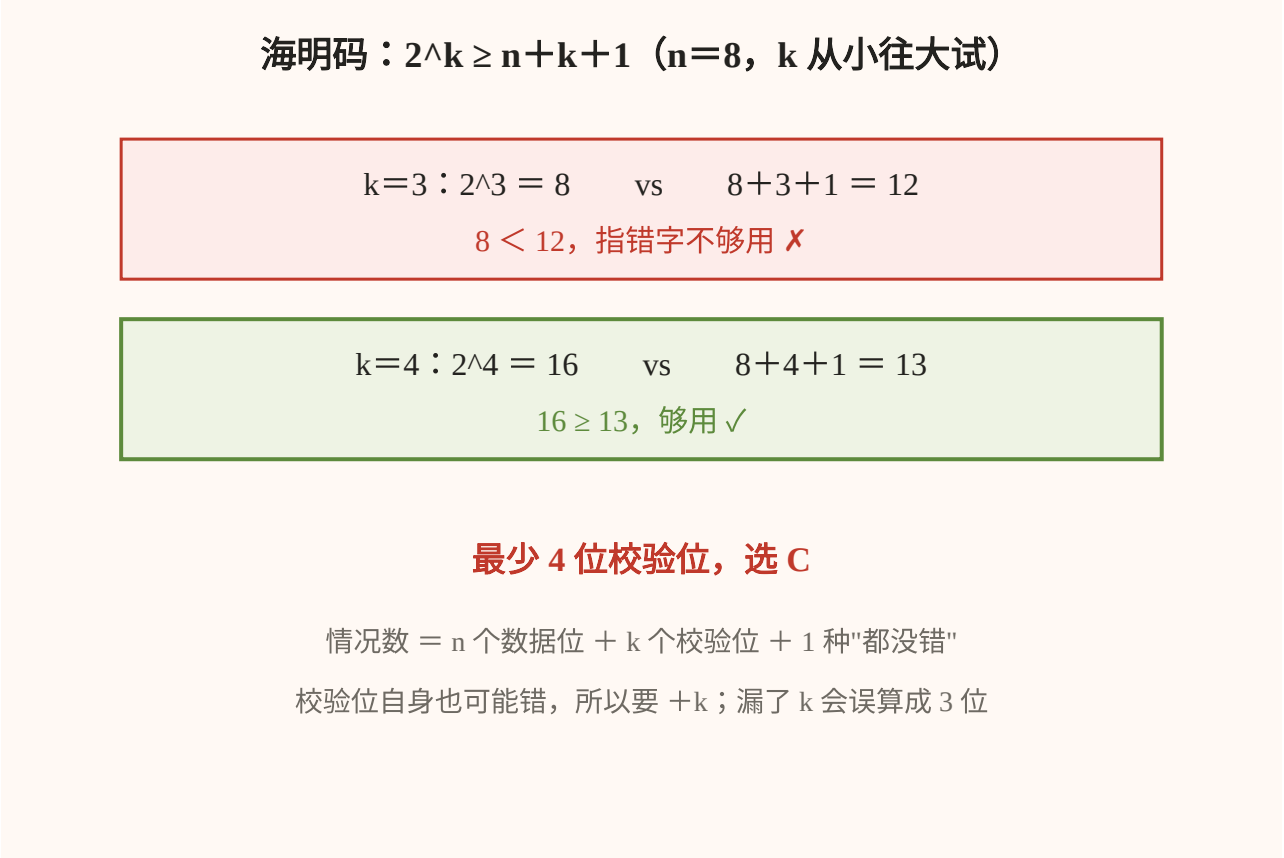
正确选项为什么对： k＝4 时 16 ⩾ 13 成立，k＝3 时不成立，最少 4 位，就是 C，选 C。

错误选项为什么错＋举例：

- 算成 3 位的：不等式漏了 k（写成了 2^k ⩾ n＋1）。教训：校验位自身也可能错，情况数是 n＋k＋1。
- 算成 5 位的：多算了，5 位虽然够用，但不是最少。教训：题目问“最少”，从小到大试，第一个满足的就是答案。
- 把 n 和 k 位置写反的：不等式列错了，算出来的 k 也错了。教训：n 是数据位 8，k 是要求的校验位。

笔记本一句话： 第 15 题＋海明码：2^k ⩾ n＋k＋1，n＝8 时 k 最少是 4。

【图像】



k=3 时指错字不够、k=4 时够用的对比

【定位】 计算机组成原理 · 第2章 数据的表示和运算 · 校验码（海明码）

【易混对比】

- "校验位"和"数据位"的区别：校验位是额外加的、专门用来定位错误；数据位是真正要传的信息；不等式 2^k≥n+k+1 里 n 是数据位，k 是校验位。
- "n+k+1"和"n+1"的区别：校验位自身也可能出错，所以情况数要把 k 也加进去；漏了 k 会算成 3 位。

2^k≥8+k+1，k=3 不成立，k=4 成立，最少 4 位校验位，选 C。

【坑位】

不等式漏了 k 会算成 3 位；题目问的是最少，从小到大试 k。

错在把不等式写成 2^k≥n+1。正确做法：情况数=n 个数据位+k 个校验位+1 种"都没错"，从 k=3 开始试，第一个满足的是 k=4。

【套路】

- 第一步，列不等式：2^k≥n+k+1，n=8。
- 第二步，从小试 k：k=3 时 8<12 不成立；k=4 时 16≥13 成立。
- 第三步，取最小：k=4。

不等式列对，从小到大试 k。

【类比】

像给每个可能出错的位置发一个"暗号"：k 位校验位能编出 2^k 种暗号，要盖住 n+k 个位置再加一个"都没错"，暗号不够就加长。术语贴回去：暗号就是"校验位组合"，每种说法就是"指错字"的一种取值。

【口诀】 校验 k 位列不等，二的 k 方盖 n＋k＋一。

【考频】 「海明码」在 2009–2022 年解析中出现 3 次 · 中频

第 16 题 · 分页系统中虚拟地址转物理地址（正确答案：A）

题干与选项：

某计算机主存地址空间大小为 256MB，按字节编址。虚拟地址空间大小为 4GB，采用页式存储管理，页面大小为 4KB，TLB（快表）采用全相联映射，有 4 个页表项，内容如下表所示。

| 有效位 | 标记 | 页框号 |

|---|---|---|

| 0 | FF180H | 0002H |

| 1 | 3FFF1H | 0035H |

| 0 | 02FF3H | 0351H |

| 1 | 03FFFH | 0153H |

则对虚拟地址 03FFF180H 进行虚实地址变换的结果是（）。

- A. 0153180H
- B. 0035180H
- C. TLB 缺失
- D. 缺页

入手点： 先把虚拟地址拆成“页号＋页内地址”，再拿页号去页表里查页框号，最后把页框号和页内地址拼起来。

专有名词从零解释：

- 先想“分页”：把虚拟地址空间和物理内存都切成同样大小的页；虚拟地址由“页号”和“页内地址”两部分组成。
- 再想“页号、页内地址”：页号是“第几页”，页内地址是“这页里的第几个字节”；页内地址在翻译时原样不动。
- 再想“页表”：页号到页框号的对照表；查表就知道这页虚拟页住在物理内存的哪个页框里。
- 再想“页框号”：物理内存里的页编号；物理地址就是“页框号＋页内地址”。

考查知识点：

- 先想页内地址占几位：页面大小是 4KB，4KB＝2 的 12 次方，所以页内地址占 12 位。
- 再想虚拟地址占几位：地址空间是 4GB，4GB＝2 的 32 次方，所以虚拟地址一共 32 位；页号占 32－12＝20 位。
- 再想拆地址：虚拟地址 03FF F180H，前 20 位是页号 03FFFH，后 12 位是页内地址 180H。
- 再想查表：页标记为 03FFFH 的表项，对应的页框号是 0153H。
- 再想拼接：页框号 0153H 加上页内地址 180H，物理地址是 0153180H，就是 A。
- 拿生活例子验算：虚拟地址像“小区第几栋第几号房”，页号是栋号、页内地址是房号；查表知道这栋楼实际在物理内存的哪个位置，再把房号原样带上，就是物理地址。
- 边界情况：如果页标记在页表里查不到，就是缺页，要先把页从外存调进来；如果快表（TLB）里没命中，只是快表缺失，还能继续查页表；本题查到了页表项，既不是缺页，也不是快表缺失。
- 易混对子：不要把页号和页内地址切错位数——4KB 是 2 的 12 次方，不是 2 的 10 次方；不要把页框号和页号拼错位置——物理地址前面是页框号，后面是页内地址。

正确选项为什么对： 页号 03FFFH 查表得页框号 0153H，拼上页内地址 180H，得 0153180H，就是 A，选 A。

错误选项为什么错＋举例：

- 页框号取错的：查表时对错了行，把别的页标记对应的页框号拿来了。教训：查表要按页号对行。
- 页内地址拼错的：页内地址切错了位数，或者拼接时位置放反了。教训：页内地址 12 位原样照搬，放在物理地址的后半段。
- 答“快表缺失”或者“缺页”的：查到了页表项，就不是缺失和缺页。教训：查到表项就正常翻译，只有查不到才是缺页。

笔记本一句话： 第 16 题＋分页地址翻译：4KB 页对应页内 12 位，页号查表得页框号，页框号＋页内地址＝物理地址。

【图像】



虚拟地址拆成页号与页内地址、查 TLB 得页框号再拼接

【定位】 计算机组成原理 · 第3章 存储系统 · 虚拟存储器（页式地址变换）

【易混对比】

- "页号"和"页框号"的区别：页号是虚拟地址里的高 20 位，页框号是查 TLB 查出来的物理页编号；物理地址=页框号+页内地址。
- "TLB 缺失"和"缺页"的区别：TLB 没命中还能继续查页表；页表里查不到才是缺页；本题一次命中，直接得结果。

页号 03FFFH 命中 TLB 得页框号 0153H，拼页内地址 180H 得 0153180H，选 A。

【坑位】

页内地址是 12 位（4KB=2 的 12 次方），切错位数后面全错。

错在把 4KB 当成 2 的 10 次方切 10 位，页号页内全错位。正确做法：4KB=2^12，低 12 位是页内地址 180H，高 20 位是页号 03FFFH。

【套路】

- 第一步，定页内位数：4KB=2^12，页内地址 12 位。
- 第二步，拆地址：03FFF180H→页号 03FFFH，页内地址 180H。
- 第三步，查表拼接：标记 03FFFH 的表项有效位为 1，页框号 0153H，拼得 0153180H。

定页内位数，拆页号页内，查表拼地址。

【类比】

像"小区第几栋第几号房"：页号是栋号，查表知道这栋楼实际在哪个位置（页框号），房号（页内地址）原样带上就是完整地址。术语贴回去：栋号对照表就是"页表/TLB"，楼的实际位置就是"页框号"。

【口诀】 页内十二拆地址，页号查表得框号，框号页内拼一起。

【考频】 「TLB」在 2009–2022 年解析中出现 11 次 · 高频

第 17 题 · 变址寻址求有效地址与操作数（正确答案：D）

题干与选项：

假设变址寄存器 R 的内容为 1000H，指令中的形式地址为 2000 H；地址 1000H 中的内容 为 2000H，地址 2000H 中的内容为 3000H，地址 3000 H 中的内容为 4000H，则变址寻址 方式下访问到的操作数是

- A. 1000H
- B. 2000H
- C. 3000H
- D. 4000 H

入手点： 变址寻址的公式就一句：有效地址＝变址寄存器内容＋形式地址；算出有效地址，再按这个地址去内存里取数。

专有名词从零解释：

- 先想“变址寻址”：一种寻址方式，有效地址不是直接写在指令里的，而是“变址寄存器的内容＋形式地址”算出来的。
- 再想“变址寄存器”：专门用来放“下标”的寄存器；程序循环时改它，就能顺序访问一组数。
- 再想“形式地址”：指令里直接给出的那部分地址，记成 A。
- 再想“有效地址”：真正用来访问内存的地址，记成 EA；操作数是按 EA 从内存里取出来的那个数。

考查知识点：

- 先想算 EA：变址寄存器内容是 1000H，形式地址是 2000H，EA＝1000H＋2000H＝3000H。
- 再想取操作数：按 EA＝3000H 去内存里取，取到的操作数是 4000H，就是 D。
- 再想为什么用变址寻址：数组、循环这种“一组数顺序访问”的场合，形式地址固定、变址寄存器每次加 1，就能一个一个往下取；改下标就能换元素。
- 边界情况：变址寄存器和形式地址相加时，超出地址范围的部分按地址位数截断（本题没有超出）；变址寻址和基址寻址的区别是：基址一般不变、变址常变（一个是“基地址”，一个是“下标”）。
- 易混对子：不要把 EA 当成操作数——3000H 是地址，不是数；不要把形式地址 2000H 当成操作数——2000H 只是指令里给的一部分地址；不要把变址寄存器内容 1000H 当成操作数——1000H 只是下标。

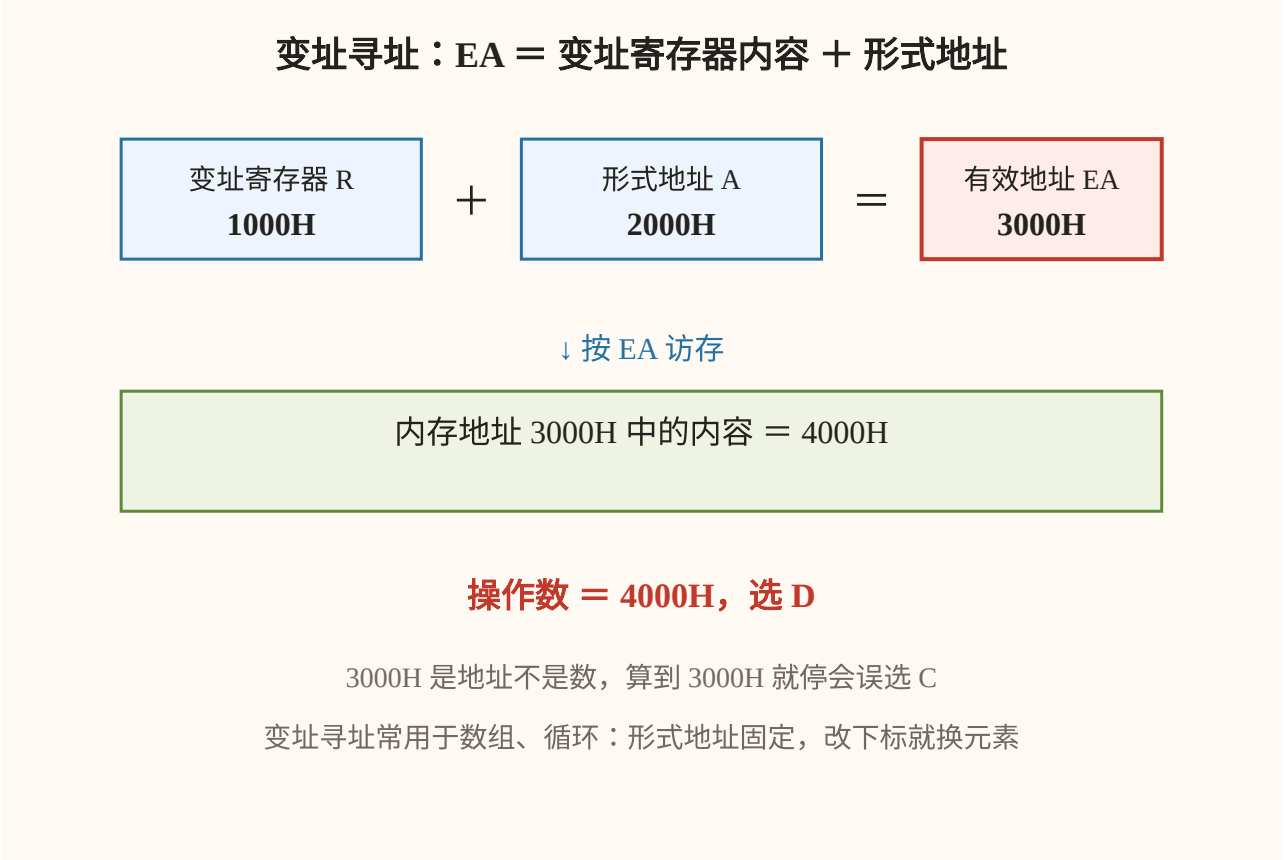
正确选项为什么对： EA＝1000H＋2000H＝3000H，按 3000H 取到的操作数是 4000H，就是 D，选 D。

错误选项为什么错＋举例：

- 1000H 的：拿了变址寄存器内容当操作数。教训：变址寄存器内容是用来算地址的，不是操作数本身。
- 2000H 的：拿了形式地址当操作数。教训：形式地址只是地址的一部分，要先和变址寄存器内容相加。
- 3000H 的：拿了有效地址当操作数。教训：有效地址是“去哪取”，操作数是“取到的数”，两者不一样。

笔记本一句话： 第 17 题＋变址寻址：EA＝（变址寄存器）＋（形式地址）＝3000H，访存得操作数 4000H。

【图像】



变址寄存器内容加形式地址得有效地址、再访存取操作数

【定位】 计算机组成原理 · 第4章 指令系统 · 寻址方式（变址寻址）

【易混对比】

- "有效地址"和"操作数"的区别：有效地址 3000H 是"去哪取"，操作数 4000H 是"取到的数"；3000H 是地址不是答案。
- "变址寄存器内容"和"形式地址"的区别：一个是寄存器里的下标 1000H，一个是指令里给出的地址部分 2000H；两者相加才得到有效地址。

EA=1000H+2000H=3000H，按 3000H 访存得操作数 4000H，选 D。

【坑位】

3000H 是地址不是数，算到 3000H 就停会误选 C。

错在把有效地址当成操作数。正确做法：算出 EA 后还要再访存一次，取到的 4000H 才是操作数。

【套路】

- 第一步，算有效地址：1000H+2000H=3000H。
- 第二步，访存取数：按 3000H 取，操作数是 4000H。

下标加形址得实址，实址访存才得数。

【类比】

像查字典：形式地址是"起始页码"，变址寄存器是"往后翻几页的偏移"，加起来得到真正的页码，再翻到那页看词条。术语贴回去：起始页码就是"形式地址"，偏移就是"变址寄存器内容"，真正的页码就是"有效地址"，词条就是"操作数"。

【口诀】 变址加形址，有效地址出；访存取操作，地址不是数。

【考频】 「变址寻址」在 2009–2022 年解析中出现 8 次 · 中频

第 18 题 · 4 级流水线执行 100 条指令的吞吐率（正确答案：C）

题干与选项：

某 CPU 主频为 1.03 GHz，采用 4 级指令流水线，每个流水段的执行需要 1 个时钟周期。 假定 CPU 执行了 100 条指令，在其执行过程中，没有发生任何流水线阻塞，此时流水线 的吞吐率为

- A. 0.25×10^9 条指令/秒
- B. 0.97×10^9 条指令/秒
- C. 1.0×10^9 条指令/秒
- D. 1.03 ×10^9 条指令/秒

入手点： 先算总共用了几个时钟周期，再用“指令数除以总时间”算吞吐率；总周期不是 100，也不是 400。

专有名词从零解释：

- 先想“流水线”：把一条指令的执行切成几段（比如取指、译码、执行、写回），像工厂流水线一样，让多条指令的不同段同时干活。
- 再想“时钟周期”：CPU 的节拍；主频 1.03GHz 就是每秒 10.3 亿个节拍。
- 再想“吞吐率”：单位时间里能完成多少条指令；本题问的是 100 条指令的平均吞吐率。

考查知识点：

- 先想总周期：第一条指令要流过 4 级，需要 4 个周期；后面每条指令每周期出来一条，一共 99 条；总周期＝4＋(100－1)＝103。
- 再想主频：1.03GHz 就是每秒 1.03G 个周期。
- 再想吞吐率：100 条指令用了 103 个周期，每秒 1.03G 个周期，所以每秒完成 1.03G×100÷103＝1.0×10^9 条指令，就是 C。
- 拿生活例子验算：洗车线有 4 道工序，第一辆车要过 4 道，后面每辆车每拍出来一辆；100 辆车一共 4＋99 拍；每拍出一辆，吞吐率就是“每秒拍数×100÷总拍数”。
- 边界情况：有阻塞时，总周期更多、吞吐率更低；稳态下（流水线填满后）每周期能出 1 条，但本题问的是 100 条的平均吞吐率，要用总周期算，不能直接用稳态。
- 易混对子：不要用 100 周期——漏了第一条指令流过 4 级的那 4 拍；不要用 400 周期——每条指令都算 4 拍，是没算重叠；不要直接答主频——主频是每秒周期数，不是每秒指令数。

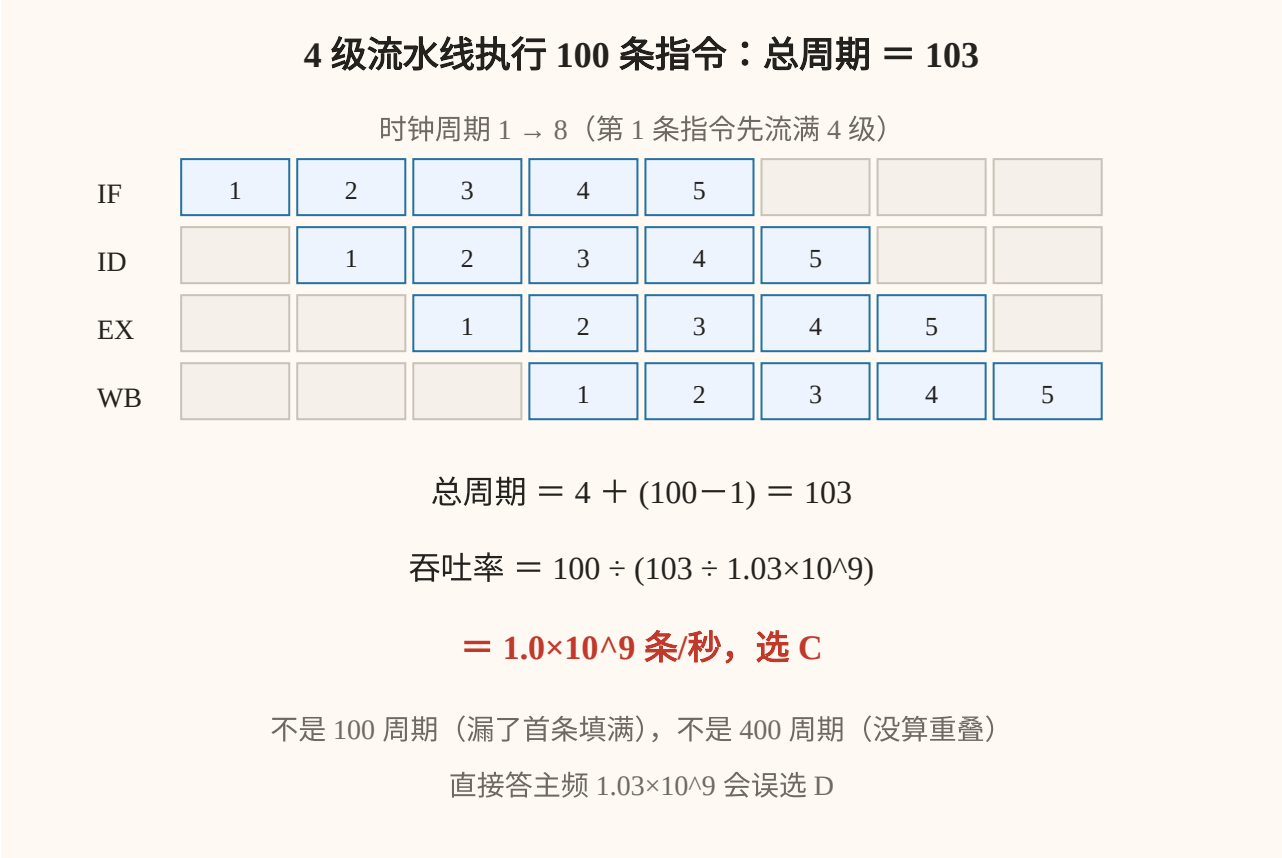
正确选项为什么对： 总周期 103，吞吐率＝1.03G×100÷103＝1.0×10^9 条／秒，就是 C，选 C。

错误选项为什么错＋举例：

- 算成 0.25×10^9 的：把 100 条都算成 4 拍，总周期算成了 400。教训：流水线是重叠执行的，不是每条都从头走 4 拍。
- 算成 0.97×10^9 的：总周期数算错了（比如少算或者多算了首条指令的级数）。教训：总周期＝级数＋（条数－1）。
- 算成 1.03×10^9 的：拿主频当成了吞吐率。教训：主频是每秒周期数，吞吐率是每秒指令数，要乘指令数、除总周期。

笔记本一句话： 第 18 题＋流水线吞吐率：总周期＝级数＋（条数－1）＝103，吞吐率＝主频×条数÷总周期。

【图像】



4 级流水线执行 100 条指令共 103 个周期的时空示意图

【定位】 计算机组成原理 · 第5章 中央处理器 · 指令流水线（吞吐率）

【易混对比】

- "总周期"和"稳态周期"的区别：100 条指令的总周期是 4+(100-1)=103；稳态下每周期出 1 条，但本题问 100 条的平均吞吐率，要用总周期算。
- "主频"和"吞吐率"的区别：主频 1.03GHz 是每秒的周期数，吞吐率是每秒的指令数；直接答主频会误选 D。

总周期 103，吞吐率=1.03G×100÷103=1.0×10^9 条/秒，选 C。

【坑位】

总周期不是 100 也不是 400；第一条指令要先流满 4 级。

错在漏算首条指令填满流水线的 4 个周期得 100，或把每条都算 4 拍得 400。正确做法：总周期=级数+(条数-1)=103，再用"主频×条数÷总周期"。

【套路】

- 第一步，算总周期：4+(100-1)=103。
- 第二步，列公式：吞吐率=指令条数÷总时间=100÷(103÷1.03G)。
- 第三步，化简：1.03G×100÷103=1.0×10^9 条/秒。

总周期等于级数加条数减一，吞吐率等于条数除以总时间。

【类比】

像 4 道工序的洗车线：第一辆车要过完 4 道，后面每拍出来一辆；100 辆车一共 4+99 拍。术语贴回去：每道工序就是"流水段"，每一拍就是"时钟周期"，每秒出几辆就是"吞吐率"。

【口诀】 流水总拍级加条减一，主频乘条数除总拍。

【考频】 「流水线」在 2009–2022 年解析中出现 4 次 · 中频

第 19 题 · USB 与系统局部总线的区分（正确答案：B）

题干与选项：

下列选项中，用于设备和设备控制器（I/O 接口）之间互连的接口标准是

- A. PCI
- B. USB
- C. AGP
- D. PCI-Express

入手点： 看接口连在谁和谁之间：连设备和控制器的，叫设备总线；连主存、显卡、网卡这些高速部件的，叫系统局部总线。

专有名词从零解释：

- 先想“总线”：计算机里各个部件之间传数据的公共通道；不同部件之间用不同的总线连。
- 再想“设备总线”：外部设备和设备控制器（I/O 接口）之间的接口标准；设备控制器是夹在设备和系统之间的那层。
- 再想“系统局部总线”：连接主存、显卡、网卡这些高速部件的总线；速度快，离 CPU 近。
- 再想“USB、PCI、AGP、PCI-E”：USB 是通用串行总线；PCI、AGP、PCI-E 都是系统局部总线的标准。

考查知识点：

- 先想 USB：USB 是连接外部设备和设备控制器的接口标准，属于设备总线，就是 B。
- 再想 PCI：PCI 是系统局部总线标准，用来连接主存、网卡、显卡这些部件，不是设备接口。
- 再想 AGP：AGP 是专门连接显卡的系统局部总线标准。
- 再想 PCI-E：PCI-E 是 PCI 的高速版，同样是系统局部总线标准。
- 拿生活例子验算：USB 像家里的插座，插鼠标、键盘、U盘这些外设；PCI、AGP、PCI-E 像房子里的主干道，连着各个房间（主存、显卡、网卡）。
- 边界情况：USB 也可以通过集线器接很多设备，但它仍然是设备总线；PCI-E 虽然速度很快，但按连接对象分，它仍然属于局部总线。
- 易混对子：不要按速度分——PCI-E 很快，但它连的是系统高速部件，不是设备；不要把“外部设备”和“高速部件”混为一谈——分类看的是连接对象，不是速度。

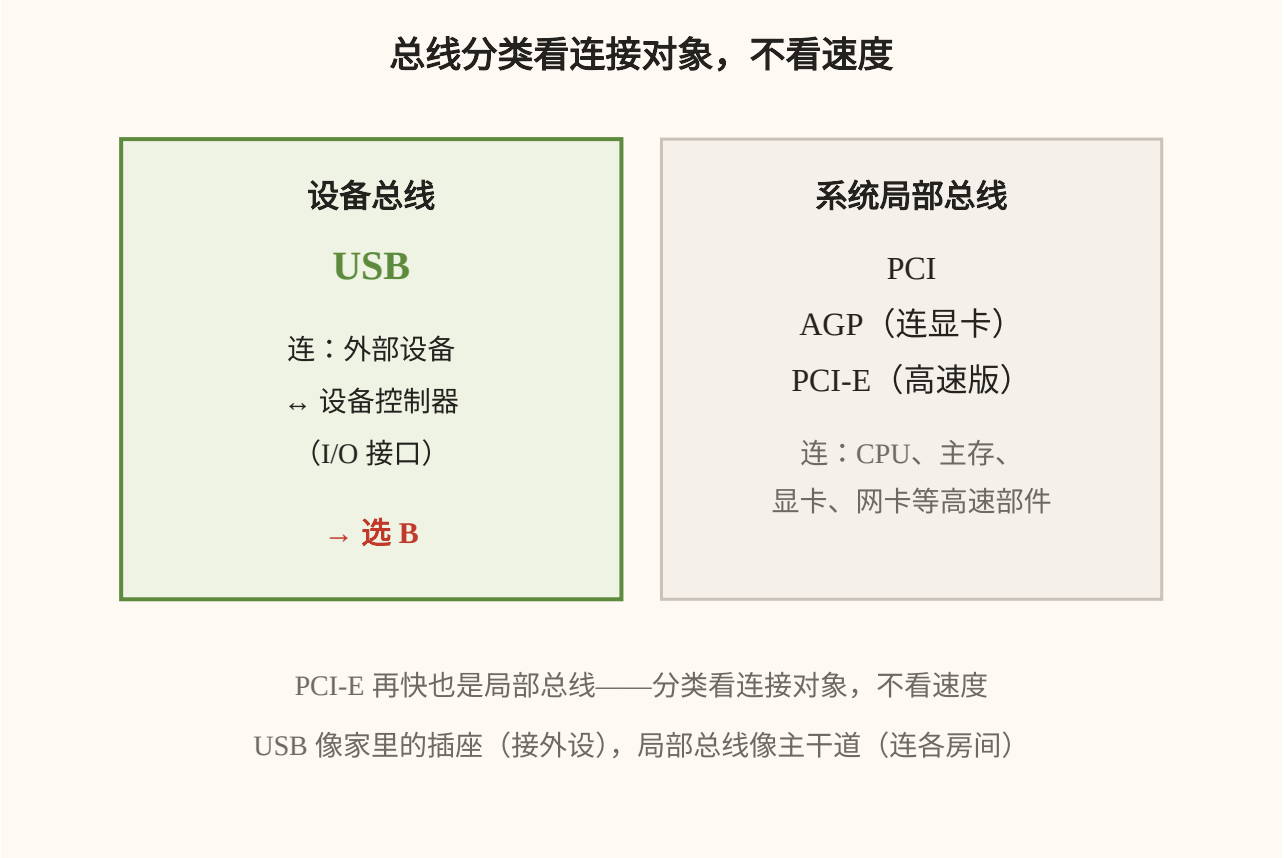
正确选项为什么对： USB 连外部设备和设备控制器，属于设备总线，就是 B，选 B。

错误选项为什么错＋举例：

- PCI 的：PCI 是系统局部总线，连主存、网卡、显卡。教训：PCI 不连外部设备。
- AGP 的：AGP 是专门连显卡的局部总线。教训：AGP 的对象是显卡，不是外部设备。
- PCI-E 的：PCI-E 是局部总线的高速版。教训：速度快不等于设备总线，分类看连接对象。

笔记本一句话： 第 19 题＋总线分类看连接对象：USB 接设备与控制器，PCI／AGP／PCI-E 接系统高速部件。

【图像】



设备总线与系统局部总线按连接对象分类对比

【定位】 计算机组成原理 · 第7章 输入/输出系统 · 总线标准

【易混对比】

- "设备总线"和"系统局部总线"的区别：看连在谁和谁之间；USB 连外部设备和设备控制器，PCI、AGP、PCI-E 连主存、显卡、网卡这些高速部件。
- "速度快"和"设备总线"的区别：PCI-E 很快，但它连的是系统高速部件，仍属局部总线；分类看连接对象，不看速度。

USB 连外部设备与设备控制器，属设备总线，选 B。

【坑位】

别按速度分类，PCI-E 再快也是局部总线。

错在觉得 PCI-E 快就归到设备接口。正确做法：只看连接对象——连外部设备和设备控制器的才是设备总线。

【类比】

USB 像家里的插座，插鼠标、键盘、U 盘这些外设；PCI、AGP、PCI-E 像房子里的主干道，连着各个房间（主存、显卡、网卡）。术语贴回去：插座就是"设备总线"，主干道就是"系统局部总线"。

【口诀】 分类不看速度看对象，USB 连外设，其余连高速部件。

【考频】 「总线」在 2009–2022 年解析中出现 25 次 · 高频

第 20 题 · 提高 RAID 可靠性的措施（正确答案：B）

题干与选项：

下列选项中，用于提高 RAID 可靠性的措施有 I. 磁盘镜像 II. 条带化 III. 奇偶校验 IV. 增加 Cache 机制

- A. 仅 I、II
- B. 仅 I、III
- C. 仅 I、III 和 IV
- D. 仅 II、III 和 IV

入手点： 可靠性就是“坏了还能恢复”；先把四个措施分成两类：加冗余的算，不加冗余的（只提速的）不算。

专有名词从零解释：

- 先想“RAID”：把多块磁盘组合起来用的技术；不同的组合方式（RAID 0、1、5 等）各有特点。
- 再想“磁盘镜像”：同样的数据存两份，分别放在两块盘上；一块坏了，另一块还有完整数据。
- 再想“奇偶校验”：多存一位校验信息；一块盘坏了，能用剩下的数据和校验位把坏的那块算回来。
- 再想“条带化”：把连续数据切成小块，分散存到多块盘上；多块盘可以同时干活，速度变快。
- 再想“Cache 机制”：用高速缓存暂存数据，读写变快。

考查知识点：

- 先想 I （磁盘镜像）：镜像是冗余，坏一块还有另一份，能提高可靠性。 I 正确。
- 再想 III（奇偶校验）：奇偶校验是冗余，坏一块能用校验位恢复，能提高可靠性。III正确。
- 再想 II （条带化）：条带化是把负载分摊、提速，不加冗余；一块盘坏了，它上面那部分数据就丢了，不能恢复。 II 错误。
- 再想 IV（增加 Cache 机制）：Cache 是提速，不加冗余；缓存里的数据丢了，磁盘上的数据还在，但可靠性没有提高。IV错误。
- 再想结论：只有 I 、III 能提高可靠性，组合就是 B。
- 拿生活例子验算：镜像像重要文件复印两份放两个抽屉；奇偶校验像多记一个总数，丢了一个数能用总数算回来；条带化像把一本书拆成几叠分给几个人看，看得快，但一个人丢了手里的那叠，书就不全了。
- 边界情况：RAID 0 只有条带化，没有冗余，所以不可靠；RAID 1 是镜像，RAID 5 是奇偶校验，这两个都可靠。
- 易混对子：不要把“提速”当成“可靠”——条带化和 Cache 都是提速，不加冗余；不要把“多块盘”当成“冗余”——多块盘分着存，不等于每份都有备份。

正确选项为什么对： 只有 I （镜像）和 III（奇偶校验）加了冗余，组合就是 B，选 B。

错误选项为什么错＋举例：

- A（仅 I 、 II）： II 条带化不加冗余。教训：条带化是提速，坏一块数据就丢。
- C（仅 I 、III、IV）： IV Cache 不加冗余。教训：Cache 是提速，不是备份。
- D（仅 II 、III、IV）： II 、IV都不加冗余，还漏了 I 。教训：提高可靠性只看冗余，提速的措施都不算。

笔记本一句话： 第 20 题＋RAID 可靠性只看冗余：镜像和奇偶校验算，条带化和 Cache 不算。

【图像】



镜像与奇偶校验加冗余、条带化与 Cache 只提速的对比

【定位】 计算机组成原理 · 第3章 存储系统 · 磁盘阵列（RAID）

【易混对比】

- "镜像"和"条带化"的区别：镜像是同样的数据存两份（加冗余），条带化是把数据切散存到多块盘（只提速）；坏一块盘时，镜像能恢复，条带化不行。
- "提速"和"可靠"的区别：条带化、Cache 都是提速，不加冗余；可靠性只看有没有多存一份能恢复的东西。

只有Ⅰ磁盘镜像和Ⅲ奇偶校验加了冗余，组合是 B（仅Ⅰ、Ⅲ）。

【坑位】

提高可靠性只看冗余，提速的措施一律不算。

错在把条带化、Cache 当成可靠性措施。正确做法：逐项问"坏一块盘能不能恢复"——能恢复的只有镜像和奇偶校验。

【类比】

镜像像重要文件复印两份放两个抽屉；奇偶校验像多记一个总数，丢了一个数能用总数算回来；条带化像把一本书拆成几叠分给几个人看，看得快但丢一叠书就不全了。术语贴回去：复印两份就是"磁盘镜像"，多记的总数就是"奇偶校验"，拆书分看就是"条带化"。

【口诀】 可靠只看冗余项，镜像奇偶算，条带缓存不算数。

【考频】 「RAID」在 2009–2022 年解析中出现 1 次 · 低频

第 21 题 · 磁盘访问总时间计算（正确答案：B）

题干与选项：

某磁盘的转速为 10 000 转/分，平均寻道时间是 6 ms，磁盘传输速率是 20 MB/s，磁盘控 制器延迟为 0.2 ms，读取一个 4 KB 的扇区所需的平均时间约为

- A. 9 ms
- B. 9.4 ms
- C. 12 ms
- D. 12.4 ms

入手点： 磁盘访问时间是四项加起来：寻道时间＋旋转等待时间＋传输时间＋控制器延迟；一项一项算，再相加。

专有名词从零解释：

- 先想“平均寻道时间”：磁头从当前位置移到目标磁道，平均要花的时间。
- 再想“平均旋转等待时间”：磁头到了目标磁道，还要等盘片转到目标扇区；平均等半圈。
- 再想“传输时间”：把数据从盘片上读出来，要花的时间，等于数据量除以传输速率。
- 再想“控制器延迟”：磁盘控制器处理这次读写，本身要花的时间。

考查知识点：

- 先想转一圈的时间：转速是 10000 转／分，一分钟 60 秒，所以转一圈是 60÷10000 秒＝6ms。
- 再想平均旋转等待：平均等半圈，就是 6ms 的一半＝3ms。
- 再想寻道时间：题目给的是 6ms。
- 再想传输时间：读 4KB，传输速率是 20MB／s，4KB÷20MB／s＝0.2ms。
- 再想控制器延迟：题目给的是 0.2ms。
- 再想总数：3＋6＋0.2＋0.2＝9.4ms，就是 B。
- 拿生活例子验算：找书的过程是：先走到书架那排（寻道），再等书转到你面前（旋转等待），再把书拿下来（传输），再登记一下（控制器延迟）；四步时间加起来就是总时间。
- 边界情况：平均旋转等待是半圈，不是整圈；传输时间算的时候单位要统一（KB 和 MB 都是 1024 进制约算，本题按 4÷20＝0.2ms 算）。
- 易混对子：不要把旋转等待算成整圈——平均是半圈；不要漏了控制器延迟——它虽然只有 0.2ms，但也要加；不要把传输时间的单位算错——4KB 除以 20MB／s 是 0.2ms，不是 0.2s。

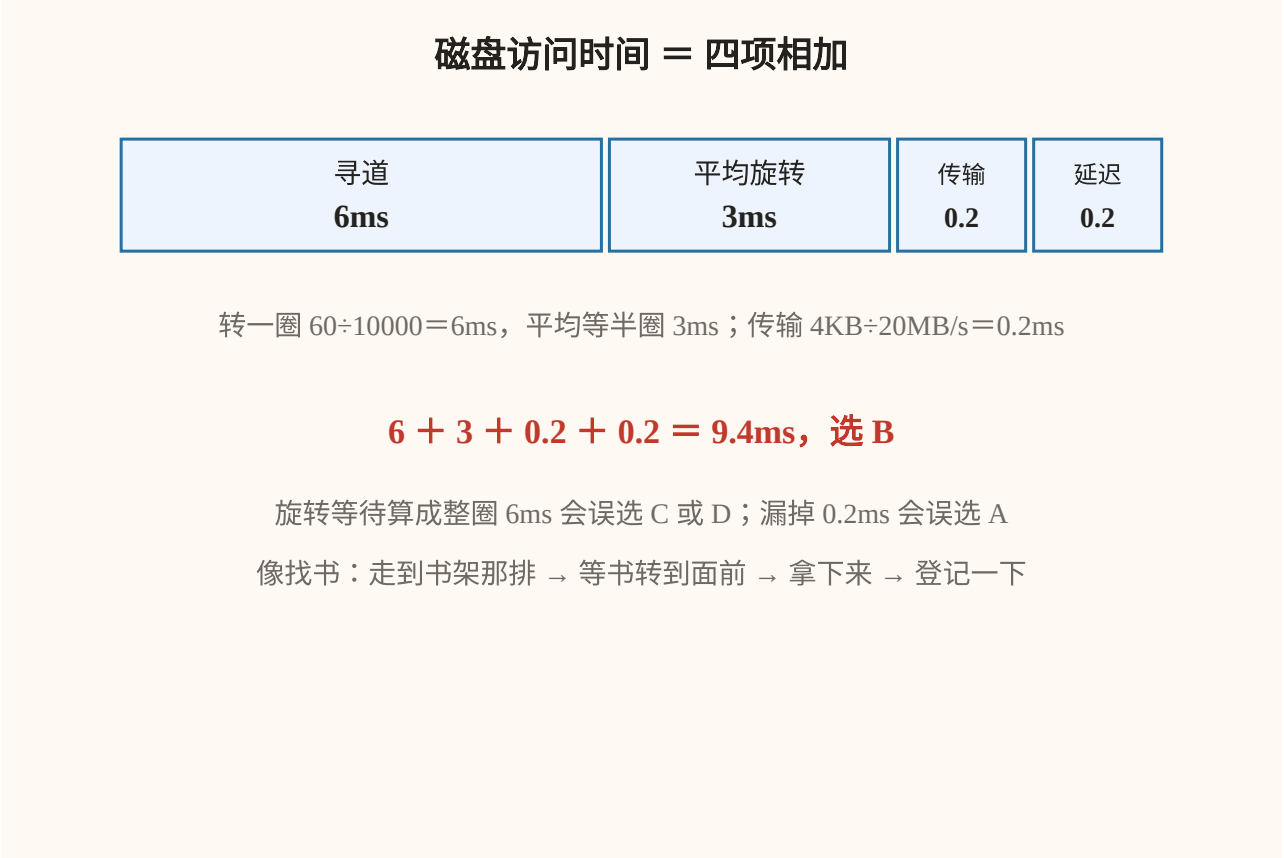
正确选项为什么对： 3＋6＋0.2＋0.2＝9.4ms，就是 B，选 B。

错误选项为什么错＋举例：

- 算成 9ms 的：漏了传输时间或者控制器延迟的 0.2ms。教训：四项都要加，少一项都不对。
- 算成 12ms 的：旋转等待算成了整圈 6ms。教训：平均旋转等待是半圈，是 3ms。
- 算成 12.4ms 的：旋转等待算成整圈，又全加起来。教训：半圈 3ms，不是 6ms。

笔记本一句话： 第 21 题＋磁盘访问时间：寻道＋半圈旋转＋传输＋控制器延迟＝3＋6＋0.2＋0.2＝9.4ms。

【图像】



寻道、旋转、传输、控制器延迟四项相加得 9.4ms

【定位】 计算机组成原理 · 第3章 存储系统 · 外部存储器（磁盘访问时间）

【易混对比】

- "平均旋转等待"和"转一圈"的区别：平均等半圈 3ms，不是整圈 6ms。
- "传输时间"和"寻道时间"的区别：传输时间=数据量÷传输速率=0.2ms，很小但不能漏；寻道时间题目直接给 6ms。

3+6+0.2+0.2=9.4ms，选 B。

【坑位】

平均旋转等待是半圈 3ms，算成整圈会误选 C 或 D。

错在旋转等待按整圈 6ms 算。正确做法：转一圈 60÷10000 秒=6ms，平均等半圈 3ms，四项全加。

【套路】

- 第一步，算旋转等待：转一圈 60÷10000 秒=6ms，平均等半圈 3ms。
- 第二步，算传输时间：4KB÷20MB/s=0.2ms。
- 第三步，四项相加：寻道 6+旋转 3+传输 0.2+控制器延迟 0.2=9.4ms。

寻道加半圈旋转加传输加延迟，四项一个不能少。

【类比】

像去书架找书：先走到那排（寻道），等书转到你面前（旋转等待），把书拿下来（传输），再登记一下（控制器延迟）。术语贴回去：走到那排就是"寻道时间"，等书转到面前就是"旋转等待时间"。

【口诀】 磁盘时间四项加，旋转只算半圈长。

【考频】 「磁盘」在 2009–2022 年解析中出现 56 次 · 高频

第 22 题 · 中断方式与 DMA 方式的区别（正确答案：D）

题干与选项：

下列关于中断 I/O 方式和 DMA 方式比较的叙述中，错误 的是

- A. 中断 I/O 方式请求的是 CPU 处理时间，DMA 方式请求的是总线使用权
- B. 中断响应发生在一条指令执行结束后，DMA 响应发生在一个总线事务完成后
- C. 中断 I/O 方式下数据传送通过软件完成，DMA 方式下数据传送由硬件完成 × ×
- D. 中断 I/O 方式适用于所有外部设备，DMA 方式仅适用于快速外部设备

入手点： 本题是选“错误”的那条，要反着看：对的三条是两种方式的真实区别，错的那条是把适用范围说反了。

专有名词从零解释：

- 先想“中断 I/O 方式”：设备每准备好一个数据，就向 CPU 发一次中断；CPU 停下手里的活，去处理这个数据，处理完再回去。
- 再想“DMA 方式”：直接存储器访问；设备和内存之间批量搬数据，由 DMA 控制器管着，CPU 只在开始和结束时插手。
- 再想“CPU 处理时间”：CPU 停下来处理中断所花的时间。
- 再想“总线使用权”：谁能用总线传数据；DMA 传数据时要先拿到总线使用权。

考查知识点：

- 先想申请对象：中断方式申请的是 CPU 处理时间（每次中断都要 CPU 停下来处理）；DMA 方式申请的是总线使用权（数据直接走总线进内存）。这条叙述正确。
- 再想响应时机：中断响应发生在一条指令执行结束之后；DMA 响应发生在一个总线事务完成之后。这条叙述正确。
- 再想数据传送主体：中断方式下数据传送通过软件完成（CPU 执行中断处理程序搬数据）；DMA 方式下数据传送由硬件完成（DMA 控制器搬数据）。这条叙述正确。
- 再想适用设备：中断方式适用于中低速设备，不适用于所有外部设备——高速设备每个数据都中断一次，会把 CPU 拖垮；DMA 方式适用于高速设备、批量传送。所以“中断方式适用于所有外部设备”这个说法错误，就是 D。
- 拿生活例子验算：中断像每来一个快递就敲一次门让你签收，快递来得太密，你就没法干别的活了；DMA 像快递员一次把一整箱货直接搬进仓库，你只在开始和结束时露个面。
- 边界情况：中断方式下，设备输入每个数据的过程中没有 CPU 干预，CPU 和 I/O 可以并行；但每传完一个数据，CPU 还是要花极短时间做中断处理。DMA 方式下，整块数据的传送在控制器控制下完成，CPU 只在块传送的开始和结束时干预。
- 易混对子：不要以为“中断方式”就万能——高速设备用中断会频繁打断 CPU；不要把“DMA 申请总线”和“中断申请 CPU 时间”弄反——一个是总线，一个是 CPU 时间。

正确选项为什么对： 本题选错误项，D 的“中断方式适用于所有外部设备”说法不对，就是 D，选 D。

错误选项为什么错＋举例：

- A（中断申请 CPU 时间、DMA 申请总线）：这条叙述正确，不是错误项。教训：申请对象是两种方式的核心区别，要记牢。

- B（中断响应在指令结束之后、DMA 响应在总线事务完成后）：这条叙述正确，不是错误项。教训：响应时机也是核心区别。
- C（中断下数据由软件传送、DMA 下数据由硬件传送）：这条叙述正确，不是错误项。教训：传送主体一个是软件、一个是硬件。

笔记本一句话： 第 22 题＋中断与 DMA：申请对象、响应时机、传送主体、适用设备四处对比；D 的“中断适用于所有设备”说法不对。

【图像】



中断与 DMA 在申请对象、响应时机、传送主体、适用设备四处对比

【定位】 计算机组成原理 · 第7章 输入/输出系统 · I/O 方式（中断与 DMA）

【易混对比】

- "申请 CPU 时间"和"申请总线使用权"的区别：中断每次都要 CPU 停下来处理，DMA 搬数据时只争总线、不长期占用 CPU。
- "中断响应"和"DMA 响应"的区别：中断响应在一条指令执行结束后，DMA 响应在一个总线事务完成后；时机不同。

本题选的是"错误"的叙述，D 说"中断适用于所有外部设备"不对，选 D。

【坑位】

本题选的是"错误"的叙述，别顺手选了正确的 A、B、C。

错在没看清"错误的是"三个字。正确做法：先逐条判对错——A、B、C 都对，D 把适用范围说反了（中断只适合中低速设备，高速设备频繁中断会拖垮 CPU），所以选 D。

【类比】

中断像每来一个快递就敲门让你签收，快递来得太密你就没法干活了；DMA 像快递员一次把整箱货直接搬进仓库，你只在开头和结尾露个面。术语贴回去：敲门签收就是"中断请求 CPU 处理时间"，整箱直搬就是"DMA 批量传送"，门被敲太多次就是"高速设备不适合中断方式"。

【口诀】 中断 CPU 时间忙，DMA 总线来帮忙；中断只宜中低速，高速批量 DMA 强。

【考频】 「DMA 方式」在 2009–2022 年解析中出现 7 次 · 中频

第 23 题 · 删除文件时应做哪些操作（正确答案：A）

题干与选项：

用户在删除某文件的过程中，操作系统不可能执行的操作是

- A. 删除此文件所在的目录
- B. 删除与此文件关联的目录项
- C. 删除与此文件对应的文件控制块
- D. 释放与此文件关联的内存级冲区

入手点： 删文件要清三样记录，但不能动文件所在的目录——目录里可能还有别的文件。

专有名词从零解释：

- 先想“目录项”：目录里的一条记录，记着文件名和这个文件的信息入口；找文件先找目录项。
- 再想“文件控制块”：文件的管理信息块，记着文件的位置、大小、权限这些信息。
- 再想“内存缓冲区”：文件数据在内存里的暂存区；文件打开时，数据可能缓存在这里。

考查知识点：

- 先想该清的第一样：删除这个文件对应的目录项，不然目录里还留着一个指向不存在文件的入口。
- 再想该清的第二样：删除这个文件的文件控制块，文件的管理信息也要清掉。
- 再想该清的第三样：释放这个文件关联的内存缓冲区，把内存腾出来。
- 再想不能动的一样：不能删除这个文件所在的目录——这个目录下可能还存在其他文件，删了目录，别的 file 也没了。
- 拿生活例子验算：删文件像退租一间房：要把门牌号摘掉（目录项）、把租房合同作废（文件控制块）、把房间里暂存的东西清走（缓冲区）；但不能把整栋楼拆了（目录），楼里还住着别人。
- 边界情况：删除目录和删除文件是两回事；删除目录要先保证目录是空的（或者递归删），不能顺手把非空目录删了。
- 易混对子：不要把“删除文件”当成“删除目录”——文件和目录是两级东西；不要以为删了目录项就完事了——文件控制块和缓冲区也要清。

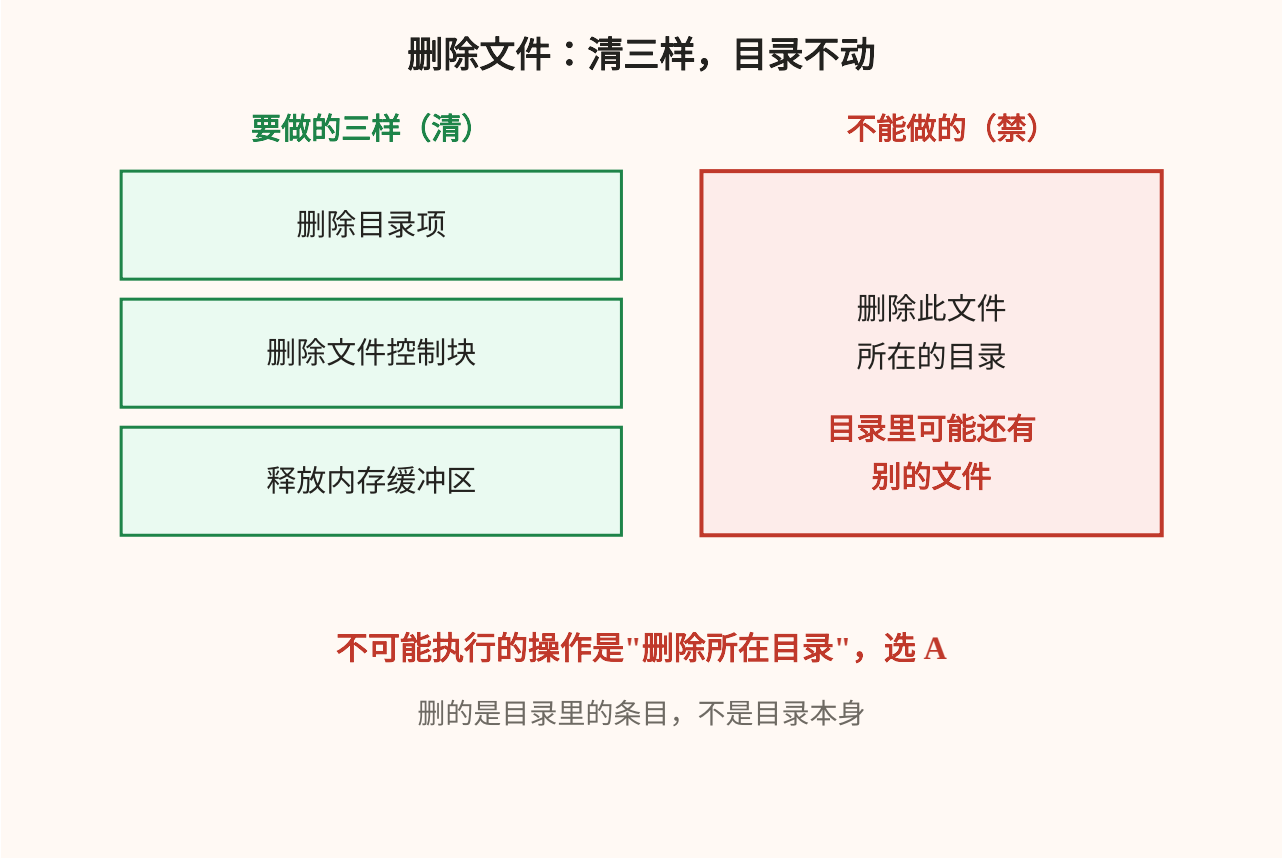
正确选项为什么对： 删目录项、删文件控制块、释放内存缓冲区，所在目录不动，就是 A，选 A。

错误选项为什么错＋举例：

- 含“删除所在目录”的：目录里可能还有别的 file，不能删。教训：删文件不动目录。
- 只删目录项的：文件控制块和内存缓冲区没清干净。教训：三样都要清，少一样都不完整。
- 不释放内存缓冲区的：内存白白占着。教训：缓冲区是资源，用完要释放。

笔记本一句话： 第 23 题＋删文件清三样：目录项、文件控制块、内存缓冲区；所在目录不动。

【图像】



删除文件时要清的三样记录和不能动的目录示意图

【定位】 操作系统 · 第4章 文件管理 · 删除文件操作

【易混对比】

- "删除文件"和"删除目录"的区别：删文件是清掉一个文件的记录，删目录是拆掉一整层容器；两者的对象层级不同，删文件不能顺手删目录。
- "目录项"和"目录"的区别：目录项是目录里的一条记录（记着文件名和文件入口），目录是装很多条目录项的容器；删的是条目，不是容器。

删文件清三样：目录项、文件控制块、内存缓冲区；所在目录不动。所以"不可能执行的操作"是 A。

【坑位】

看到"删除文件"，第一反应要排除"删除所在目录"。

目录里可能还有别的文件，删了目录别的 file 也没了；这是整题唯一的禁止动作，也是答案。错选别的选项，一般是把"删目录项"看成了"删目录"——一字之差，动作对象完全不同。正确做法：先把四个选项按"能不能做"分类，A 是禁区，直接锁定。

【类比】

删文件像退租一间房：摘掉门牌号（目录项）、把租房合同作废（文件控制块）、清走房间里暂存的东西（内存缓冲区）；但不能把整栋楼拆了（目录），楼里还住着别人。术语贴回去：门牌号就是"目录项"，租房合同就是"文件控制块"，暂存的东西就是"内存缓冲区"，整栋楼就是"目录"。

【口诀】 删文件清三样，目录项、控制块、缓冲区；目录不动别拆楼。

【考频】 「删除文件」在 2009–2022 年解析中出现 7 次 · 中频

第 24 题 · 为实现最快随机存取、选哪种文件物理结构（正确答案：A）

题干与选项：

为支持 CD-ROM 中视频文件的快速随机播放，播放性能最好的文件数据块组织方式是

- A. 连续结构
- B. 链式结构
- C. 直接索引结构
- D. 多级索引结钩

入手点： 随机存取要“算出地址直接到”；看哪种结构能直接算出地址，哪种要顺着找、哪种要多查一次表。

专有名词从零解释：

- 先想“文件物理结构”：文件在外存上实际怎么放；常见的有连续结构、链接结构、索引结构。
- 再想“连续结构”：文件的块在磁盘上连续存放；知道起始块号和逻辑块号，直接加一下就是物理块号。
- 再想“链接结构”：文件的块用指针串起来；要找第几块，得从第一块顺着指针一个一个找过去。
- 再想“索引结构”：另外有一张索引表，记着每块逻辑块号对应的物理块号；找块要先查索引表。
- 再想“随机存取”：不按顺序、直接跳到文件的任意位置读写；和“顺序存取”（从头到尾一个一个读）相对。

考查知识点：

- 先想连续结构：物理块号＝起始块号＋逻辑块号，直接算出来，一次定位，最快。
- 再想链接结构：要顺着指针一个一个找，找第 n 块要走 n 步，随机存取慢。
- 再想索引结构：要先查索引表，再按表里的物理块号去定位，比连续结构多一次访问。
- 再想结论：要保证最短查询时间，不能选链接结构和索引结构，连续结构最优，就是 A。
- 拿生活例子验算：连续结构像一排连号的储物柜，知道柜号直接走过去；链接结构像寻宝游戏，每个柜子里放着下一个柜子的钥匙，得一个一个找；索引结构像先查登记本、再去找柜子，多了一步。
- 边界情况：连续结构也有缺点——容易产生外碎片、文件长大后不好扩展；但本题只问“最快随机存取”，不问缺点。
- 易混对子：不要以为索引结构最快——索引要多查一次表；不要以为链接结构也能直接算——链接必须顺藤摸瓜。

正确选项为什么对： 连续结构地址直接算、一次定位、最快，就是 A，选 A。

错误选项为什么错＋举例：

- 链接结构的：必须顺着指针找，不能直接定位。教训：随机存取要直接算地址，顺指针太慢。
- 索引结构的：要先查索引表，多一次访问。教训：多一次查表，就不是最快。
- 顺序存取最快的：顺序存取和随机存取是两回事；本题问的是随机存取。教训：先看清题目问的是哪种存取。

笔记本一句话： 第 24 题＋随机存取最快是连续结构：地址直接算，不用顺指针、不用查表。

【图像】



连续、链接、索引三种文件物理结构随机存取方式对比示意图

【定位】 操作系统 · 第4章 文件管理 · 文件的物理结构

【易混对比】

- "连续结构"和"索引结构"的区别：连续直接算出物理块号，一次定位；索引要先查索引表，多一次访问。差的就是这一次查表。
- "连续结构"和"链接结构"的区别：连续能随机跳，链接只能顺指针从头找；找第 n 块，链接要走 n 步，随机存取慢。

随机存取要"算出地址直接到"：连续结构一次定位最快，选 A。

【坑位】

题眼是"随机播放"四个字：随机存取，不是顺序存取。

顺序存取的话链接结构也能用；但随机存取要跳到任意位置，链接得从头顺指针，索引得先查表，只有连续能直接算地址一次到位。错在把"顺序也行"当成"随机也行"。正确做法：先确认题型是随机还是顺序，随机直接找"能直接算地址"的那个。

【类比】

连续结构像一排连号的储物柜，知道柜号直接走过去；链接结构像寻宝游戏，每个柜子里装着下一个柜子的钥匙，得一个一个找；索引结构像先查登记本、再去找柜子，多了一步。术语贴回去：柜号就是"物理块号"，连号排列就是"连续结构"，寻宝找钥匙就是"链接结构"，登记本就是"索引表"。

【口诀】 随机存取连续快，地址直接算出来。

【考频】 「文件物理结构」在 2009–2022 年解析中出现 4 次 · 中频

第 25 题 · 计算磁盘号、磁头号、扇区号的工作由谁完成（正确答案：C）

题干与选项：

用户程序发出磁盘 I/O 请求后，系统的处理流程是：用户程序→系统调用处理程序→设备驱动程序→中断处理程序。其中，计算数据所在磁盘的柱面号、磁头号、扇区号的程序是（）。

- A. 用户程序
- B. 系统调用处理程序
- C. 设备驱动程序
- D. 中断处理程序

入手点： 磁盘号、磁头号、扇区号是“硬件相关”的活；谁懂具体硬件的几何结构，谁就干这个活。

专有名词从零解释：

- 先想“逻辑块号”：操作系统看文件的编号方式，不管磁盘长什么样，只管“第几块”。
- 再想“磁盘号、磁头号、扇区号”：磁盘的物理地址三要素——哪块盘、哪个磁头（哪一面）、哪个扇区。
- 再想“设备驱动程序”：厂家提供的、懂具体硬件的软件；同一种设备，不同厂家、不同型号的几何结构可能不一样。
- 再想“设备无关性”：操作系统里不依赖具体硬件的那部分；它只处理逻辑结构，不管硬件细节。

考查知识点：

- 先想翻译需要什么信息：要知道这块磁盘有几面、每面多少磁道、每道多少扇区；这些几何结构因设备而异。
- 再想谁知道这些信息：厂家提供的设备驱动程序知道具体设备的几何结构，所以翻译工作由它做，就是 C。
- 再想为什么不是别人：操作系统通用部分只处理逻辑块号，不管硬件细节；用户程序连硬件长什么样都不知道，更不可能做翻译。
- 拿生活例子验算：逻辑块号像“第几本书”，磁盘号、磁头号、扇区号像“第几排第几架第几格”；只有熟悉这个图书馆布局的管理员（驱动程序）才知道第几本书在第几排第几架。
- 边界情况：驱动程序是内核的一部分，但它是“厂家提供、针对具体设备”的那部分；设备无关的通用 I/O 软件只管逻辑块号。
- 易混对子：不要以为是文件系统做的——文件系统只管逻辑块号，不管物理几何；不要以为是用户程序做的——用户程序不碰硬件细节。

正确选项为什么对： 磁盘几何结构因设备而异，翻译工作由厂家提供的设备驱动程序做，就是 C，选 C。

错误选项为什么错＋举例：

- 用户程序的：用户程序不接触硬件细节。教训：用户程序只提“我要第几块”，不管这块在盘的哪个物理位置。
- 文件系统通用部分的：文件系统只处理逻辑结构，不管具体磁盘几何。教训：逻辑块号到物理地址的翻译是硬件相关的活。
- 中断处理程序的：中断处理管的是“设备好了通知 CPU”，不是地址翻译。教训：分清“通知”和“翻译”是两件不同的事。

笔记本一句话： 第 25 题＋逻辑块号→磁盘号／磁头号／扇区号是硬件相关的翻译，由厂家设备驱动程序做。

【图像】



磁盘 I/O 请求处理流程中地址翻译环节的位置示意图

【定位】 操作系统 · 第5章 输入/输出(I/O)管理 · 逻辑块号到物理地址的转换

【易混对比】

- "设备驱动程序"和"系统调用处理程序"的区别：驱动是厂家提供、懂具体硬件几何的；系统调用处理程序是操作系统通用部分，只管逻辑结构不管硬件细节。翻译是硬件相关的活，只能归驱动。
- "逻辑块号"和"物理地址"的区别：逻辑块号是操作系统看文件的编号（第几块），物理地址是磁盘的几何位置（哪块盘、哪个磁头、哪个扇区）。翻译就是把前者换成后者。

磁盘几何因设备而异，翻译工作由厂家提供的设备驱动程序做，选 C。

【坑位】

看到题干流程链别被带跑：翻译只在驱动层发生。

题干把"用户程序→系统调用处理程序→设备驱动程序→中断处理程序"摆出来，像是要按顺序选；但问的是"计算柱面号、磁头号、扇区号"，这是硬件相关翻译，只能是设备驱动程序。选系统调用处理程序是错把通用 I/O 软件当成懂硬件的；选用户程序是错把"提需求"当成"懂细节"——用户程序连磁盘长什么样都不知道；选中断处理程序是把"通知"和"翻译"混了。正确做法：问翻译，先想"谁懂硬件几何"，答案就是驱动。

【类比】

逻辑块号像"第几本书"，磁盘号、磁头号、扇区号像"第几排第几架第几格"；只有熟悉这个图书馆布局的管理员（设备驱动程序）才知道第几本书在第几排第几架。术语贴回去：管理员就是"设备驱动程序"，书架布局就是"磁盘几何结构"，找书格就是"地址翻译"。

【口诀】 逻辑转物理要翻译，硬件几何驱动知。

【考频】 「磁盘 I/O」在 2009–2022 年解析中出现 7 次 · 中频

第 26 题 · 哪个量与单个文件长度无关（正确答案：A）

题干与选项：

若某文件系统索引结点（inode）中有直接地址项和间接地址项，则下列选项中，与单个文件长度无关的因素是（）。

- A. 索引结点的总数
- B. 间接地址索引的级数
- C. 地址项的个数
- D. 文件块大小

入手点： 逐个看“文件变长时它变不变”；变长的量就是有关的，不变的量就是无关的。

专有名词从零解释：

- 先想“索引结点”：文件的元信息块，记着文件的大小、权限、地址项这些信息；一个文件对应一个索引结点。
- 再想“间接地址索引”：地址项不够用时，再套一层索引；一级间接、二级间接、三级间接，级数越高，能管的文件越长。
- 再想“地址项”：索引结点里记着文件块地址的那一项；文件越长，需要的地址项越多。
- 再想“文件块大小”：一块能装多少数据；块大小决定了每块的容量。

考查知识点：

- 先想 A（索引结点的总数）：索引结点总数就是文件的总数；单个文件变长，文件总数不变，所以索引结点总数和单个文件长度无关。A 正确。
- 再想 B（间接地址索引的级数）：文件越长，越可能用上多级间接索引；级数和文件长度有关。B 错误。
- 再想 C（地址项的个数）：文件越长，需要的地址项越多；个数和文件长度有关。C 错误。
- 再想 D（文件块大小）：文件块大小决定了每块能装多少数据，和文件长度的规模相关；不是与长度完全无关的量。D 错误。
- 拿生活例子验算：索引结点总数像“学生总数”，单个学生长多高，不影响学生总数；地址项个数像“每个学生要填的表格页数”，学生资料越多，表格页数越多。
- 边界情况：文件总数变化时，索引结点总数跟着变；但题目问的是“单个文件长度”，不是“文件总数”。
- 易混对子：不要把“索引结点总数”和“索引结点里的地址项个数”混为一谈——前者是文件个数，后者是单个文件的块数；不要以为“文件块大小”是固定不变的就和长度无关——块大小决定每块容量，和文件长度规模相关。

正确选项为什么对： 索引结点总数就是文件总数，和单个文件长度无关，就是 A，选 A。

错误选项为什么错＋举例：

- B（间接地址索引的级数）：文件越长，越可能用上多级间接索引。教训：级数是跟着文件长度走的。
- C（地址项的个数）：文件越长，需要的地址项越多。教训：地址项个数就是块数，文件长了块数就多。
- D（文件块大小）：块大小决定每块容量，和文件长度规模相关。教训：块大小不是与长度完全无关的量。

笔记本一句话： 第 26 题＋与单个文件长度无关的只有索引结点总数（＝文件总数）。

【图像】



文件变长时四个选项量是否变化的对比表

【定位】 操作系统 · 第4章 文件管理 · 索引结点(inode)

【易混对比】

- "索引结点的总数"和"地址项的个数"的区别：前者是文件个数（一个文件一个结点），后者是单个文件的块数。文件变长，块数变、文件个数不变。
- "间接地址索引的级数"和"地址项的个数"的区别：两者都跟着文件长度走——文件越长，越可能用上多级间接、需要更多地址项。但一个是"套了几层"，一个是"有多少项"。

只有索引结点总数（＝文件总数）和单个文件长度无关，选 A。

【坑位】

逐个看"文件变长时它变不变"，不变的才是答案。

B 间接级数：文件越长越可能用多级间接，跟着变；C 地址项个数：文件越长块数越多，跟着变；D 文件块大小：块大小决定每块容量，和文件长度规模相关。只有 A 索引结点总数＝文件总数，单个文件变长，文件总数不变。错在把"索引结点"里的地址项和"索引结点总数"混成一回事。正确做法：把四个量都放到"文件变长"的场景里逐个验，找那个不变的。

【类比】

索引结点总数像"学生总数"，单个学生长多高，不影响学生总数；地址项个数像"每个学生要填的表格页数"，学生资料越多，表格页数越多。术语贴回去：学生就是"文件"，表格页数就是"地址项"，学生总数就是"索引结点的总数"。

【口诀】 结点总数是文件数，单文件长它不变。

【考频】 「索引结点」在 2009–2022 年解析中出现 8 次 · 中频

第 27 题 · 外存数据块调入用户工作区与处理过程重叠时的总时间（正确答案：C）

题干与选项：

设系统缓冲区和用户工作区均采用单缓冲，从外设读入 1 个数据块到系统缓冲区的时间为 100，从系统缓冲区读入 1 个数据块到用户工作区的时间为 5，对用户工作区中的 1 个数据块进行分析的时间为 90（如下图所示）。进程从外设读入并分析 2 个数据块的最短时间是（ ）。

> 注：单缓冲示意图以原卷为准。

- A. 200
- B. 295
- C. 300
- D. 390

入手点： 画一条时间线，看哪段是串行的、哪段是并行的；并行的那段只算一次，取传输和处理的最大值。

专有名词从零解释：

- 先想“用户工作区”：用户程序在内存里干活的那片地方；数据块要先从外存调到这里，才能被处理。
- 再想“调入”：把数据块从外存搬到用户工作区的过程。
- 再想“处理”：用户程序对工作区里的数据块进行计算的过程。
- 再想“重叠”：调入和处理同时进行；一个数据块在处理时，另一个数据块在调入，两件事并行。

考查知识点：

- 先想第一段：数据块 1 从外存调入用户工作区，花 105；这段时间里数据块 2 还没有开始调入。
- 再想第二段：处理数据块 1 的同时，数据块 2 从外存调入（花 105）；这两件事是并行的，并行段取传输和处理的最大值，就是 105。
- 再想第三段：处理数据块 2，花 90。
- 再想总数：105＋105＋90＝300，就是 C。
- 再想关键提醒：并行段的 105 是“传输和处理的最大值”，不是“数据块 1 的处理时间＝105”；数据块 1 的处理时间没有单独给出，只要它不超过 105，并行段就是 105。
- 拿生活例子验算：做饭时，第一道菜下锅（调入）要 105 秒；第一道菜在炒（处理）的同时，第二道菜在备菜（调入），这两件事并行，只算 105 秒；最后炒第二道菜 90 秒；总共 105＋105＋90＝300 秒。
- 边界情况：如果数据块 1 的处理时间超过 105，并行段就是处理时间，总数会变大；如果调入和处理完全不能并行，总数就是 105＋处理 1＋105＋90。
- 易混对子：不要把并行段写成“数据块 1 处理时间＝105”——105 是并行段的最大值，不是处理时间的数值；不要把重叠的两段加两遍——并行就是只算一次；不要漏了数据块 2 的处理 90。

正确选项为什么对： 105（块 1 调入）＋105（并行段）＋90（块 2 处理）＝300，就是 C，选 C。

错误选项为什么错＋举例：

- 算成 390 的：某一段被重复算了一次。教训：并行段只算一次，调入、并行、处理三段各算一遍。
- 算成 210 的：漏了数据块 2 的处理，或者漏了一段调入。教训：三段都要算，调入、并行、处理一项都不能少。
- 算成 195 的：只算了单块流程，没算两块的重叠。教训：两块数据的时间线要连起来画。

笔记本一句话： 第 27 题＋重叠段取最大值：105（调入）＋105（并行段）＋90（处理块 2）＝300。

【图像】



单缓冲下两数据块调入与处理重叠的时间线示意图

【定位】 操作系统 · 第5章 输入/输出(I/O)管理 · 缓冲区与重叠处理

【易混对比】

- "传输时间"和"并行段"的区别：传输时间是单次调入的时长（105），并行段是"传输和处理同时进行的那段时间"；并行段的长度取两者最大值，本题是 105。
- "单缓冲"和"无重叠"的区别：单缓冲允许调入和处理同时进行；完全无重叠时，所有时间只能串行相加，总数更大。

105（块 1 调入）＋105（并行段）＋90（块 2 处理）＝300，选 C。

【坑位】

并行段 105 是"传输和处理的最大值"，不是"数据块 1 的处理时间＝105"。

数据块 1 的处理时间题干没有单独给出；只要它不超过 105，并行段就是 105。把 105 硬当成处理时间去列式子，逻辑就错了。另外两处常见漏项：漏掉数据块 2 的处理 90（时间线只画了一半），或者把并行段算两遍。正确做法：先画时间线，三段各算一次——串行调入、并行段取大、尾处理，相加。

【套路】

- 第一步，算块 1 的调入时间：外设→系统缓冲区 100，加系统缓冲区→用户工作区 5，得 105；这是时间线第一段，串行。
- 第二步，找重叠段并取最大值：块 1 在处理时，块 2 在调入，两者并行；并行段长度＝传输和处理的最大值＝105。

- 第三步，算块 2 的处理时间：时间线尾段，块 2 处理 90，没有别的动作和它并行。
- 第四步，三段相加：105＋105＋90＝300。

先画时间线，再看哪两段并行取大，其余串行相加。

【类比】

做饭时，第一道菜下锅备菜（调入）要 105 秒；炒第一道菜（处理）的同时，第二道菜在备菜（调入），这两件事并行，只算 105 秒；最后炒第二道菜 90 秒；总共 105＋105＋90＝300 秒。术语贴回去：备菜就是"调入"，炒菜就是"处理"，同时备菜和炒菜就是"重叠"。

【口诀】 调入处理若重叠，取大一段再相加。

【考频】 「单缓冲」在 2009–2022 年解析中出现 2 次 · 低频

第 28 题 · 哪些操作在系统内核态执行、哪些在用户态执行（正确答案：B）

题干与选项：

下列选项中，会导致用户进程从用户态切换到内核态的操作是（）。

I. 整数除以零

II. sin() 函数调用

III. read 系统调用

- A. 仅 I、II
- B. 仅 I、III
- C. 仅 II、III
- D. I、II 和 III

入手点： 看这个操作会不会碰到“只有内核能碰的东西”：会碰的进内核态，不会碰的留在用户态。

专有名词从零解释：

- 先想“内核态”：操作系统核心运行的状态；能执行特权指令、能碰硬件、能管内存和设备。
- 再想“用户态”：普通程序运行的状态；只能执行普通指令，不能直接碰硬件。
- 再想“系统调用”：用户程序请求内核服务的入口；用户程序想干内核才能干的事（比如读写文件），就通过系统调用请内核帮忙。
- 再想“中断、异常”：硬件或者程序出错时，CPU 自动切换到内核态去处理；处理完再回到用户态。

考查知识点：

- 先想 I （整数除以零）：整数除以零触发异常，CPU 转入中断处理程序处理，这个处理程序在内核态执行。 I 是内核态。
- 再想 III（read 系统调用）：read 是系统调用，执行时切换到内核态，由内核做实际的 I/O 操作。III是内核态。
- 再想 II （sin() 函数调用）：sin() 是库函数调用，纯算数计算，在用户态算完，不进内核。 II 是用户态。
- 再想结论：内核态的是 I 、III，组合就是 B。
- 拿生活例子验算：内核态像“后台机房”，只有工作人员能进；用户态像“前台大厅”，普通人只能在这里活动；想进机房办事，要通过系统调用这个“窗口”申请。
- 边界情况：库函数不等于系统调用——有的库函数里包着系统调用（比如 printf 里可能有 write），但 sin() 这种纯算数库函数不包系统调用；系统调用执行完，要回到用户态。
- 易混对子：不要把库函数调用都当成系统调用——sin() 是纯算数，不进内核；不要把“算数出错”当成用户态能处理的事——除零异常必须由内核的中断处理程序接管。

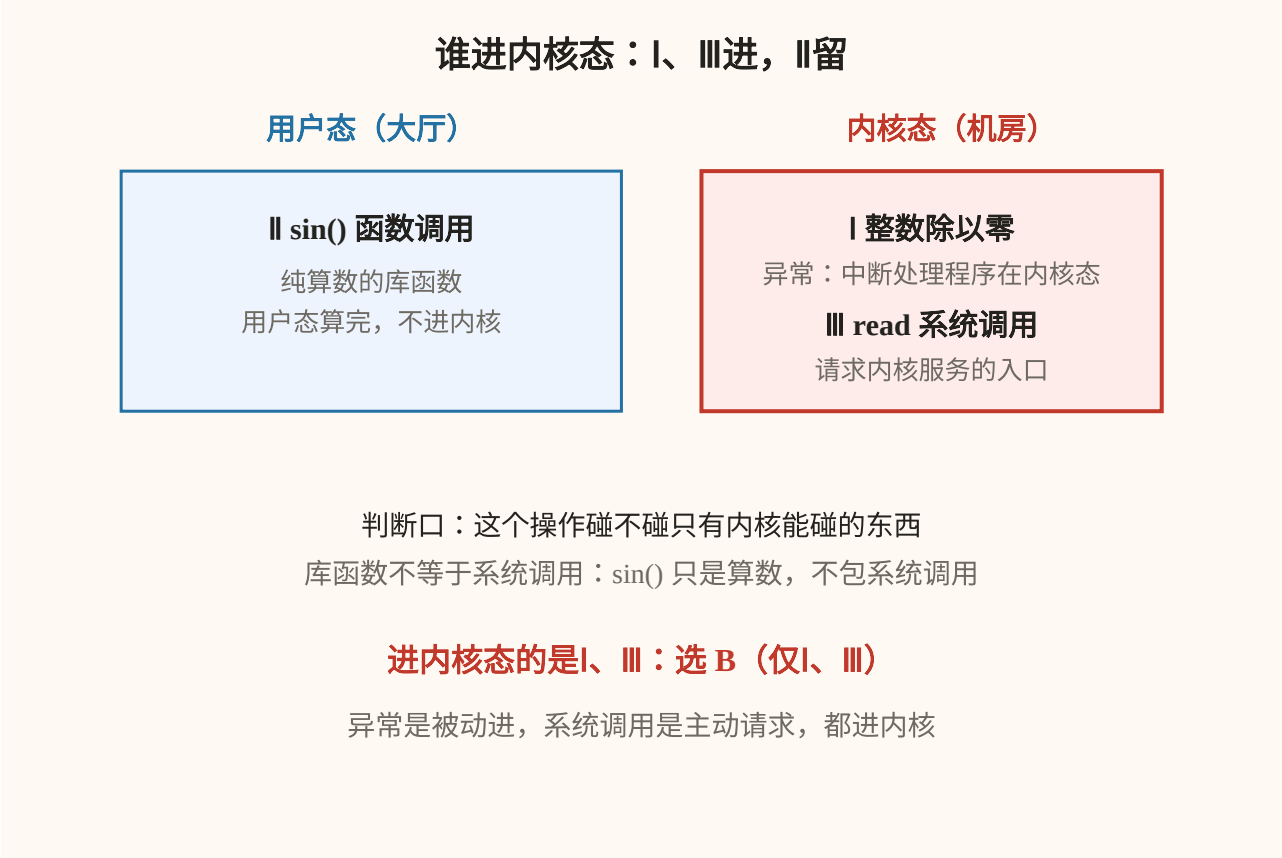
正确选项为什么对： 内核态的是 I （除零异常）和 III（read 系统调用），组合就是 B，选 B。

错误选项为什么错＋举例：

- A（仅 I 、 II ）： II sin() 不进内核。教训：sin() 是用户态库函数，不是系统调用。
- C（仅 II 、III）： II 不对，还漏了 I 。教训：除零异常也是内核态，不能漏。
- D（ I 、 II 、III）： II 不对。教训：三个里只有 I 、III 进内核， II 不进。

笔记本一句话： 第 28 题＋进内核态的：除零异常＋read 系统调用；sin() 是用户态库函数。

【图像】



用户态与内核态分界及三个操作归属示意图

【定位】 操作系统 · 第1章 操作系统概述 · 用户态与内核态

【易混对比】

- "库函数调用"和"系统调用"的区别：sin() 是纯算数的库函数，在用户态算完，不进内核；系统调用（如 read）是请求内核服务的入口，执行时切换到内核态。名字都带"调用"，去处不同。
- "异常"和"系统调用"的区别：异常（如整数除零）是被动触发，CPU 自动转入内核态的中断处理；系统调用是用户程序主动请求。被动和主动，都进内核。

进内核态的是 Ⅰ（除零异常）和Ⅲ（read 系统调用），Ⅱ sin() 留在用户态，组合是 B。

【坑位】

sin() 只是库函数，不是系统调用；别被"函数调用"四个字骗进内核态。

库函数里有的包着系统调用（如 printf 里可能有 write），但 sin() 这种纯算数函数不包。Ⅰ 整数除零触发异常，由内核的中断处理程序接管，必须进内核；Ⅲ read 是系统调用，进内核做实际 I/O。正确做法：逐个问"这个操作碰不碰只有内核能碰的东西"——碰的进内核，不碰的留在用户态。

【类比】

内核态像"后台机房"，只有工作人员能进；用户态像"前台大厅"，普通人只能在这里活动；想进机房办事，要通过系统调用这个"窗口"申请。sin() 是自己在大厅里算题，不用进机房。术语贴回去：机房就是"内核态"，大厅就是"用户态"，申请窗口就是"系统调用"。

【口诀】 除零异常读系统调用进内核，sin 纯算留在用户态。

【考频】 「用户态」在 2009–2022 年解析中出现 21 次 · 高频

第 29 题 · 系统开机后操作系统程序被加载到哪里（正确答案：D）

题干与选项：

计算机开机后，操作系统最终被加载到（ ）。

- A. BIOS
- B. ROM
- C. EPROM
- D. RAM

入手点： 开机时内存是空的，操作系统平时住在外存里；开机后要先把它从外存搬到内存里，CPU 才能执行它。

专有名词从零解释：

- 先想“引导程序”：开机后最先跑的一小段程序；它的任务就是把操作系统从外存加载到内存里。
- 再想“内存系统区”：内存里专门留给操作系统用的那片区域；和用户程序用的用户区相对。
- 再想“RAM”：内存，断电后数据就丢了；开机前内存是空的。
- 再想“外存”：硬盘这种断电后数据还在的存储器；操作系统平时就住在这里。

考查知识点：

- 先想开机顺序：开机后，引导程序先跑起来，把操作系统程序从外存加载到内存的系统区。
- 再想加载目标：操作系统程序被加载到内存中的系统区，这段区域是 RAM。
- 再想加载完之后：CPU 就在内存里执行操作系统了；之后用户程序也被陆续加载到内存的用户区执行。
- 拿生活例子验算：操作系统像一本工具书，平时放在书架上（外存）；开工前，先把工具书拿到工作台上（内存系统区），干活时随手就能翻。
- 边界情况：现代系统里，操作系统不是一次性全搬进内存，而是按需调页；但本题问的是“开机后被加载到哪里”，落点就是内存系统区。
- 易混对子：不要以为操作系统一直在内存里——开机前内存是空的；不要以为加载到外存——外存是它原来的家，加载是“从外存到内存”；不要以为加载到寄存器或者 Cache——那是执行时的暂存，不是程序的家。

正确选项为什么对： 开机后操作系统被加载到内存系统区（RAM），就是 D，选 D。

错误选项为什么错＋举例：

- 答外存的：外存是操作系统原来的存放处，不是加载目标。教训：加载是“从外存到内存”，方向不能反。
- 答寄存器或者 Cache 的：寄存器和 Cache 容量小，只是执行时的暂存。教训：程序的家是内存，寄存器和 Cache 只是临时放手头数据的。
- 答用户区的：用户区是给用户程序用的，操作系统住系统区。教训：内存里系统区和用户区是分开的。

笔记本一句话： 第 29 题＋开机后操作系统被加载到内存系统区（RAM）。

【图像】



开机后操作系统从外存加载到内存系统区的过程示意图

【定位】 操作系统 · 第1章 操作系统概述 · 操作系统引导

【易混对比】

- "加载到哪里"和"原来住哪里"的区别：加载是"从外存到内存"的方向，原来住的地方是外存；问的是目标，所以是内存。
- "RAM"和"ROM"的区别：RAM 是开机时空着、断电数据丢失的内存，程序加载进来才能运行；ROM 断电数据不丢，住的是固化好的引导程序，不是加载目标。

开机后引导程序把操作系统从外存搬到内存系统区（RAM），选 D。

【坑位】

开机前内存是空的：方向是外存→内存，别选反。

选外存是把"原来的家"当成了"加载目标"；选用户区是把内存的分区弄混了——操作系统住系统区，用户程序才住用户区；选寄存器或 Cache 是把执行时的暂存当成了程序的家。正确做法：先定方向（外存到内存），再定内存里的位置（系统区，RAM）。

【类比】

操作系统像一本工具书，平时放在书架上（外存）；开工前，先把工具书拿到工作台上（内存系统区），干活时随手就能翻。术语贴回去：书架就是"外存"，工作台就是"内存系统区"，拿过去的动作就是"加载"。

【口诀】 开机搬书到内存，系统区住在 RAM。

【考频】 「引导」在 2009–2022 年解析中出现 2 次 · 低频

第 30 题 · 用户程序访问内存时发生缺页中断、系统会做什么（正确答案：B）

题干与选项：

若用户进程访问内存时产生缺页，则下列选项中，操作系统可能执行的操作是 I. 处理越界错 II. 置换页 III. 分配内存

- A. 仅 I、II
- B. 仅 II、III
- C. 仅 I、III
- D. I、II 和 III

入手点： 缺页和越界是两回事：缺页是地址合法、页暂时不在内存；越界是地址本身非法。先分清，再看系统怎么做。

专有名词从零解释：

- 先想“缺页中断”：程序访问的页不在内存里，硬件触发的一种中断，通知系统“这页要从外存调进来”。
- 再想“页面置换”：内存满了，要把内存里暂时不用的页换出去，腾出地方给需要的页。
- 再想“越界”：访问的地址超出了程序合法的地址范围；这是程序出错，不是缺页。
- 再想“重新执行”：缺页处理完，CPU 重新执行那条产生缺页的指令，这次页已经在内存里了，能正常执行。

考查知识点：

- 先想缺页时系统做什么：系统执行的操作可能是置换页面（内存满了，换一页出去）或者分配内存（有空闲页框，直接分配），然后把需要的页从外存调进来。
- 再想为什么不是越界出错：系统里没有越界的错误——缺页的地址是合法的，只是页暂时不在内存；越界是地址本身非法，两者不一样。
- 再想处理完之后：重新执行产生缺页的那条指令，程序继续跑下去，不会终止。
- 拿生活例子验算：缺页像要看的书不在手边（在书架上），去书架上取回来（调页），继续看；越界像要看一本根本不存在的书，那才是出错。
- 边界情况：如果内存实在没有空闲页框，又换无可换，系统可能有其他处理；但本题的落点就是置换页面或者分配内存。
- 易混对子：不要把缺页当成越界——缺页地址合法；不要以为缺页就终止程序——缺页是正常机制，处理完继续执行。

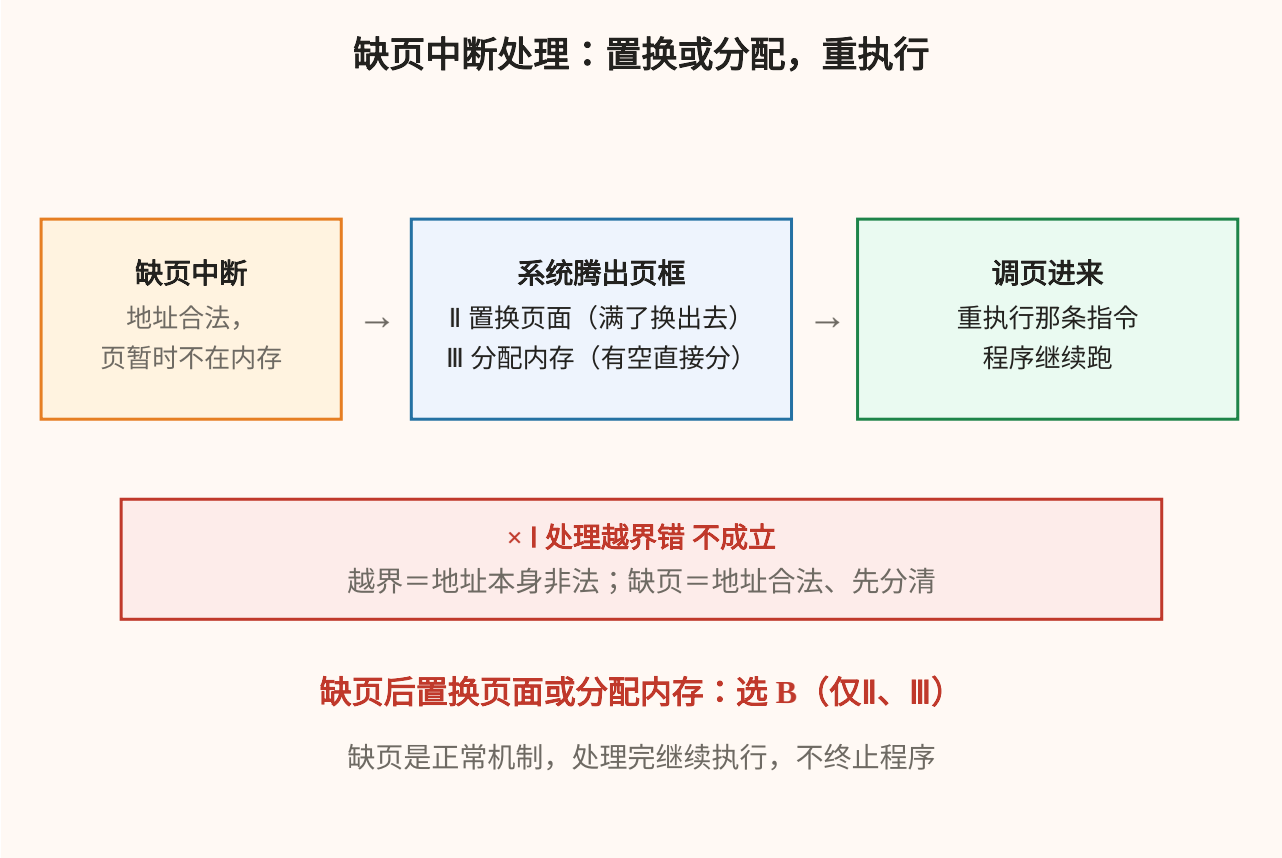
正确选项为什么对： 缺页中断后系统置换页面或者分配内存，把页调进来，就是 B，选 B。

错误选项为什么错＋举例：

- 越界出错处理的：缺页不是越界，地址合法。教训：先分清“页不在内存”和“地址非法”。
- 终止程序的：缺页处理完重新执行指令，程序继续跑。教训：缺页是正常机制，不是致命错误。
- 什么都不做的：缺页不处理，指令就执行不下去。教训：缺页必须由系统处理完才能继续。

笔记本一句话： 第 30 题＋缺页中断→置换页面或分配内存→重执行指令；不是越界出错。

【图像】



缺页中断发生后操作系统处理流程示意图

【定位】 操作系统 · 第3章 内存管理 · 缺页中断处理

【易混对比】

- "缺页"和"越界"的区别：缺页是地址合法、页暂时不在内存（Ⅰ不成立）；越界是地址本身非法。缺页是正常机制，越界才是程序出错。
- "置换页"和"分配内存"的区别：内存满了就把暂时不用的页换出去（置换），有空闲页框就直接分配（分配）；两种都是"腾出地方把页调进来"的办法。

缺页后系统置换页面或分配内存，把页调进来，重执行指令；没有越界出错处理，组合是 B。

【坑位】

先分清缺页还是越界：地址合法、只是页不在内存，这是缺页，不是越界。

Ⅰ 处理越界错不成立——缺页的地址合法；Ⅱ 置换页和Ⅲ 分配内存都可能，看内存有没有空闲页框。错选含 Ⅰ 的选项，是把"页不在内存"当成了"地址非法"。正确做法：第一步先判断地址合不合法，合法→缺页路线（置换或分配＋调页＋重执行），非法→越界路线。

【套路】

- 第一步，判断地址合法性：缺页的地址合法，只是页暂时不在内存；越界的地址本身非法。先定性，再定处理。
- 第二步，系统腾出页框：内存满了就置换页面（Ⅱ），有空闲页框就分配内存（Ⅲ），把需要的页从外存调进来。
- 第三步，重执行指令：调页完成后，CPU 重新执行产生缺页的那条指令，这次页已在内存，正常执行，程序继续跑。

合法→缺页路线：置换或分配，调页，重执行。

【类比】

缺页像要看的书不在手边（在书架上），去书架上取回来（调页），继续看；越界像要看一本根本不存在的书，那才是出错。术语贴回去：手边就是"内存"，书架就是"外存"，取书就是"调页"，根本不存在的书就是"越界"。

【口诀】 缺页置换或分配，重执指令接着跑。

【考频】 「缺页中断」在 2009–2022 年解析中出现 14 次 · 高频

第 31 题 · 合理设置进程优先级的原则（正确答案：B）

题干与选项：

某系统正在执行三个进程 P1、P2 和 P3，各进程的计算（CPU）时间和 I/O 时间比例如下表所示。

| 进程 | 计算时间 | I/O 时间 |

|---|---|---|

| P1 | 90% | 10% |

| P2 | 50% | 50% |

| P3 | 15% | 85% |

为提高系统资源利用率，合理的进程优先级设置应为（）。

- A. P1>P2>P3
- B. P3>P2>P1
- C. P2>P1=P3
- D. P1>P2=P3

入手点： 优先级要看“谁等 I/O、谁占 CPU”：让 I/O 设备早点开工，系统资源利用率才高。

专有名词从零解释：

- 先想“I/O 密集型进程”：经常要等 I/O（比如读写磁盘、等键盘输入），CPU 用得少，一算完就要等 I/O。
- 再想“CPU 密集型进程”：埋头计算，很少等 I/O，CPU 一直被它占着。
- 再想“优先级”：调度程序决定先让谁用 CPU 的依据；优先级高的先被调度。
- 再想“系统资源利用率”：CPU 和 I/O 设备是不是都在忙、有没有空转。

考查知识点：

- 先想原则：进程的 CPU 时间和 I/O 时间要综合考虑；对于 CPU 占用时间较少、I/O 占用时间较多的进程（I/O 密集型），应该给更高的优先级。
- 再想为什么：I/O 密集型进程算一会儿就要等 I/O；早点调度它，I/O 设备就能早点开工，CPU 和 I/O 并行度高，系统资源利用率高。
- 再想反过来：如果给 CPU 密集型进程更高优先级，它一直占着 CPU，I/O 设备就在旁边空转，利用率低。
- 拿生活例子验算：两个工人，一个老要等材料（I/O 密集型），一个埋头干活不用等材料（CPU 密集型）；先让等材料的工人开工，材料一到他就能干，两个人都不闲着；反过来，先让埋头干活的工人占着机器，等材料的工人就只能干等。
- 边界情况：优先级不是“谁重要谁高”——本题讲的是调度效率原则；优先级太低的进程可能饿死，要用老化等机制缓解，这是进阶情形。
- 易混对子：不要以为 CPU 用得多的优先级高——那样 I/O 设备会空转；不要以为优先级只看“谁先到”——先来先服务是另一种算法，不是优先级原则。

正确选项为什么对： CPU 少、I/O 多的进程优先级更高，让 I/O 早开工、资源利用率高，就是 B，选 B。

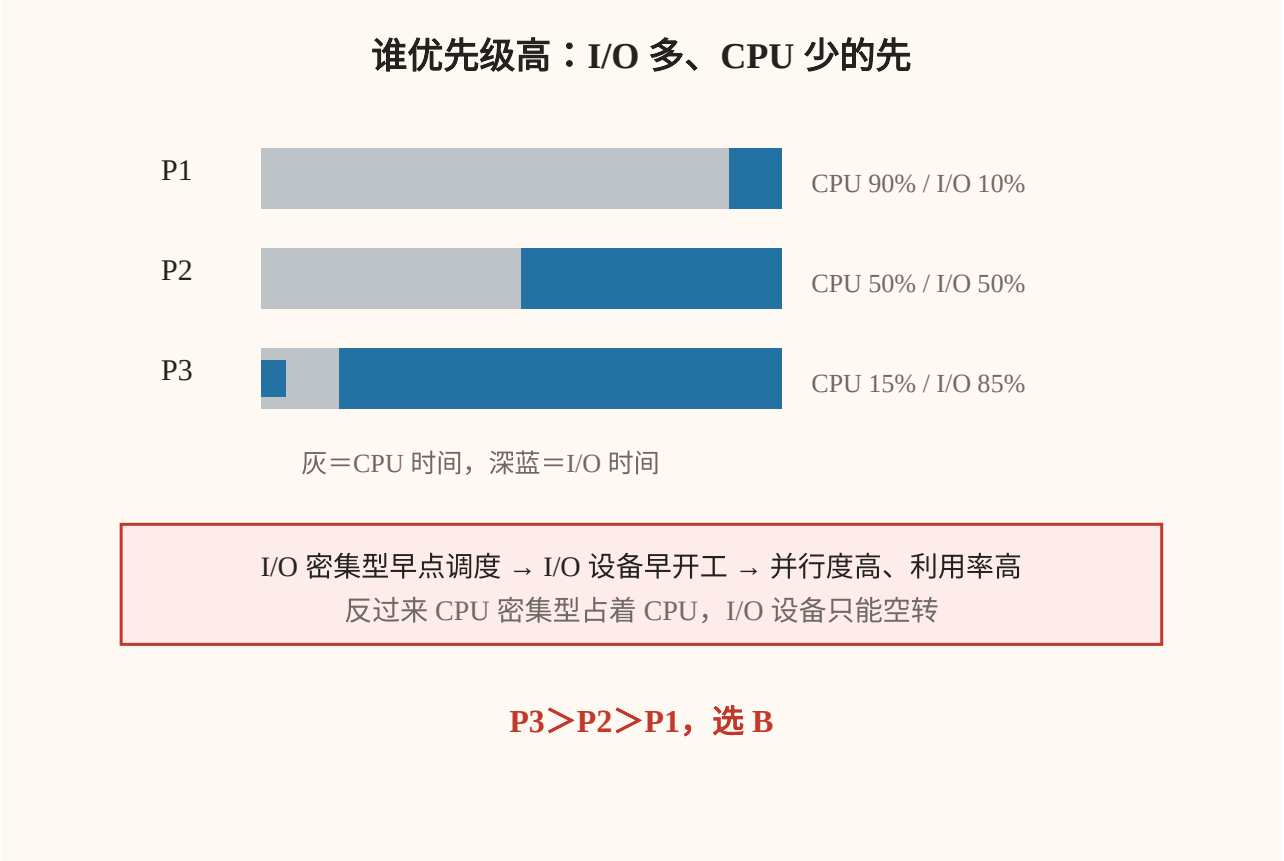
错误选项为什么错＋举例：

- 给 CPU 密集型更高优先级的：I/O 设备空转，利用率低。教训：CPU 密集型占着 CPU，I/O 设备没活干。
- 只看到达顺序的：那是先来先服务，不是优先级原则。教训：优先级原则看的是 CPU 时间和 I/O 时间的综合。

- 只看 CPU 时间的：漏了 I/O 时间。教训：CPU 时间和 I/O 时间要综合考虑。

笔记本一句话： 第 31 题＋I/O 多、CPU 少的进程优先级更高，让 I/O 早开工、资源利用率高。

【图像】



三个进程 CPU 时间与 I/O 时间比例及优先级排序示意图

【定位】 操作系统 · 第2章 进程管理 · 进程调度优先级

【易混对比】

- "I/O 密集型"和"CPU 密集型"的区别：前者 CPU 用得少、老等 I/O（P3：15%/85%），后者埋头计算很少等 I/O（P1：90%/10%）。优先级给的是前者。
- "优先级高"和"先到先得"的区别：先来先服务看到达顺序，优先级原则看 CPU 时间和 I/O 时间的综合；本题问的是优先级，别按到达顺序排。

I/O 多、CPU 少的进程优先级更高：P3＞P2＞P1，选 B。

【坑位】

别把优先级给 CPU 用得多的那个：P1 一直占着 CPU，I/O 设备只能在旁边空转。

P3 算一会儿就要等 I/O，早点调度它，I/O 设备早开工，CPU 和 I/O 并行度高，系统资源利用率才高。反过来先让 P1 占着 CPU，I/O 设备没活干，利用率低。正确做法：逐个看"谁等 I/O 多"，等得越多优先级越高；再按排序写出 P3＞P2＞P1。

【套路】

- 第一步，判断每进程的类型：看 CPU 时间和 I/O 时间的比例——P1 是 CPU 密集型（90%/10%），P2 居中（50%/50%），P3 是 I/O 密集型（15%/85%）。

- 第二步，按"I/O 多优先"排序：I/O 密集型先调度，让 I/O 设备早开工，得 P3＞P2＞P1，对应选项 B。

谁等 I/O 多，谁的优先级高；按 I/O 比例从大到小排。

【类比】

两个工人，一个老要等材料（I/O 密集型），一个埋头干活不用等材料（CPU 密集型）；先让等材料的工人开工，材料一到他就能干，两个人都不闲着；反过来，先让埋头干活的工人占着机器，等材料的工人只能干等。术语贴回去：材料就是"I/O"，机器就是"CPU"，先开工就是"高优先级"。

【口诀】 CPU 少、I/O 多，优先级先给它。

【考频】 「进程优先级」在 2009–2022 年解析中出现 16 次 · 高频

第 32 题 · 银行家算法与安全状态、死锁的关系（正确答案：B）

题干与选项：

下列关于银行家算法的叙述中，正确的是

- A. 银行家算法可以预防死锁
- B. 当系统处于安全状态时，系统中一定无死锁进程
- C. 当系统处于不安全状态时，系统中一定会出现死锁进程
- D. 银行家算法破坏了死锁必要条件中的“请求和保持”条件

入手点： 安全状态、不安全状态、死锁是三档，不是两档：安全一定没死锁，不安全只是可能死锁。

专有名词从零解释：

- 先想“银行家算法”：分配资源前先试算的算法；试算一下“这次分配后，系统是不是还安全”，安全才分配，不安全就不分配。
- 再想“安全状态”：存在一个安全序列，能让所有进程按这个顺序一个一个拿到资源、跑完；系统处于安全状态，就一定不会死锁。
- 再想“不安全状态”：不存在安全序列；系统可能死锁，但不是一定死锁。
- 再想“死锁”：几个进程互相等对方手里的资源，谁都跑不下去。
- 再想“预防死锁”：破坏死锁产生的必要条件（互斥、占有等待、非抢占、循环等待），是另一种办法，和银行家算法不一样。

考查知识点：

- 先想银行家算法是干什么的：银行家算法是避免死锁的方法——分配前动态试算，不安全就不分配。
- 再想安全状态：系统处于安全状态时，就可以避免死锁，也就是此时必然没有死锁。
- 再想不安全状态：系统进入不安全状态后，便可能进入死锁状态，但不是必然——不安全只是“不存在安全序列”，进程实际运行中可能并不真的互相卡死。
- 再想结论：安全→必然无死锁；不安全→只是可能死锁，不是必然。
- 拿生活例子验算：安全状态像手里还留着足够的周转资金，怎么借都还得上；不安全状态像周转资金不够了，可能还得上、也可能卡住，但还没卡住。
- 边界情况：预防死锁是破坏必要条件，是静态的办法；避免死锁（银行家算法）是动态试算；检测与解除是事后处理；三者不要混为一谈。
- 易混对子：不要把“不安全”等同于“死锁”——不安全只是可能；不要把“避免”和“预防”混为一谈——避免是动态试算，预防是静态破条件。

正确选项为什么对： 安全状态必然无死锁，不安全状态只是可能死锁，就是 B，选 B。

错误选项为什么错＋举例：

- 不安全必然死锁的：不安全只是可能，不是必然。教训：三档要分清，安全、不安全、死锁不是两档。
- 安全也可能死锁的：安全状态的定义就是能全部跑完，不可能死锁。教训：安全序列存在，死锁就不会发生。
- 把银行家算法说成预防死锁的：银行家算法是避免死锁（动态试算），预防是破坏必要条件。教训：避免和预防是两种办法。

笔记本一句话： 第 32 题＋安全→必然无死锁；不安全→只是可能死锁，不是必然。

【图像】



安全状态、不安全状态、死锁三档关系与银行家算法定位示意图

【定位】 操作系统 · 第2章 进程管理 · 死锁的避免：银行家算法

【易混对比】

- "安全状态"和"不安全状态"的区别：安全是有安全序列、必然无死锁；不安全是无安全序列、只是可能死锁。差的是"必然"和"可能"。
- "避免死锁"和"预防死锁"的区别：避免是银行家算法，分配前动态试算；预防是静态破坏死锁必要条件。一种是动态算，一种是静态破。

安全→必然无死锁；不安全→只是可能死锁，不是必然。选 B。

【坑位】

不安全不等于死锁：三档要分清，安全、不安全、死锁不是两档。

A 说银行家算法预防死锁——银行家是避免（动态试算），预防是破坏必要条件，两词不能换；C 说不安全一定死锁——不安全只是可能，进程实际运行可能并不卡死；D 说银行家破坏"请求和保持"——破坏条件的是预防，不是避免。正确做法：先记住"安全必无死锁，不安全只可能"，再逐个验选项。

【类比】

安全状态像手里还留着足够的周转资金，怎么借都还得上；不安全状态像周转资金不够了，可能还得上、也可能卡住，但还没卡住。术语贴回去：周转资金就是"可分配资源"，还得上就是"存在安全序列"，卡住就是"死锁"。

【口诀】 安全必无死锁，不安全只可能。

【考频】 「银行家算法」在 2009–2022 年解析中出现 13 次 · 高频

第 33 题 · OSI 参考模型中应用层相邻下层是哪一层（正确答案：B）

题干与选项：

在 OSI 参考模型中，下列功能需由应用层的相邻层实现的是

- A. 对话管理
- B. 数据格式转换
- C. 路由选择
- D. 可靠数据传输

入手点： 从上往下数七层，应用层是第一层，它下面那层就是答案；数的时候别跳层。

专有名词从零解释：

- 先想“OSI 参考模型”：把网络通信分成七层的模型；从上到下依次是应用层、表示层、会话层、传输层、网络层、数据链路层、物理层。
- 再想“应用层”：最上面那层，直接给用户用的那层（比如浏览器、邮件客户端都在这一层）。
- 再想“表示层”：第六层，管数据长什么样：数据字符转换、数据格式化、文本压缩、数据加密和解密。
- 再想“相邻下层”：紧挨着的下一层；应用层的相邻下层，就是从上往下数的第二层。

考查知识点：

- 先想七层顺序：应用层（第 7 层）、表示层（第 6 层）、会话层（第 5 层）、传输层（第 4 层）、网络层（第 3 层）、数据链路层（第 2 层）、物理层（第 1 层）。
- 再想应用层的下层：应用层下面就是表示层。
- 再想表示层干什么：表示出用户看得懂的数据格式，实现和数据表示有关的功能，主要完成数据字符的转换、数据格式化和文本压缩、数据加密和解密。
- 拿生活例子验算：应用层像“说话的人”，表示层像“翻译”，说话的人说完，翻译先把话变成对方听得懂的样子，再往下传。
- 边界情况：TCP/IP 模型没有表示层——应用层直接把表示层的活也干了；但本题问的是 OSI 模型，要按七层数。
- 易混对子：不要答会话层——会话层是表示层的下一层，不是应用层的下一层；不要答传输层——传输层隔了两层。

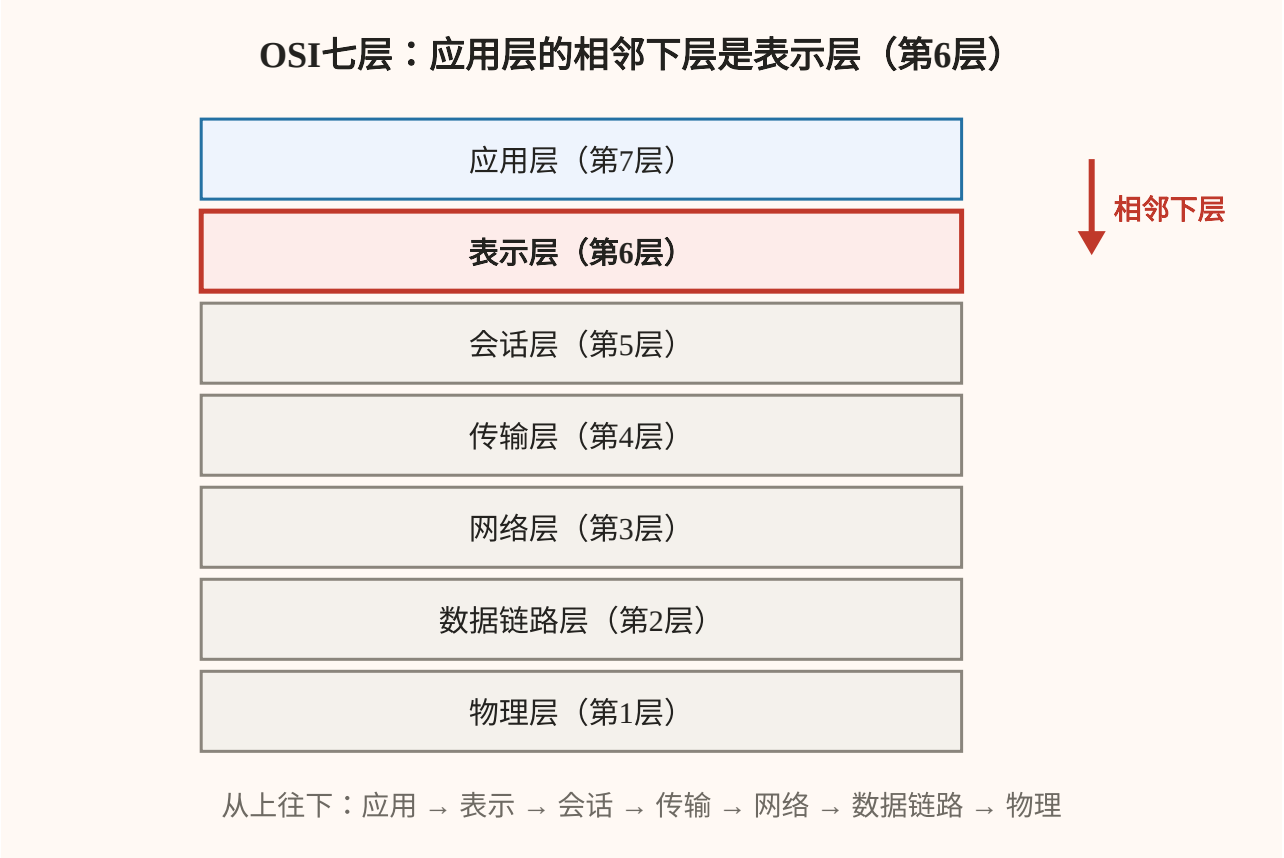
正确选项为什么对： 应用层之下就是表示层（第 6 层），就是 B，选 B。

错误选项为什么错＋举例：

- 会话层的：会话层是表示层的下一层，不是应用层的下一层。教训：从上往下数，应用层下面先是表示层。
- 传输层的：传输层隔了两层。教训：数层不能跳，表示层和会话层都在传输层上面。
- 物理层或者数据链路层的：那是下面几层，离应用层更远。教训：相邻下层就是紧挨着的那一层。

笔记本一句话： 第 33 题＋OSI 应用层之下是表示层（第 6 层），管格式、压缩、加密解密。

【图像】



OSI七层从上往下排列，应用层之下第一层是表示层（第6层）的示意图

【定位】 计算机网络 · 第1章 计算机网络体系结构 · OSI参考模型七层顺序

【易混对比】

- "应用层"和"表示层"的区别：应用层直接给用户用（浏览器、邮件客户端都在这一层），表示层只管数据的"长相"——格式转换、压缩、加密解密，不直接面对用户。
- "表示层"和"会话层"的区别：表示层是应用层的相邻下层（第6层），会话层还在表示层之下（第5层）；数层时表示层先到，不能跳过它去答会话层。

应用层的相邻下层是表示层（第6层），对应选项 B，选 B。

【坑位】

问的是"相邻下层"，数到应用层之下第一层就停，别顺手数到会话层。

会话层（对话管理）是表示层的下一层，不是应用层的下一层；错选 A 的人都是多往下数了一层。正确做法：从上往下背出七层顺序——应用、表示、会话、传输、网络、数据链路、物理，应用层之下第一层就是表示层，停。

【类比】

像说话的人和翻译：应用层是说话的人，表示层是翻译；话说完先由翻译变成对方听得懂的样子，再往下传。术语贴回去：说话的人就是"应用层"，翻译就是"表示层"，翻译的活（格式转换、压缩、加密解密）就是"数据表示"。

【口诀】 七层从上往下数，应用下面是表示。

【考频】 「OSI七层」在 2009–2022 年解析中出现 8 次 · 中频

第 34 题 · 10Base-T 曼彻斯特编码对应的比特串（正确答案：A）

题干与选项：

若下图为 10BaseT 网卡接收到的信号波形，则该网卡收到的比特串是（ ）。

> 注：信号波形图以原卷为准。

- A. 0011 0110
- B. 1010 1101
- C. 0101 0010
- D. 1100 0101

入手点： 曼彻斯特编码的规矩是“一个码元分两半”；先看每半段的电平高低，再按约定翻译成 0 或者 1。

专有名词从零解释：

- 先想“10Base-T”：10Mbps 的以太网，用双绞线传数据。
- 再想“曼彻斯特编码”：每位码元分成前后两个相等的间隔，前半段和后半段的电平总是相反的；每位中间都有一次跳变。
- 再想“约定”：前半段低电平、后半段高电平表示 1，前半段高电平、后半段低电平表示 0；也可以反过来规定，本题按这个规定读。
- 再想“自带时钟”：因为每位中间都跳变，接收方能从跳变里恢复出时钟，不用另外传时钟信号。

考查知识点：

- 先想翻译办法：把波形按码元切开，每个码元看前半段和后半段的电平组合；前低后高读成 1，前高后低读成 0。
- 再想本题的串：按这个规定，题目给的电平波形对应的比特串就是 A。
- 再想约定反过来的情形：如果规定反过来（前高后低为 1），同一段波形读出来的就是互补的串；所以曼彻斯特编码的读法依赖约定，不能只看波形不看规定。
- 拿生活例子验算：曼彻斯特编码像“每句话说两遍，一遍正着说、一遍反着说”，听的人从“正反组合”里判断这句话是肯定还是否定，而不是只听半句。
- 边界情况：差分曼彻斯特是另一套规矩——看每位开头跳不跳变，不是看前后半段的高低；本题是曼彻斯特编码，不是差分曼彻斯特。
- 易混对子：不要把电平高低直接当成 0、1——曼彻斯特看的是“前半段和后半段的组合”，不是单个电平；不要忘了两种约定——正反规定读出互补的串，约定要先定。

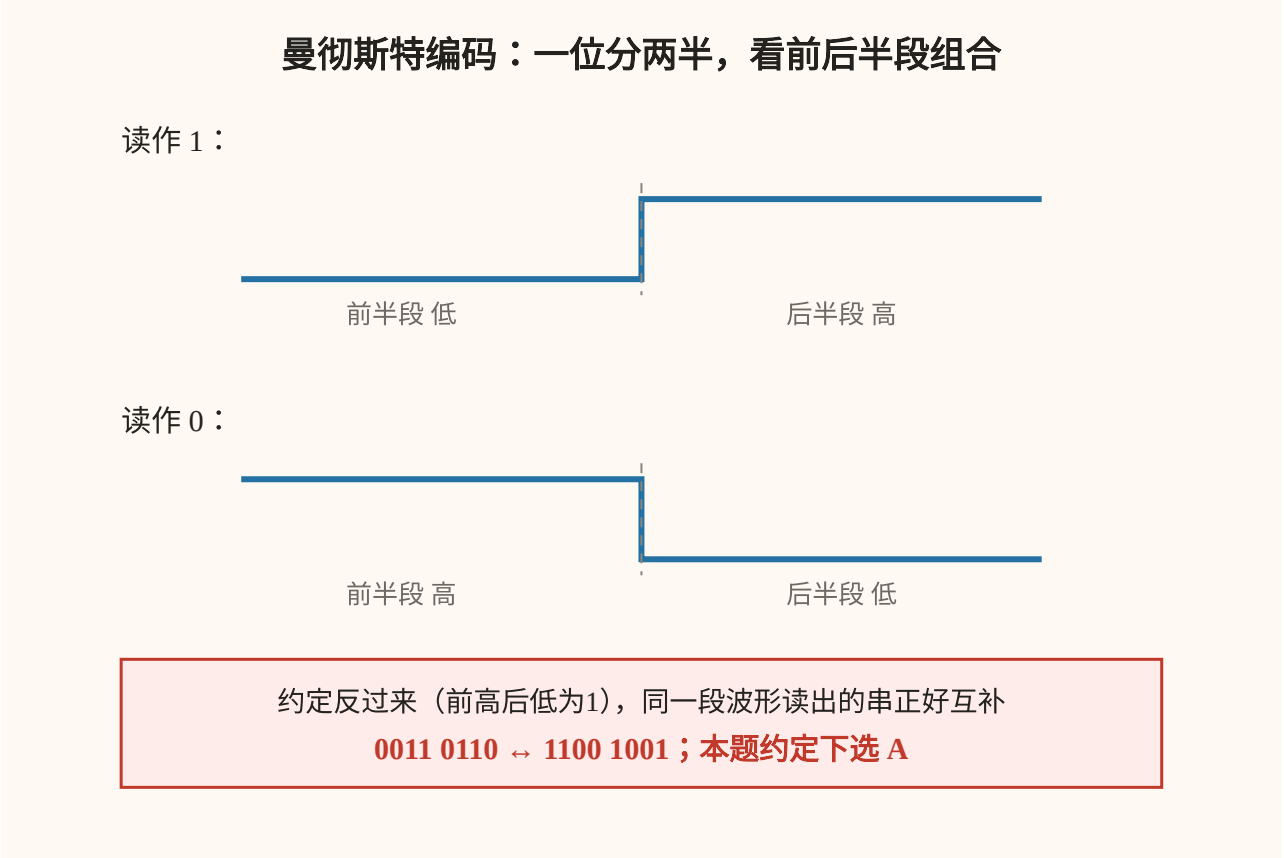
正确选项为什么对： 按“前低后高为 1”的规定，波形对应的比特串就是 A，选 A。

错误选项为什么错＋举例：

- 按“高电平＝1、低电平＝0”直接读的：那是单极性码，不是曼彻斯特编码。教训：曼彻斯特看前后半段的组合。
- 约定反了读出的互补串：规定反了，读出来的串正好取反。教训：约定要先定，再读串。
- 把差分曼彻斯特的规矩套上来的：差分看开头跳不跳变，本题不是差分。教训：先分清是哪种曼彻斯特。

笔记本一句话： 第 34 题＋曼彻斯特编码：一位分两半，前低后高为 1（约定要先定），本题对应 A。

【图像】



曼彻斯特编码规则：一个码元分两半，前低后高读1、前高后低读0，两种约定读出互补串的示意图

【定位】 计算机网络 · 第2章 物理层 · 编码与调制（曼彻斯特编码）

【易混对比】

- "曼彻斯特编码"和"差分曼彻斯特编码"的区别：曼彻斯特看每个码元前后半段的电平组合，差分看每个码元开头跳不跳变；本题是前者。
- "前低后高为1"和"前高后低为1"的区别：两种约定互相取反，同一段波形读出的比特串正好互补（0011 0110 和 1100 1001）；约定要先定，再读串。

按"前低后高为1"的约定，波形读出的比特串是 0011 0110，选 A。

【坑位】

别把单个电平的高低直接当成0、1，也别在两种约定之间来回横跳。

曼彻斯特编码看的是"前半段＋后半段"的组合，不是单个电平；按高电平＝1直接读出来的是单极性码，读法就错了。约定反过来读出的串是互补的 1100 1001，读串之前先把约定钉死。本题题面给的约定下答案是 A。

【套路】

- 第一步，钉死约定：前半段低电平、后半段高电平读作1，反之读作0。
- 第二步，按码元切开：把波形按每位码元切成前后两半。
- 第三步，逐位翻译：每位看前后半段的组合，翻成0或1，连成串。

先定约定，再按"前后半段组合"逐位翻译。

【类比】

像每句话说两遍——一遍正着说、一遍反着说，听的人从"正反组合"里判断这句话是肯定还是否定，而不是只听半句。术语贴回去：说两遍就是"一个码元分前后两半"，正反组合就是"前半段和后半段的电平组合"，肯定或否定就是"比特1或0"。

【口诀】 曼彻斯特分两半，前低后高读作一；约定反了串互补。

【考频】 「曼彻斯特编码」在 2009–2022 年解析中出现 3 次 · 中频

第 35 题 · 报文交换与分组交换的时延比较（正确答案：D）

题干与选项：

主机甲通过 1 个路由器（存储转发方式）与主机乙互联，两段链路的数据传输速率均为 10 Mbps，主机甲分别采用报文交换和分组大小为 10 kb 的分组交换向主机乙发送 1 个大小为 8 Mb（1M=106）的报文。.若忽略链路传播延迟、分组头开销和分组拆装时间，则两种交 换方式完成该报文传输所需的总时间分别为

- A. 800 ms、1 600 ms
- B. 801 ms、1 600 ms
- C. 1 600 ms、800 ms
- D. 1 600 ms、801 ms

入手点： 两种交换各算一遍总时延：报文交换是“整段发两次”，分组交换是“流水式转发，只多一个分组的时间”。

专有名词从零解释：

- 先想“报文交换”：整个报文一次发完；中间结点（路由器）把整个报文存下来，再转发给下一段。
- 再想“分组交换”：报文切成小分组；路由器收到第一个分组就开始转发，不用等整个报文收完，后面的分组流水式跟进。
- 再想“时延”：数据从甲到乙一共花的时间；本题只算发送时延（数据量除以链路速率），不算传播时延这些零头。

考查知识点：

- 先想报文交换：不分组，发送一个报文的时延＝8Mb÷10Mbps＝800ms；主机甲发送报文要 800ms，报文到达路由器、路由器存下来再转发又要 800ms，总时延＝800ms×2＝1600ms。
- 再想分组交换：每组 10kb，发送一个分组的时延＝10kb÷10Mbps＝1ms；一共 8Mb÷10kb＝800 个分组；主机甲发送 800 个分组要 1ms×800＝800ms；路由器收到第一个分组就开始转发，后面的分组流水跟进，只多 1ms；总时延＝800ms＋1ms＝801ms。
- 再想结论：分组交换的总时延是 801ms，就是 D；分组交换比报文交换快（801ms 小于 1600ms）。
- 拿生活例子验算：报文交换像整车货到了中转站、卸完再装车发走，要等整车；分组交换像流水线，第一个箱子一到中转站就发走，后面的箱子跟着走，只多一个箱子的时间。
- 边界情况：分组数＝报文大小÷分组大小＝800；分组首部开销本题不计（题目给的数据就是这么算的）；如果中间结点更多段，分组流水的优势更大。
- 易混对子：不要给分组也算两次 800ms——路由器不用等全部收完，收到第一个分组就转发；不要给报文只算一次 800ms——中间结点存转发还要一次。

正确选项为什么对： 分组交换总时延＝800ms＋1ms＝801ms，就是 D，选 D。

错误选项为什么错＋举例：

- 1600ms 的：那是报文交换的总时延，不是分组的。教训：报文和分组的算法不一样，报文要乘 2，分组是流水。
- 800ms 的：漏了最后那 1ms 的流水尾巴。教训：路由器转发第一个分组也要 1ms，总时延是 800＋1。
- 只算发送时延、不算中间结点转发的：中间结点存转发或者流水转发都要算时间。教训：总时延是甲发送＋中间结点转发，两段都要算。

笔记本一句话： 第 35 题＋报文交换 1600ms，分组交换 801ms（流水只多 1ms）。

【图像】



报文交换整段发两次共1600ms、分组交换流水转发共801ms的时延对比示意图

【定位】 计算机网络 · 第1章 计算机网络体系结构 · 数据交换方式

【易混对比】

- "报文交换"和"分组交换"的区别：报文交换中间结点要把整个报文存下来再转发，总时延是"发送时延×段数"；分组交换路由器收到第一个分组就开始转发、后面流水跟进，总时延是"全部发送完＋多一个分组时间"。

报文交换总时延＝800ms×2＝1600ms，分组交换总时延＝800ms＋1ms＝801ms，对应选项 D，选 D。

【坑位】

分组交换别乘2，报文交换别只算一次；分组交换最后那1ms的流水尾巴别漏。

给分组交换也乘2的人，是把"路由器不用等整份报文、首个分组到就转"这条忘了；报文交换只算800ms的人，是漏了路由器存转发还要一次发送时延。正确做法：先算单次发送时延（8Mb÷10Mbps＝800ms；10kb÷10Mbps＝1ms），报文交换乘以段数2，分组交换加流水尾巴1ms。

【套路】

- 第一步，算单次发送时延：报文 8Mb÷10Mbps＝800ms，分组 10kb÷10Mbps＝1ms。
- 第二步，数分组个数：8Mb÷10kb＝800 个分组，甲发完要 800ms。
- 第三步，按交换方式收尾：报文交换×2＝1600ms；分组交换流水，只多 1ms＝801ms。

报文乘段数，分组加尾巴。

【类比】

报文交换像整车货到中转站、卸完再装车发走，要等整车；分组交换像流水线，第一个箱子一到中转站就发走，后面的箱子跟着走，只多一个箱子的时间。术语贴回去：整车货就是"整个报文"，中转站就是"路由器"，流水线的箱子就是"分组"。

【口诀】 报文交换乘段数，分组流水多一时。

【考频】 「分组交换」在 2009–2022 年解析中出现 3 次 · 中频

第 36 题 · 哪种信道划分协议在发送过程中可能发生冲突（正确答案：B）

题干与选项：

下列介质访问控制方法中，可能发生冲突的是

- A. CDMA
- B. CSMA
- C. TDMA
- D. FDMA

入手点： 先分两类：静态划分的不会冲突，动态争用的可能冲突；再看四个协议各属于哪类。

专有名词从零解释：

- 先想“信道划分协议”：把一条信道静态地分给多个用户用的办法；分好之后，各用各的那份，互不干扰。
- 再想“FDMA”：频分多址；把频段切成几块，每个用户用一块频段。
- 再想“TDMA”：时分多址；把时间切成时隙，每个用户轮流用一个时隙。
- 再想“CDMA”：码分多址；每个用户用不同的码字，信号叠加在一起，靠码字分开。
- 再想“CSMA”：载波侦听多路访问；发送前先听信道空不空，空闲才发送；发送过程中如果别人也同时发送，就会冲突。

考查知识点：

- 先想 A（CDMA）：码分是静态划分，每个用户有自己的码字，互不干扰，不会冲突。
- 再想 C（TDMA）：时分是静态划分，每个用户有自己的时隙，互不干扰，不会冲突。
- 再想 D（FDMA）：频分是静态划分，每个用户有自己的频段，互不干扰，不会冲突。
- 再想 B（CSMA）：CSMA 是动态争用；发送前先侦听，信道空闲才发送；但两个结点可能同时侦听到空闲、同时发送，发送过程中就会发生冲突。这就是 CSMA/CD 里“碰撞”的原因。
- 拿生活例子验算：静态划分像每人分一个固定车道，各走各的，不会撞；CSMA 像过路口前先左右看看，没车才走，但两个人同时看到没车、同时起步，还是会撞。
- 边界情况：CSMA 分坚持、非坚持、p 坚持几种，侦听策略不同，但“发送过程中可能冲突”这一点不变；静态划分不会冲突，但信道利用率低——没人用的时候，那份频段或者时隙也空着。
- 易混对子：不要以为 CSMA 侦听了就不会冲突——侦听只能降低冲突概率，信号传播有延迟，同时侦听到空闲就撞了；不要把 CDMA 当成争用——CDMA 是码分，静态的。

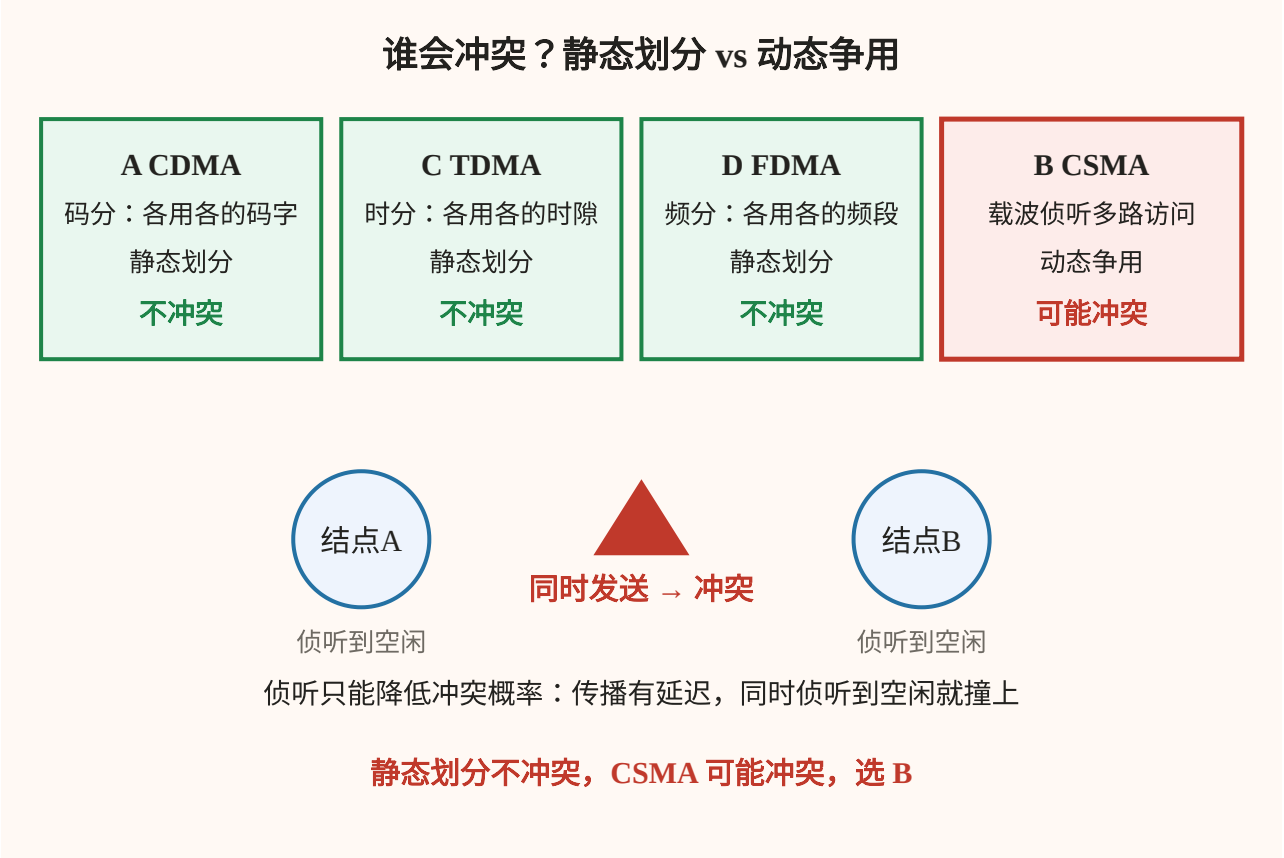
正确选项为什么对： CSMA 先听后发，发送过程中仍可能冲突，就是 B，选 B。

错误选项为什么错＋举例：

- A（CDMA）：码分静态划分，不冲突。教训：码分是各用各的码字，不是争用。
- C（TDMA）：时分静态划分，不冲突。教训：时分是各用各的时隙，不是争用。
- D（FDMA）：频分静态划分，不冲突。教训：频分是各用各的频段，不是争用。

笔记本一句话： 第 36 题＋静态划分（CDMA／TDMA／FDMA）不冲突；CSMA 先听后发，发送中仍可能冲突。

【图像】



CDMA、TDMA、FDMA静态划分不冲突，CSMA先听后发仍可能冲突的对比示意图

【定位】 计算机网络 · 第4章 数据链路层 · 介质访问控制

【易混对比】

- "CSMA"和"CDMA／TDMA／FDMA"的区别：后三者是静态划分信道——各用各的码字、时隙、频段，互不干扰；CSMA 是动态争用——先听后发，但两个结点可能同时侦听到空闲、同时发送，发送过程中照样冲突。

静态划分的 A、C、D 不会冲突，CSMA 发送过程中可能冲突，选 B。

【坑位】

别以为"先侦听"等于"不会冲突"。

侦听只能降低冲突概率：信号在信道上传播要时间，两个结点可能同时侦听到信道空闲、同时发送，撞上就是 CSMA/CD 里说的碰撞。错选静态划分的人，是把"划分"和"争用"两类搞混了。正确做法：先分两类——静态划分的不会冲突，动态争用的可能冲突，再看四个选项各属哪类。

【类比】

静态划分像每人分一条固定车道，各走各的，撞不上；CSMA 像过路口前先左右看看、没车才走，但两个人同时看到没车、同时起步，还是会撞。术语贴回去：固定车道就是"静态信道划分"，路口先看就是"载波侦听"，同时起步撞上就是"冲突"。

【口诀】 频分时分码分稳，载波侦听会撞车。

【考频】 「CSMA」在 2009–2022 年解析中出现 7 次 · 中频

第 37 题 · HDLC 协议的零比特填充结果（正确答案：A）

题干与选项：

HDLC 协议对 01111100 01111110 组帧后对应的比特串为

- A. 01111100 00111110 10
- B. 01111100 01111101 01111110
- C. 01111100 01111101 0
- D. 01111100 01111110 01111101

入手点： 数 1 的个数，满 5 个连续的 1 就填 1 个 0；填完再数一遍，确认新增的 0 都在对的位置上。

专有名词从零解释：

- 先想“HDLC”：高级数据链路控制协议；一种面向比特的链路层协议。
- 再想“帧标志”：HDLC 数据帧首尾的标志，是位模式 01111110；接收方靠这个标志认出“帧从这里开始、到那里结束”。
- 再想“零比特填充”：数据里出现 5 个连续的 1 时，就在后面插入 1 个 0；这样数据里就不会出现 6 个连续的 1，不会和帧标志混淆。
- 再想“去填充”：接收方看到 5 个 1 后面跟一个 0，就把这个 0 删掉，还原数据。

考查知识点：

- 先想规矩：HDLC 数据帧以 01111110 标识开始和结束；帧数据里凡是出现 5 个连续的 1，输出的位流里就在后面填充 1 个 0。
- 再想本题的填充：新增的 0 一共 2 个，在全串的第 7 位、第 16 位。
- 再想填充后的串：01111100 00111110 10，就是 A。
- 再想接收方怎么还原：看到 5 个 1 后面跟 0，就把 0 删掉；看到 5 个 1 后面跟 1、再跟 0，就是帧标志 01111110，不是数据。
- 拿生活例子验算：帧标志像“起止符”，数据里不能出现和起止符一样的串；零比特填充像“数据里快要撞上起止符时，插一个标记隔开”，接收方看到标记就知道“这里不是起止符”。
- 边界情况：填 0 只针对数据部分，帧标志本身不填；5 个 1 后面跟 1、再跟 0 是标志，不是数据，不能去填充。
- 易混对子：不要数到 6 个 1 才填——规矩是 5 个；不要把标志里的 5 个 1 也填了——标志不填；不要把新增 0 的位置数错——本题是第 7 位和第 16 位。

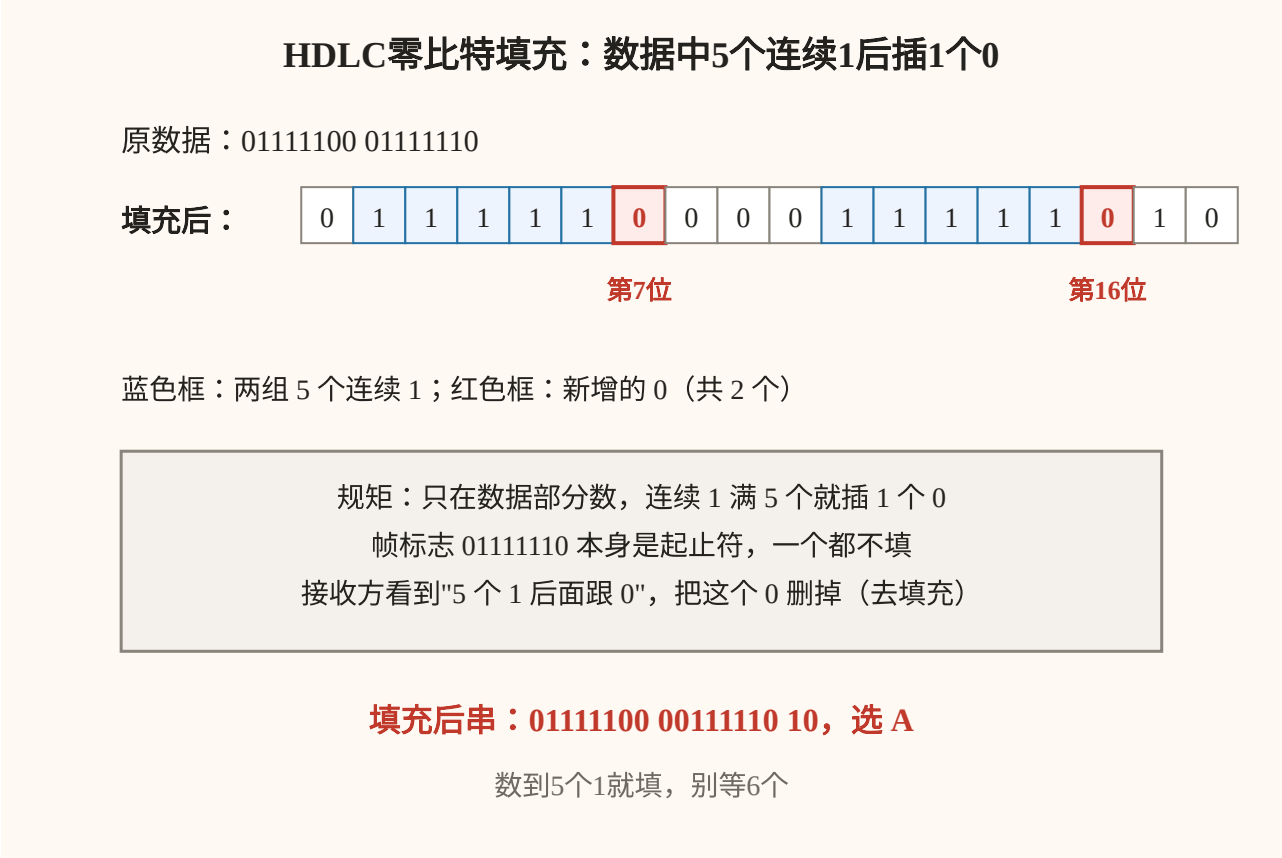
正确选项为什么对： 5 个连续 1 后填 1 个 0，新增 0 在第 7 位、第 16 位，填充后串是 01111100 00111110 10，就是 A，选 A。

错误选项为什么错＋举例：

- 新增 0 个数不对的（比如只填 1 个）：数 1 的个数数错了，漏了一处。教训：从头数到尾，满 5 个 1 就填 1 个 0，一处都不能漏。
- 新增 0 位置不对的：第 7 位、第 16 位没对上。教训：填完再数一遍位置，确认新增的 0 都在对的位置上。
- 把标志也填了的：帧标志 01111110 不填。教训：填 0 只针对数据部分。

笔记本一句话： 第 37 题＋HDLC 零比特填充：5 个连续 1 后填 1 个 0，本题新增 0 在第 7 位、第 16 位。

【图像】



HDLC零比特填充：数据中出现5个连续1就插入1个0，新增0在全串第7位和第16位的示意图

【定位】 计算机网络 · 第4章 数据链路层 · HDLC协议

【易混对比】

- "零比特填充"和"字符填充"的区别：零比特填充是"数1填0"（面向比特的 HDLC 用），字符填充是"加转义字符隔开"（面向字符的协议用）；本题是前者。
- "数据里的5个1"和"帧标志里的5个1"的区别：数据里的 5 个连续 1 后面要填 0，帧标志 01111110 本身是起止符，一个都不填。

5 个连续 1 后填 1 个 0，新增的 0 共 2 个、在全串第 7 位和第 16 位，填充后串为 01111100 00111110 10，选 A。

【坑位】

数到5个1就填，别等6个；帧标志本身一个都不填。

等到 6 个 1 才填的人，是把"防 6 个连 1 撞上标志"的结果当成了触发条件——规矩是 5 个就填。把标志里的 5 个 1 也填了的人，是没分清数据和标志。正确做法：只在数据部分从头数，连续 1 满 5 就插 1 个 0，填完再数一遍新增 0 的位置（本题第 7 位、第 16 位）。

【套路】

- 第一步，从头数连续1：只数数据部分，帧标志跳过。
- 第二步，满5插1个0：连续 1 一到 5 个，立刻在后面插 1 个 0，继续往后数。
- 第三步，数位置核对：确认新增 0 的个数和位置（本题 2 个，第 7 位、第 16 位）。

数据里五连一就插零，插完数位再核对。

【类比】

帧标志像一对起止符，数据里不能出现和起止符一样的串；零比特填充像"数据快要撞上起止符时，插一个标记隔开"，接收方看到"5 个 1 后面跟 0"就知道这里不是起止符、把 0 删掉。术语贴回去：起止符就是"帧标志 01111110"，插标记就是"零比特填充"，删标记就是"去填充"。

【口诀】 数据五连一后添个零，标志本身不填充。

【考频】 「零比特填充」在 2009–2022 年解析中出现 1 次 · 低频

第 38 题 · 直通交换方式的最短传输延迟（正确答案：B）

题干与选项：

对于 100Mbps 的以太网交换机，当输出端口无排队，以直通交换（cut-through switching） 方式转发一个以太网帧（不包括前导码）时，引入的转发延迟至少是

- A. 0 μs
- B. 0.48 μs
- C. 5.12 μs
- D. 121.44 μs

入手点： 直通交换只看帧头，不等整个帧；先想它要看几个字节，再用“字节数除以速率”算延迟。

专有名词从零解释：

- 先想“直通交换”：交换机读到帧的目的地址，就开始转发；不用等整个帧收完。
- 再想“存储转发”：交换机把整个帧收完、校验完，再转发；和直通交换相对。
- 再想“目的地址”：帧头里记着“这个帧要发给谁”的地址；以太网里是 6 个字节。
- 再想“传输延迟”：把这段数据发到链路上要花的时间，等于数据量除以链路速率。

考查知识点：

- 先想直通交换看多少：直通交换只检查帧的目的地址，一共 6B。
- 再想算延迟：6B＝6×8＝48bit；链路速率是 100Mbps；延迟＝48÷100M＝0.48μs，就是 B。
- 再想为什么是最短：直通交换不等整帧，只等目的地址这 6 个字节；这是直通方式下最短的延迟。
- 拿生活例子验算：直通交换像快递分拣员只看包裹上的收件人名字就转发，不用等整车货到齐；存储转发像等整车货到齐、验完货再发。
- 边界情况：直通交换不校验帧，坏帧也会被转发，这是直通方式的代价；存储转发的延迟是整帧长度除以速率，再加上处理时间。
- 易混对子：不要用整帧长度算——那是存储转发；不要把 6B 当成 6bit——字节和比特差 8 倍；不要把单位算错——Mbps 是每秒百万比特，不是每秒兆字节。

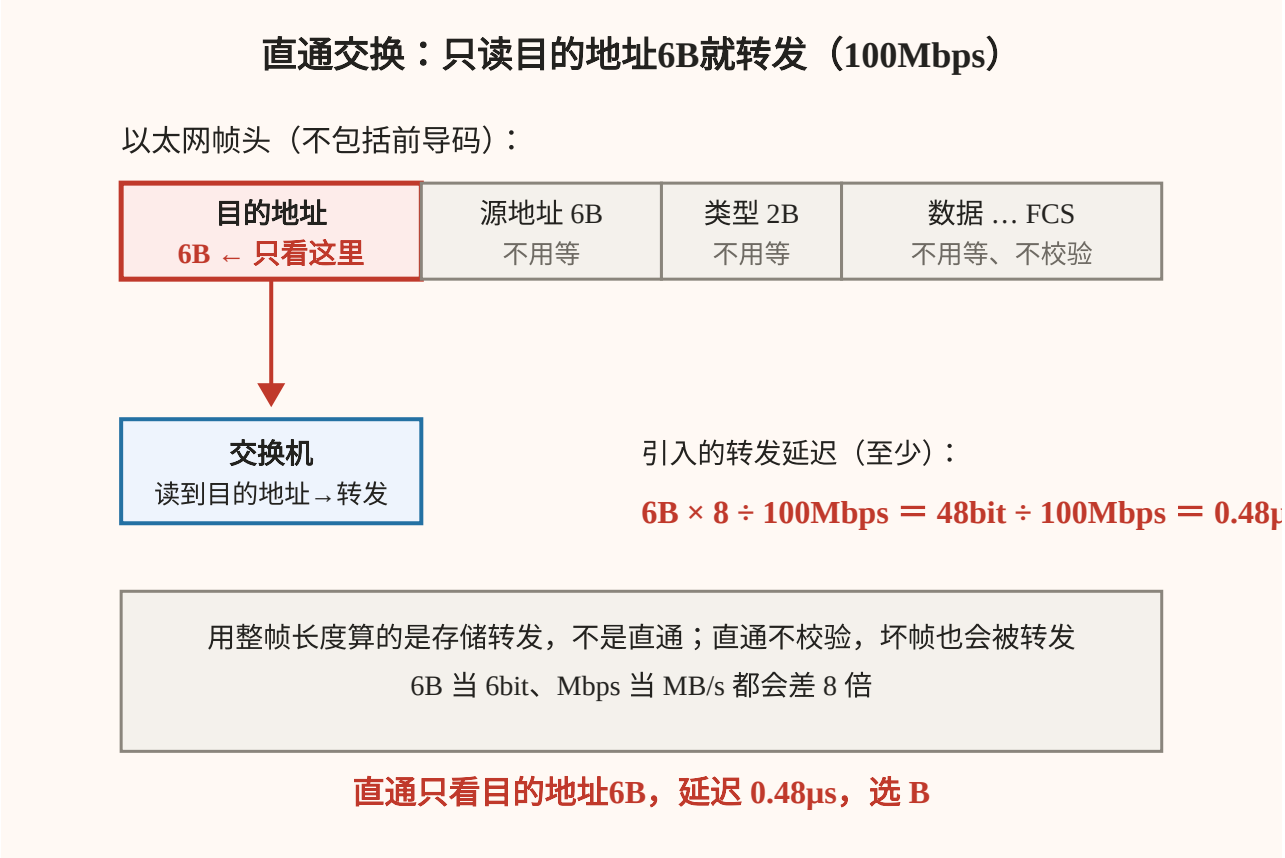
正确选项为什么对： 直通只看目的地址 6B，延迟＝6×8÷100Mbps＝0.48μs，就是 B，选 B。

错误选项为什么错＋举例：

- 用整帧长度算的：那是存储转发，不是直通。教训：直通只等目的地址。
- 把 6B 当成 6bit 算的：字节和比特没换算，差了 8 倍。教训：1B＝8bit，先换算再除。
- 单位算错的：Mbps 当成了 MB／s。教训：Mbps 是每秒百万比特，MB／s 是每秒兆字节，两者差 8 倍。

笔记本一句话： 第 38 题＋直通交换只看目的地址 6B，延迟＝6×8÷100Mbps＝0.48μs。

【图像】



直通交换只读帧头目的地址6字节就转发，延迟等于48比特除以100Mbps的示意图

【定位】 计算机网络 · 第4章 数据链路层 · 以太网交换机

【易混对比】

- "直通交换"和"存储转发"的区别：直通只读到目的地址（6B）就开始转发，不等整帧、不校验；存储转发要把整帧收完、校验完再转发，延迟按整帧长度算。

直通只看目的地址 6B，延迟＝6×8÷100Mbps＝0.48μs，选 B。

【坑位】

别用整帧长度算，别把6B当成6bit，单位别把Mbps看成MB/s。

用整帧长度算的是存储转发，不是直通；6B 当 6bit 算会差 8 倍；Mbps 是每秒兆比特，MB/s 是每秒兆字节，两者也差 8 倍。正确做法：先确认直通只看 6B 目的地址，字节先×8 换成比特（48bit），再除以速率（100Mbps）。

【套路】

- 第一步，确认看多少字节：直通交换只等目的地址，6B。
- 第二步，字节换比特：6×8＝48bit。
- 第三步，除以速率：48÷100Mbps＝0.48μs。

直通看六字节，乘八除速率。

【类比】

直通交换像快递分拣员只看包裹上的收件人名字就转发，不用等整车货到齐；存储转发像等整车货到齐、验完货再发。术语贴回去：收件人名字就是"目的地址"，整车货就是"整个帧"，验货就是"校验"。

【口诀】 直通只看六字节，乘八除速得延迟。

【考频】 「直通交换」在 2009–2022 年解析中出现 14 次 · 高频

第 39 题 · TCP 报文段的序号与确认号计算（正确答案：B）

题干与选项：

主机甲与主机乙之间已建立一个 TCP 连接，双方持续有数据传输，且数据无差错与丢 失。若甲收到 1 个来自乙的 TCP 段，该段的序号为 1913、确认序号为 2046、有效载荷为 100 字节，则甲立即发送给乙的 TCP 段的序号和确认序号分别是

- A. 2046、2012
- B. 2046、2013
- C. 2047、2012
- D. 2047、2013 ×

入手点： 序号和确认号各指“下一个字节”：序号是“我发的第一个字节”，确认号是“我期望对方发的下一个字节”。

专有名词从零解释：

- 先想“序号”：本报文段所发送数据的第一个字节的序号；接收方靠序号把数据按顺序拼起来。
- 再想“确认号”：期望收到对方下一个报文段数据的第一个字节的序号；意思是“这个号之前的字节我都收到了”。
- 再想“有效载荷”：报文段里真正装的数据的字节数；序号就是按有效载荷的字节数往前走的。

考查知识点：

- 先想甲收到的段：序号 seq＝1913、确认号 ack＝2046、有效载荷 100 字节。
- 再想甲回段的确认号：乙的数据从 1913 起，100 个字节，到 1913＋100－1＝2012；甲已经收到了乙到 2012 的数据，期望乙的下一个字节是 2013；所以甲回段的确认号 ack1＝seq＋100＝1913＋100＝2013。
- 再想甲回段的序号：乙的确认号 ack＝2046 告诉甲“我已经收到了你到 2045 的数据，你下一个从 2046 发”；所以甲回段的序号 seq1＝ack＝2046。
- 再想结论：甲发给乙的段，序号是 2046、确认号是 2013，就是 B。
- 拿生活例子验算：确认号像“我已经看到第几页了，下一页从哪页开始”；序号像“我这封信从第几页开始写”；两者都是“下一个字节”，不是“最后一个字节”。
- 边界情况：确认号是“下一个期望字节”，不是“已收到的最后一个字节”，两者差 1；纯 ACK 段没有载荷，不耗序号；本题是数据段，按数据段算。
- 易混对子：不要把序号和确认号的位置写反——序号是“我发的”，确认号是“我期望对方发的”；不要把确认号写成 2012——2012 是已收到的最后一个字节，确认号是下一个 2013；不要把序号写成 1913——1913 是乙发来的段的序号，不是甲要发的。

正确选项为什么对： 回段序号＝对方确认号＝2046，确认号＝对方序号＋载荷＝2013，就是 B，选 B。

错误选项为什么错＋举例：

- 序号和确认号互换的：序号是“我发的第一个字节”，确认号是“我期望对方发的下一个字节”，两者不能互换。教训：先分清“发”和“收”。
- 确认号写成 2012 的：2012 是已收到的最后一个字节，确认号是下一个 2013。教训：确认号永远是“下一个”，不是“最后一个”。
- 序号写成 1913 的：1913 是乙发来的段的序号，不是甲要发的。教训：回段的序号取对方的确认号。

笔记本一句话： 第 39 题＋TCP 回段：序号＝对方确认号＝2046，确认号＝对方序号＋载荷＝2013。

【图像】



甲乙TCP往返：乙发来段序号1913确认号2046载荷100，甲回段序号取对方确认号2046、确认号取对方序号加载荷2013的示意图

【定位】 计算机网络 · 第5章 传输层 · TCP可靠传输

【易混对比】

- "序号"和"确认号"的区别：序号是"我这段发的第一个字节"，确认号是"我期望对方发的下一个字节"；一个管发，一个管收，不能互换。

回段序号＝对方确认号＝2046，确认号＝对方序号＋载荷＝1913＋100＝2013，选 B。

【坑位】

确认号是"下一个期望字节"，不是"已收到的最后一个字节"。

乙的数据从 1913 起、100 个字节，到 2012 结束；确认号要写 2013，写成 2012 的人是把"最后一个"当成了"下一个"，差 1。序号写成 1913 的人，是把乙发来的段的序号当成了甲要发的。正确做法：回段序号照抄对方确认号（2046），确认号＝对方序号＋对方载荷（1913＋100＝2013）。

【套路】

- 第一步，定回段序号：照抄对方段的确认号，seq1＝ack＝2046。
- 第二步，定回段确认号：对方序号＋对方载荷，ack1＝1913＋100＝2013。

序号照抄对方确认号，确认号等于对方序号加对方载荷。

【类比】

确认号像"我已经看到第几页了，下一页从哪页开始"；序号像"我这封信从第几页开始写"。术语贴回去：看到第几页就是"已收到的数据"，下一页从哪开始就是"确认号"，信从第几页开始写就是"序号"。

【口诀】 回段序号照抄确，确认等于序加荷。

【考频】 「TCP序号确认号」在 2009–2022 年解析中出现 5 次 · 中频

第 40 题 · SMTP 协议的用途与编码限制（正确答案：A）

题干与选项：

下列关于 SMTP 协议的叙述中，正确的是（ ）。

I. 只支持传输 7 比特 ASCII 码内容

II. 支持在邮件服务器之间发送邮件

III. 支持从用户代理向邮件服务器发送邮件

IV. 支持从邮件服务器向用户代理发送邮件

- A. 仅 I、II 和 III
- B. 仅 I、II 和 IV
- C. 仅 I、III 和 IV
- D. 仅 II、III 和 IV

入手点： SMTP 只管“发出去”，不管“收下来”；先把“发”和“收”分开，再一条一条验。

专有名词从零解释：

- 先想“SMTP”：简单邮件传输协议；负责把邮件发送出去的协议。
- 再想“POP3”：邮局协议；负责从邮件服务器上把邮件取回来的协议。
- 再想“用户代理”：你用的邮件客户端（比如手机上的邮件 App、电脑上的邮件软件）。
- 再想“邮件服务器”：替你收发、暂存邮件的服务器。

考查知识点：

- 先想 Ⅰ（只支持 7 比特 ASCII）：SMTP 只支持传输 7 比特的 ASCII 码内容；多媒体内容要靠 MIME 编码成 ASCII 再传。Ⅰ正确。
- 再想 II（支持服务器之间发送）：邮件服务器之间传邮件，用 SMTP。II 正确。
- 再想 III（支持用户代理→服务器发送）：用户代理把邮件发给自己的邮件服务器，用 SMTP。III正确。
- 再想 IV（支持服务器→用户代理发送）：服务器→用户代理是“取邮件”，这是 POP3（或者 IMAP）的活，不是 SMTP 的。IV错误。
- 再想结论：正确的只有 Ⅰ、II、III，组合就是 A。
- 拿生活例子验算：SMTP 像“寄信的邮局”，只管把信寄出去；POP3 像“去邮局取信”，只管把信取回来；寄和取是两套活，不能混为一谈。
- 边界情况：SMTP 的端口是 25 号（发送），POP3 的端口是 110 号（接收）；MIME 是扩展，不改变“SMTP 本身只支持 7 比特 ASCII”这个说法。
- 易混对子：不要以为收邮件也用 SMTP——收用 POP3 或者 IMAP；不要以为 SMTP 能直接传图片视频——要 MIME 转成 ASCII 才能传。

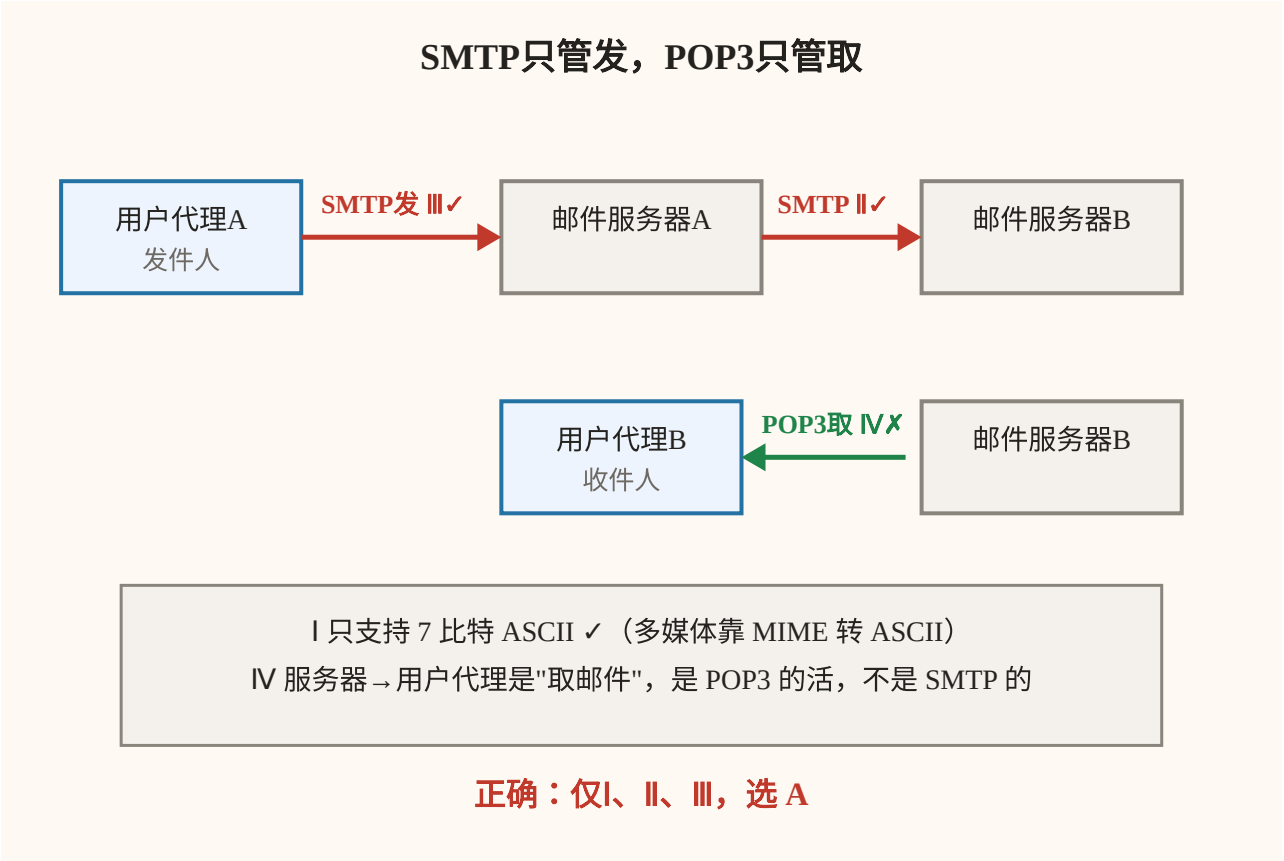
正确选项为什么对： 正确的只有 Ⅰ、II、III，IV 是 POP3 的活，组合就是 A，选 A。

错误选项为什么错＋举例：

- B（仅 Ⅰ、II、IV）：IV是 POP3 的活，不是 SMTP 的。教训：收邮件不归 SMTP 管。
- C（仅 Ⅰ、III、IV）：IV错了，还漏了 II。教训：服务器之间传邮件也是 SMTP。
- D（仅 II、III、IV）：IV错了，还漏了 Ⅰ。教训：SMTP 只支持 7 比特 ASCII 也是正确的说法。

笔记本一句话： 第 40 题＋SMTP 只管发：7 比特 ASCII、服务器之间、用户代理→服务器；收邮件是 POP3 的活。

【图像】



SMTP只管发送（用户代理到服务器、服务器之间），POP3只管从服务器取回邮件的邮件收发流程示意图

【定位】 计算机网络 · 第6章 应用层 · 电子邮件

【易混对比】

- "SMTP"和"POP3"的区别：SMTP 只管把邮件"发出去"（用户代理→邮件服务器、邮件服务器之间），POP3 只管把邮件"取回来"（邮件服务器→用户代理）；发和取是两套活。

正确的只有 Ⅰ、Ⅱ、Ⅲ；Ⅳ（服务器→用户代理）是 POP3 的活，不是 SMTP 的，组合对应 A，选 A。

【坑位】

别把"收邮件"算到SMTP头上，也别漏了 Ⅰ。

看到"服务器→用户代理"就勾选的人，是把"发"和"取"混了——取邮件用 POP3（或 IMAP）。Ⅰ（只支持 7 比特 ASCII）也是正确的，多媒体要靠 MIME 转成 ASCII 再传，漏选的人常栽在这里。正确做法：一条一条验——Ⅰ对、Ⅱ对、Ⅲ对、Ⅳ是 POP3 的活，只有 Ⅰ、Ⅱ、Ⅲ。

【类比】

SMTP 像"寄信的邮局"，只管把信寄出去；POP3 像"去邮局取信"，只管把信取回来；寄和取是两套活，不能混为一谈。术语贴回去：寄信就是"SMTP 发送"，取信就是"POP3 接收"，邮局就是"邮件服务器"。

【口诀】 发信用SMTP，取信用POP3。

【考频】 「SMTP」在 2009–2022 年解析中出现 6 次 · 中频

附录 · 答案速查表

- 1–8：D、C、D、B、A、C、C、D
- 9–16：C、A、C、C、A、A、C、A
- 17–24：D、C、B、B、B、D、A、A
- 25–32：C、A、C、B、D、B、B、B
- 33–40：B、A、D、B、A、B、B、A